ADVANCED BIG DATA ANALYTICS · ASSIGNMENT I · OCTOBER 2026

Smart Energy Grid Analytics

Ingestion pipeline, chunk and compression experiments, continuous aggregates, dashboards and day-ahead forecast for 4,000 smart meters (1–28 September 2026)

Student	Student ID
TWAHIRWA Joshua	101396
UBANJE Albert	101402
MUTONI Denyse	101207

Code repository: **https://github.com/twahirwajoshua1/auca-smart-grid** (the same files are in the submitted ZIP)

Report date: 16 October 2026

All times are CAT (Africa/Kigali, UTC+2). All benchmarks were run on one machine (host name **Josh**).

How this report is organised. Subsections follow the assignment order (1.1 to 9.3). Each subsection gives the method, the measured results (tables), the screenshots of the executed commands (each showing the command, its result, the taskbar and the clock) and, where the assignment asks for it, our observations and explanations. **Observation** boxes state what was measured; **Explanation** boxes give our reasoning about the cause.

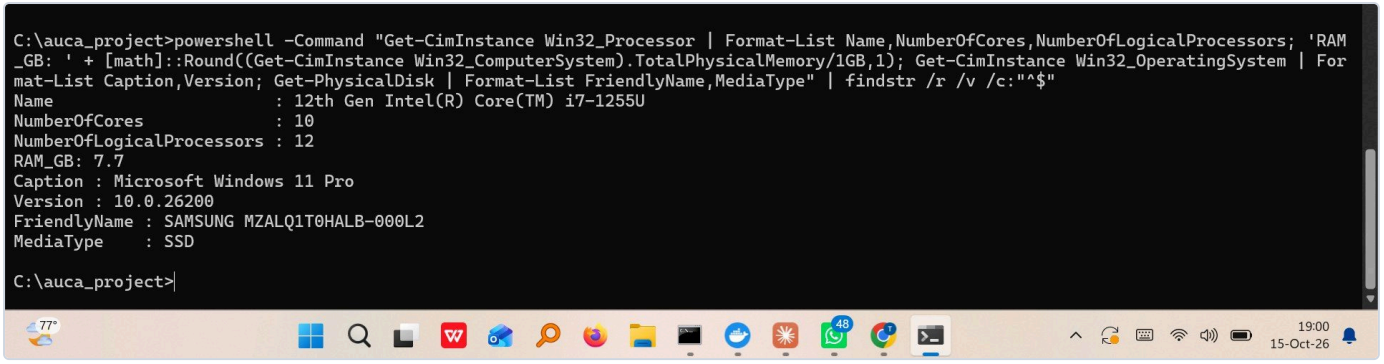
1 Build the ingestion pipeline

1.1 Confirm the environment

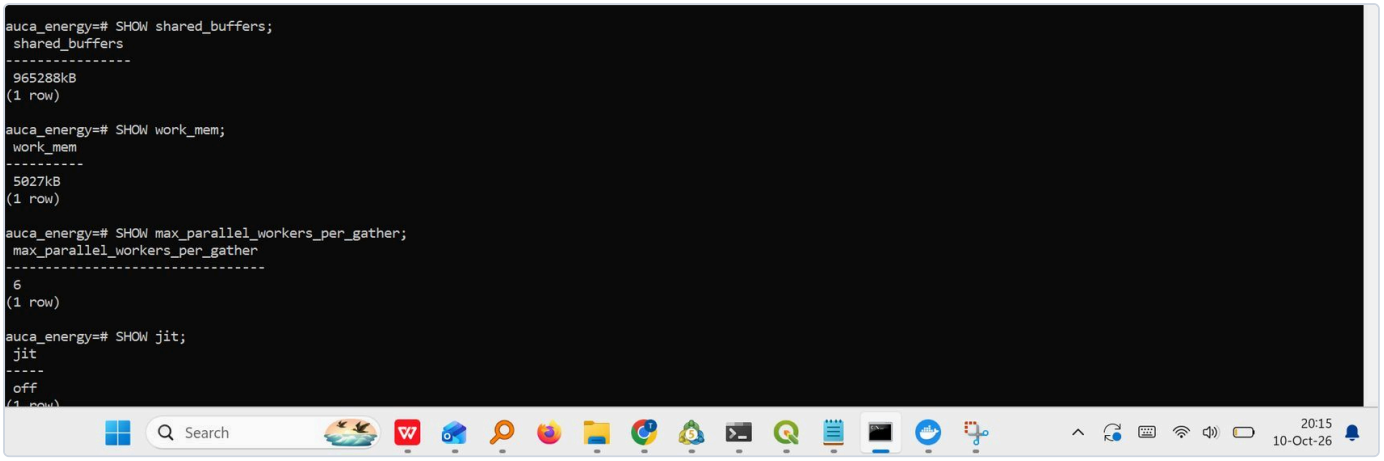
Services. EMQX runs in the Docker container `emqx` (MQTT mapped to host port 1884, dashboard on 18083). PostgreSQL with TimescaleDB runs in the container `iot-stack-db-1` (image `timescale/timescaledb:latest-pg17`, port 5432). We created one database, `auca_energy`, enabled the extension with `CREATE EXTENSION timescaledb` and set its time zone with `ALTER DATABASE auca_energy SET timezone TO 'Africa/Kigali'`. Every program also issues `SET timezone TO 'Africa/Kigali'` when it connects. The same Docker host also runs a Mosquitto container (`iot-stack-mqtt-1`, port 1883) from the compose stack; this assignment does not use it, which is why EMQX is mapped to host port 1884. Daily buckets are aligned to CAT midnight by filtering on `+02:00` timestamps and by using `time_bucket('1 day', event_time, 'Africa/Kigali')` in the continuous aggregates (Section 6).

Item	Recorded value
CPU	12th Gen Intel Core i7-1255U — 10 cores, 12 logical processors
RAM	7.7 GB installed. Docker's Linux VM reports 12 CPUs and MemTotal 3,953,799,168 bytes (about 3.7 GiB)
Operating system	Microsoft Windows 11 Pro, version 10.0.26200
Storage type	SSD (SAMSUNG MZALQ1T0HALB-000L2)
Python	3.13.14
EMQX	6.3.1 (node emqx@172.17.0.2)
PostgreSQL	17.11 (x86_64-pc-linux-musl, compiled by gcc Alpine 15.2.0, 64-bit)
TimescaleDB	2.30.2 (extension `timescaledb`, compatible with PostgreSQL 17)
shared_buffers	965288 kB (about 943 MiB)
work_mem	5027 kB
max_parallel_workers_per_gather	6
jit	off
How PostgreSQL runs	**In a Docker container** on the Windows host. The container has **no CPU or RAM limit** (HostConfig.Memory = 0, NanoCpus = 0), so it can use the 12 CPUs and about 3.7 GiB of memory that Docker's VM exposes.
Clock / time zone	Windows time zone “South Africa Standard Time” (UTC+2 = CAT). Windows Time service synchronised with time.windows.com (after a manual `w32tm /resync` on 16 October 2026 at 00:32 the status was Leap Indicator 0, last successful sync 00:32:16; a status check at 23:20 on 15 October had reported “not synchronized” with the last sync at 19:30:06). Execution times in this report are the taskbar clock in each screenshot and the CAT timestamps stored in the benchmark CSV files.

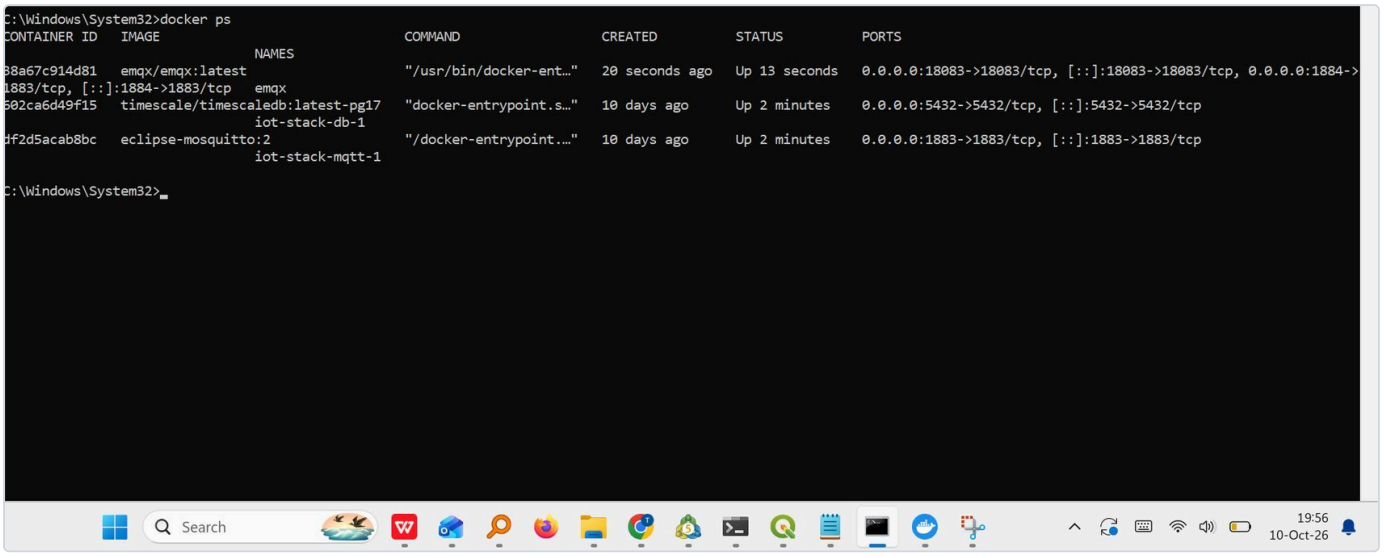
Screenshots: machine details, the four configuration values, the running containers (broker), the connection to the assignment database with the enabled extension and the session time zone.



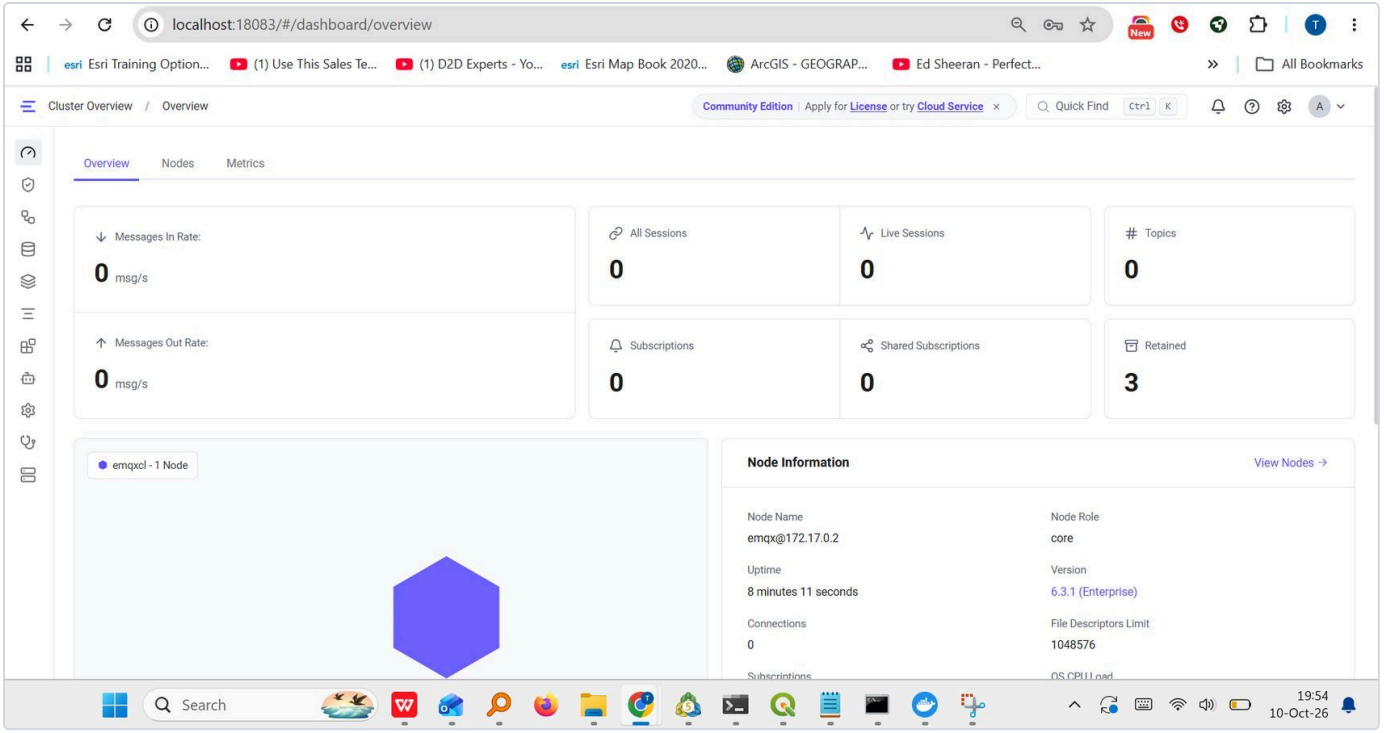
1.1 – hardware query with its output: CPU, cores, RAM, operating system and storage type (15 October 2026, 19:00)



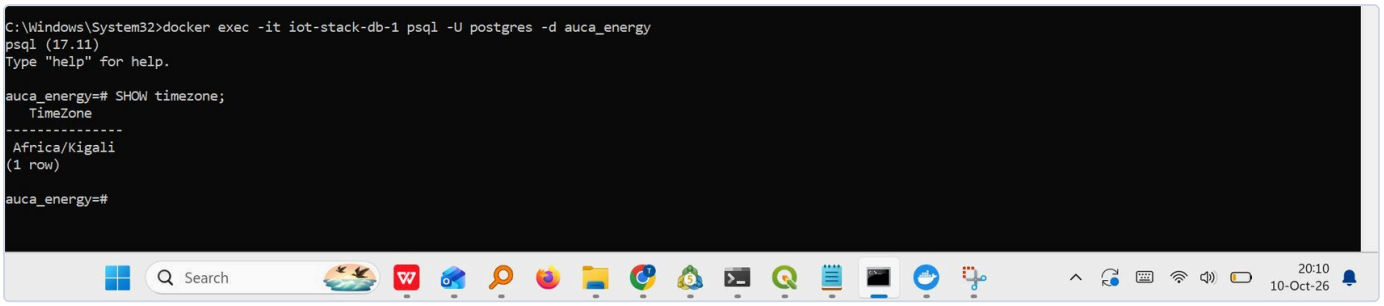
1.1 – shared_buffers, work_mem, max_parallel_workers_per_gather and jit



1.1 – running containers: emqx (broker) and iot-stack-db-1 (PostgreSQL + TimescaleDB)



1.1 – EMQX dashboard (running broker)



1.1 – connection to auca_energy, enabled extension and SHOW timezone

Recorded output for the versions, container limits and clock (text copy of the commands we ran in CMD and psql on 15 October 2026):

```
C:\auca_project>python --version
Python 3.13.14

C:\auca_project>docker exec iot-stack-db-1 psql -U postgres -d auca_energy -c "SELECT version();"
 PostgreSQL 17.11 on x86_64-pc-linux-musl, compiled by gcc (Alpine 15.2.0) 15.2.0, 64-bit

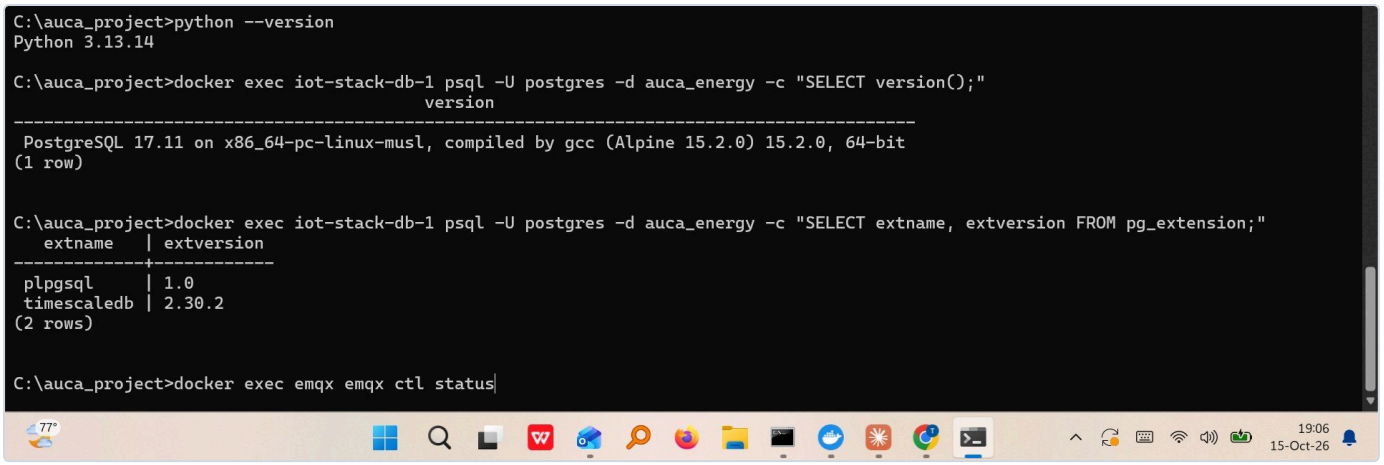
C:\auca_project>docker exec iot-stack-db-1 psql -U postgres -d auca_energy -c "SELECT extname, extversion FROM pg_extensi
on;"
   extname   | extversion
 plpgsql     | 1.0
 timescaledb | 2.30.2

C:\auca_project>docker exec emqx emqx ctl status
Node 'emqx@172.17.0.2' 6.3.1 is started

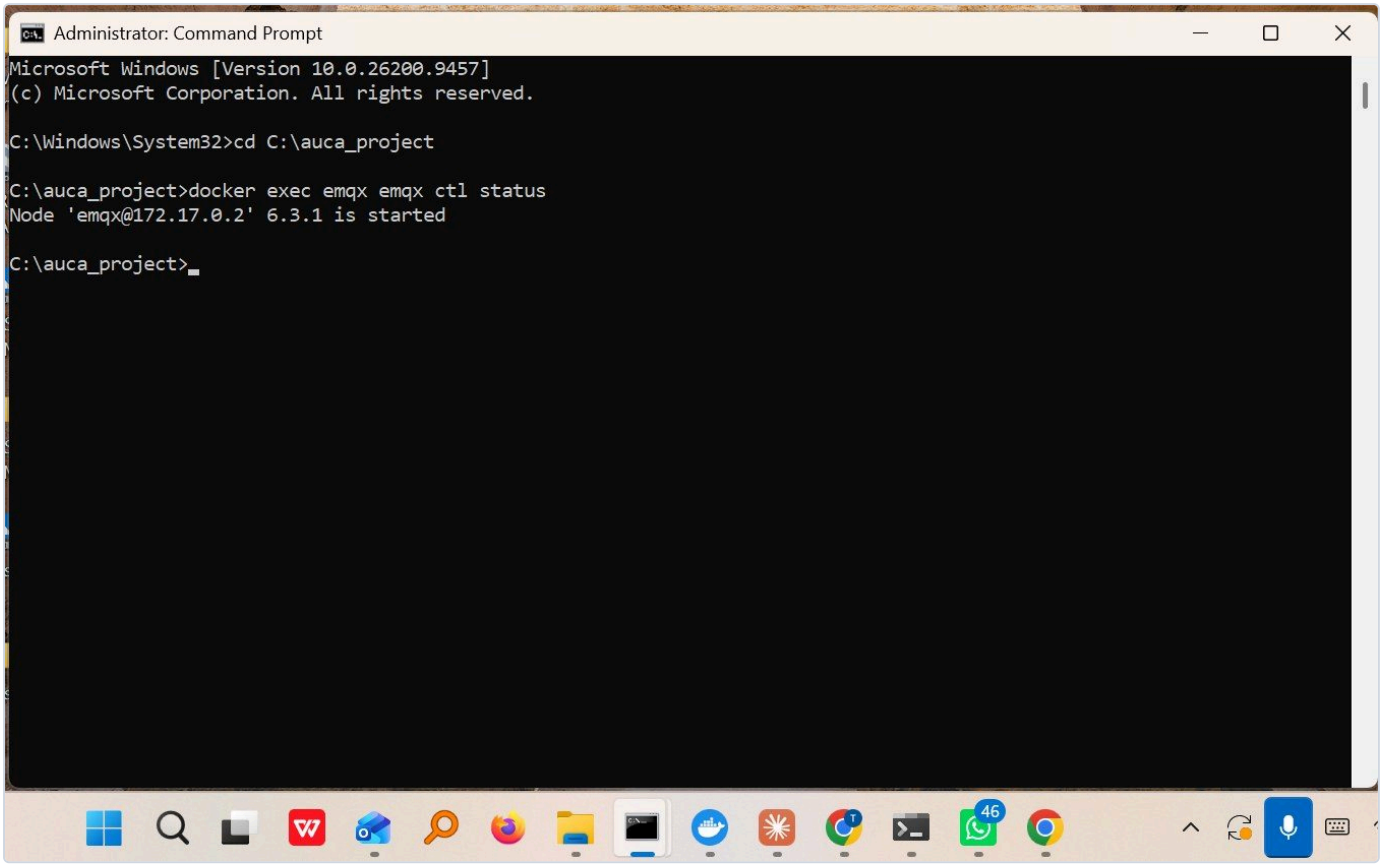
C:\auca_project>docker info --format "Docker CPUs={{.NCPU}} MemTotalBytes={{.MemTotal}}"
Docker CPUs=12 MemTotalBytes=3953799168

C:\auca_project>docker inspect iot-stack-db-1 --format "ContainerMemoryLimit={{.HostConfig.Memory}} ContainerNanoCpus={{.
HostConfig.NanoCpus}}"
ContainerMemoryLimit=0 ContainerNanoCpus=0

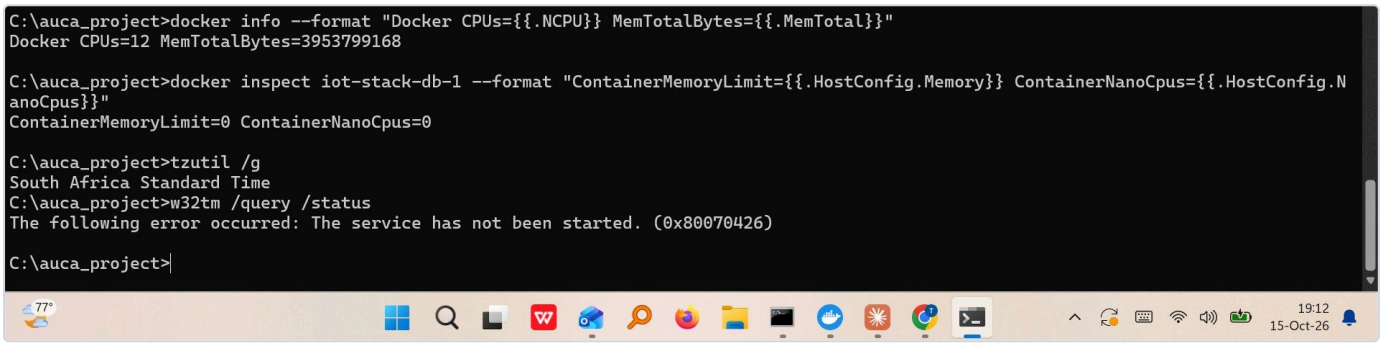
C:\auca_project>tzutil /g
South Africa Standard Time
```



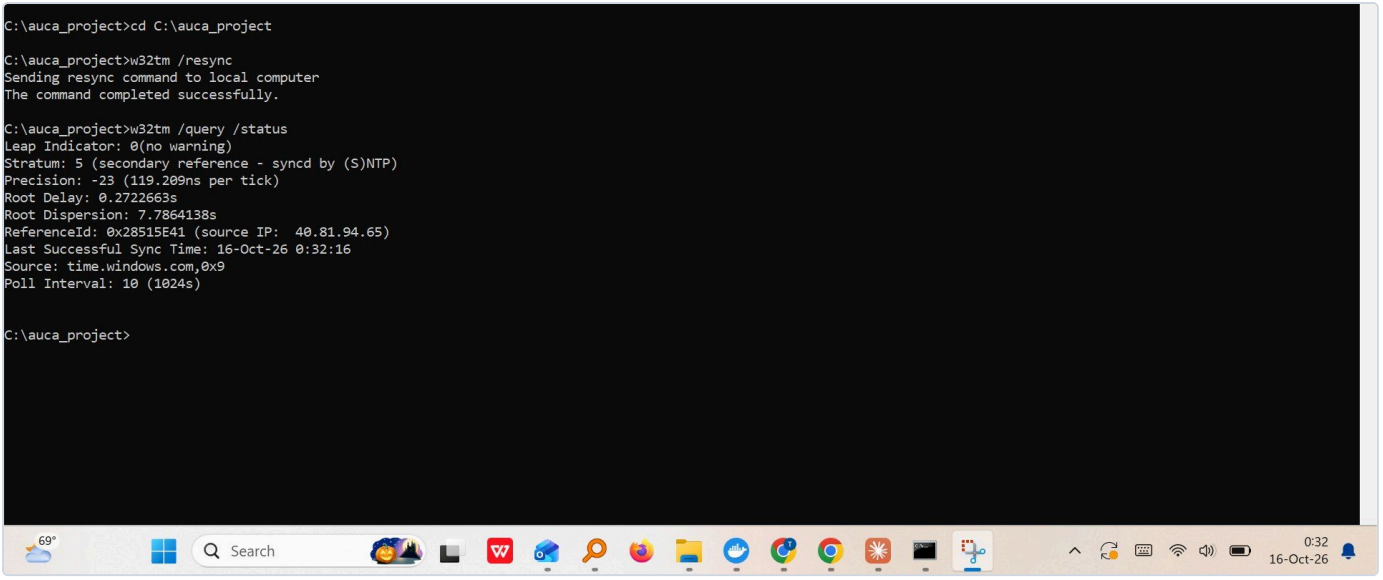
1.1 – Python 3.13.14, PostgreSQL 17.11 and the TimescaleDB 2.30.2 extension (15 October 2026, 19:06). The EMQX status result (6.3.1) is in the text record above.



1.1 – EMQX status: node emqx@172.17.0.2, version 6.3.1, started (15 October 2026, 23:14)



1.1 – Docker CPUs and memory, container limits (0 = no limit) and Windows time zone (15 October 2026, 19:12). At this moment the Windows Time service had not been started yet; it was started afterwards and synchronised (see the clock row in the table above).



1.1 – clock synchronisation (16 October 2026, 00:32): manual w32tm /resync completed successfully; status Leap Indicator 0 (no warning), stratum 5, source time.windows.com, last successful sync 16-Oct-26 00:32:16.

1.2 Create the database structures

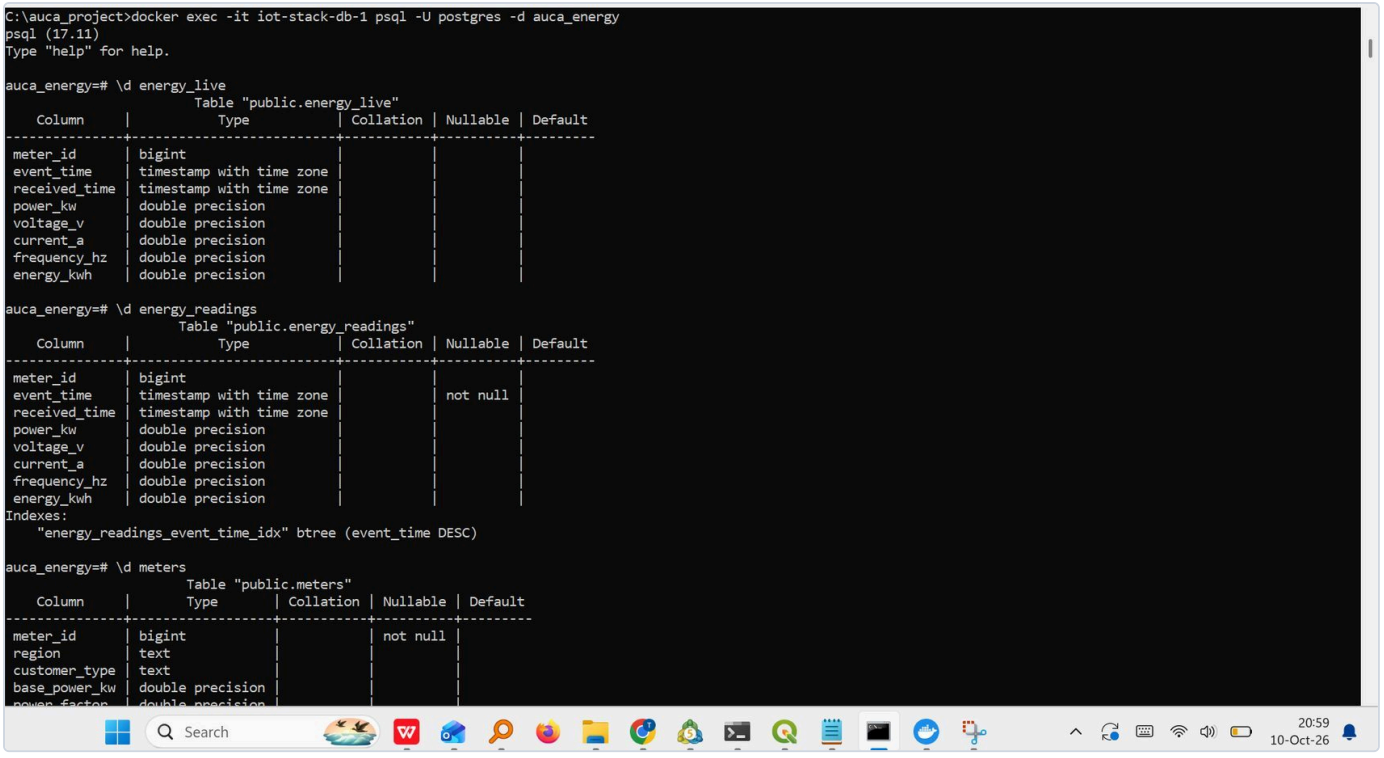
`create_tables.sql` creates `energy_live` (regular table), `energy_readings` (converted with `create_hypertable('energy_readings', 'event_time', chunk_time_interval => INTERVAL '1 day')`) and `meters`. Both reading tables have the same eight columns and no unique constraint, so duplicates are allowed at this stage. `load_meters.py` imports `smart_meter_simulator`, calls `meter_metadata()` and inserts each of the 4,000 returned dictionaries as one row in `meters`.

Data dictionary

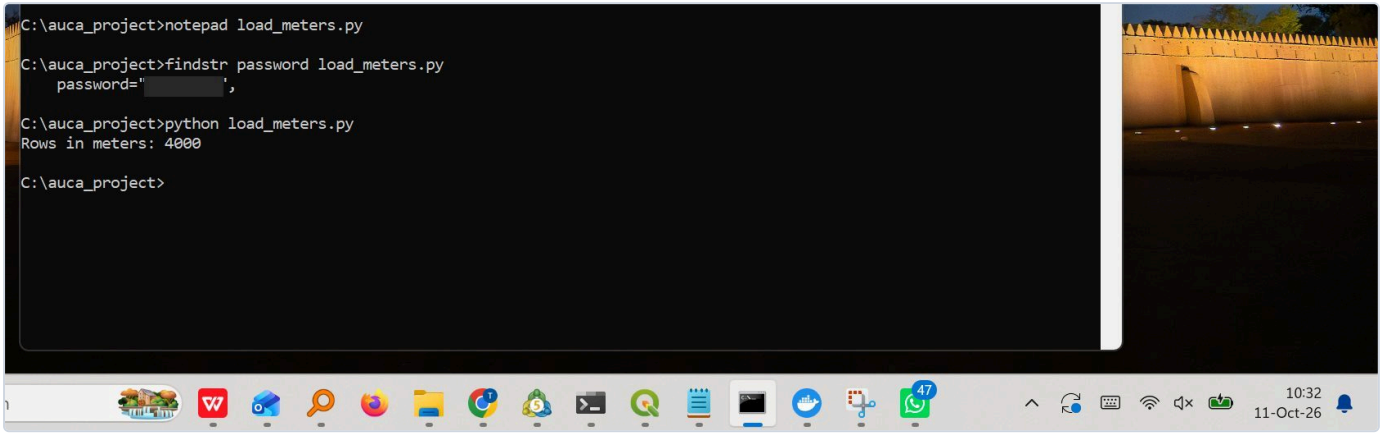
Table	Column	PostgreSQL type	Meaning	Unit
energy_live regular table: pilot messages	meter_id	BIGINT	Smart-meter identifier (1000000000–1000003999)	none (identifier)
	event_time	TIMESTAMPTZ	Start of the 15-minute interval that the reading covers (CAT, +02:00)	timestamp
	received_time	TIMESTAMPTZ	Simulated arrival time of the reading at the utility (not the database insertion time)	timestamp
	power_kw	DOUBLE PRECISION	Average power over the interval	kW
	voltage_v	DOUBLE PRECISION	Supply voltage	V
	current_a	DOUBLE PRECISION	Current	A
	frequency_hz	DOUBLE PRECISION	Grid frequency	Hz
	energy_kwh	DOUBLE PRECISION	Energy in the interval, power_kw × 0.25 h	kWh
energy_readings hypertable, 1-day chunks on event_time: full dataset	meter_id	BIGINT	Smart-meter identifier (1000000000–1000003999)	none (identifier)
	event_time	TIMESTAMPTZ	Start of the 15-minute interval that the reading covers (CAT, +02:00)	timestamp
	received_time	TIMESTAMPTZ	Simulated arrival time of the reading at the utility (not the database insertion time)	timestamp
	power_kw	DOUBLE PRECISION	Average power over the interval	kW
	voltage_v	DOUBLE PRECISION	Supply voltage	V
	current_a	DOUBLE PRECISION	Current	A
	frequency_hz	DOUBLE PRECISION	Grid frequency	Hz
	energy_kwh	DOUBLE PRECISION	Energy in the interval, power_kw × 0.25 h	kWh
meters metadata: 4,000 rows	meter_id	BIGINT (primary key)	Meter identifier, joins to the readings tables	none (identifier)
	region	TEXT	Region label, “Region 1” to “Region 10” (400 meters each)	none (label)
	customer_type	TEXT	residential, commercial or industrial	none (label)
	base_power_kw	DOUBLE PRECISION	Meter's base power level; the generator scales it by a daily load shape for the customer type	kW
	power_factor	DOUBLE PRECISION	Power factor (0.95 for every meter)	none (ratio)

Later sections add further tables that reuse the same eight columns: `energy_readings_3h` and `energy_readings_week` (Section 3), `energy_time_compressed` and `energy_meter_compressed` (Section 4), the continuous aggregates

`cagg_15min_region`, `cagg_hour_region` and `cagg_day_meter` (Section 6.1) and the test hypertable `fresh_test` with its view `fresh_test_hourly` (Section 6.2).



1.2 – column names and types of energy_live, energy_readings and meters

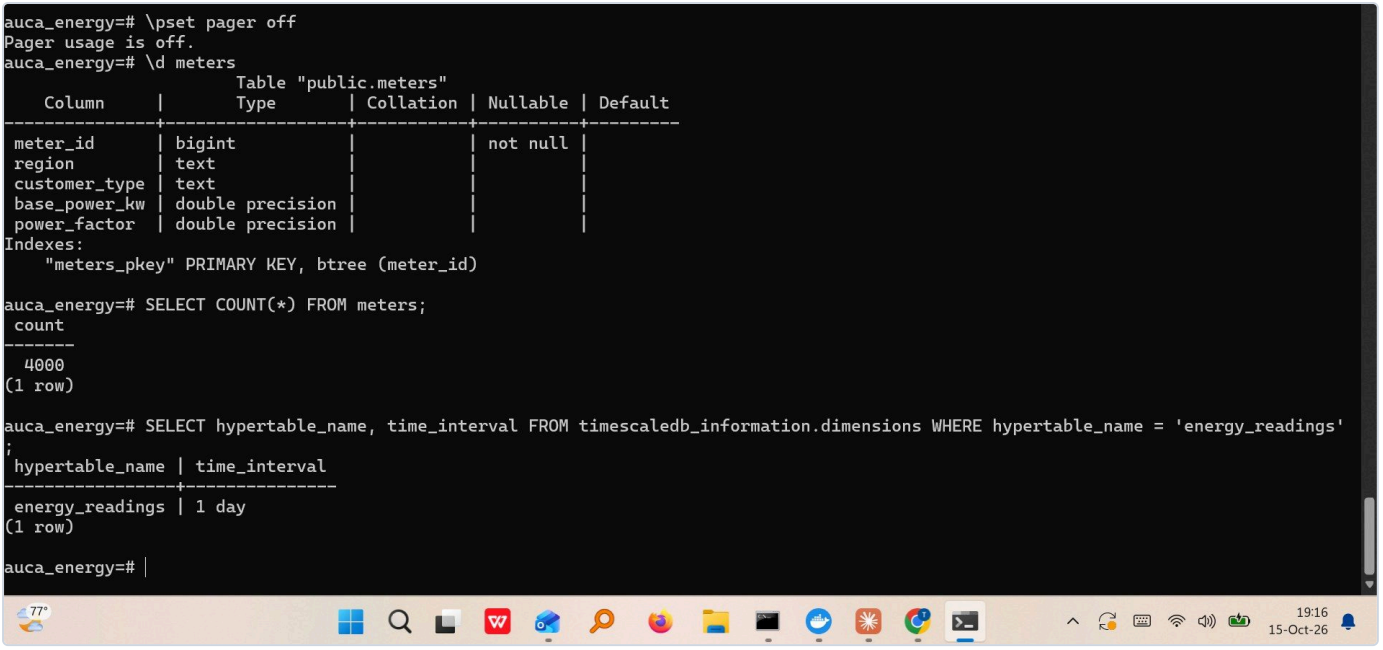


1.2 – load_meters.py inserts the metadata: “Rows in meters: 4000” (the database password is hidden)

Recorded output for the chunk interval and the metadata count (psql, 15 October 2026 19:16 CAT):

```
auca_energy=# SELECT COUNT(*) FROM meters;
 count
-------
  4000

auca_energy=# SELECT hypertable_name, time_interval FROM timescaledb_information.dimensions WHERE hypertable_name = 'ener
gy_readings';
 hypertable_name | time_interval
-----------------+---------------
 energy_readings | 1 day
```



1.2 – \d meters, 4,000 rows in meters and the energy_readings chunk interval of 1 day (15 October 2026, 19:16)

1.3 Capture and publish a pilot stream

Programs. `publisher.py` imports the unchanged generator, takes the first 2,000 dictionaries from `iter_readings()` with `itertools.islice`, converts each to JSON and publishes it to `energy/meters/{meter_id}` with QoS 1 and `retain=False`. It counts the dictionaries taken and the messages acknowledged by the broker (`wait_for_publish()` / `is_published()`). `subscriber.py` subscribes to `energy/meters/#` at QoS 1, parses each JSON message and inserts its eight fields into `energy_live`, counting received messages and failed inserts. Before every attempt `energy_live` was emptied with `TRUNCATE` and the subscriber was started before the publisher.

Counts at each stage (target 2,000)

Attempt	Taken from generator	Acknowledged by broker	Received by subscriber	Rows in energy_live (committed)	Difference from target
1 (12-Oct, 18:33–19:38)	2,000	2,000	1,032	1,032	−968 (48.4%)
2 (after the first correction)	2,000	2,000	1,091	1,091	−909 (45.5%)
3 (final, 12-Oct, ~20:00–20:20)	**2,000**	**2,000**	**2,000**	**2,000**	**0**

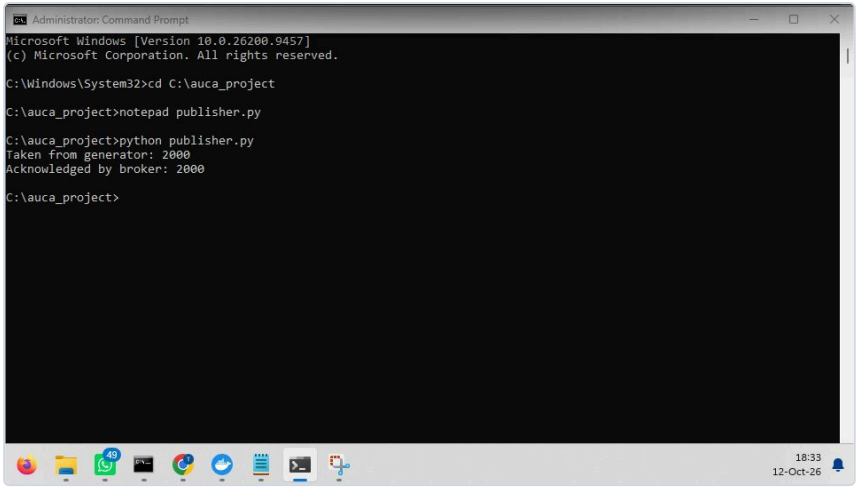
Deliberate generator duplicates are counted as separate messages and rows. Attempt 2 is listed from our own record of the run; its screenshots were not kept, so only attempts 1 and 3 are shown below.

> **Observation.** In attempt 1 the broker acknowledged all 2,000 publisher messages, but the subscriber received only 1,032 and the database held 1,032 rows, so **968 messages were lost between broker and subscriber**. The subscriber logged **0 failed inserts** (no parsing errors, no failed inserts) and no message was delivered twice (received = stored). The EMQX metrics show the cause: **delivery.dropped = 968**, all of them **delivery.dropped.queue_full = 968**, with 2,000 published and 1,032 acknowledged by the subscriber (1,032 + 968 = 2,000).

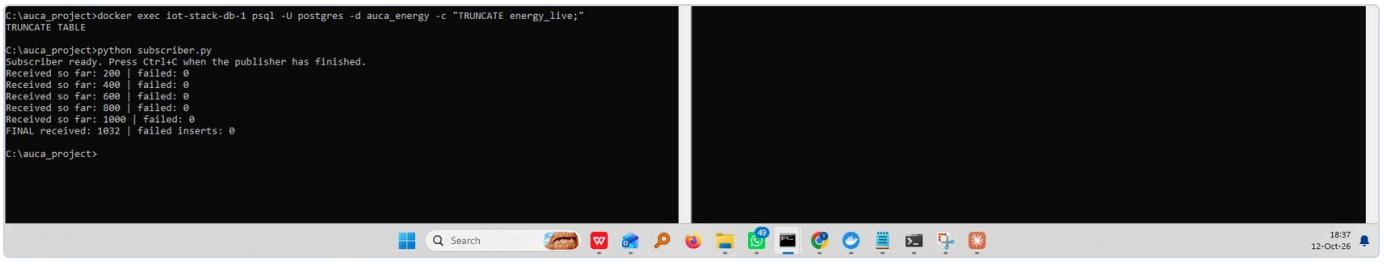
> **Explanation.** The first subscriber committed every insert synchronously (one disk flush per message), so it consumed messages more slowly than the publisher sent them. EMQX keeps a bounded queue for the subscriber's QoS 1 session; once the in-flight window and the queue were full the broker discarded new deliveries (queue_full). The publisher still received its acknowledgements because the broker had accepted the messages.

Corrections. (1) `subscriber.py` now uses `autocommit = True` and `SET synchronous_commit TO off`, so each insert returns immediately. (2) `publisher.py` now sleeps 0.02 s between messages, which keeps the subscriber's queue from

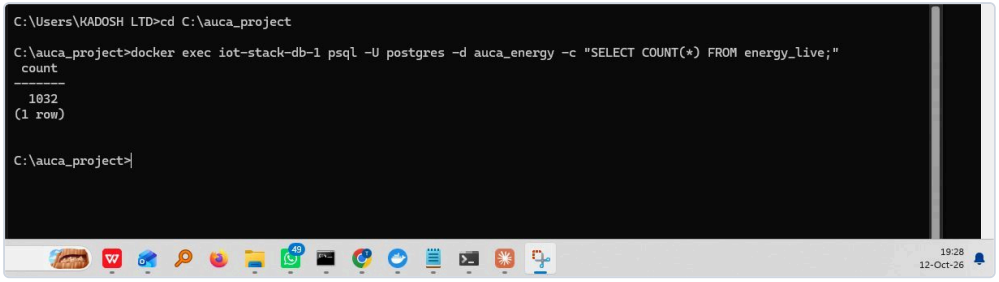
overflowing. The intermediate attempt 2 (1,091 stored, 909 still missing) was run before both changes were in place; with both changes the pilot was repeated and all four counts were 2,000 (attempt 3).



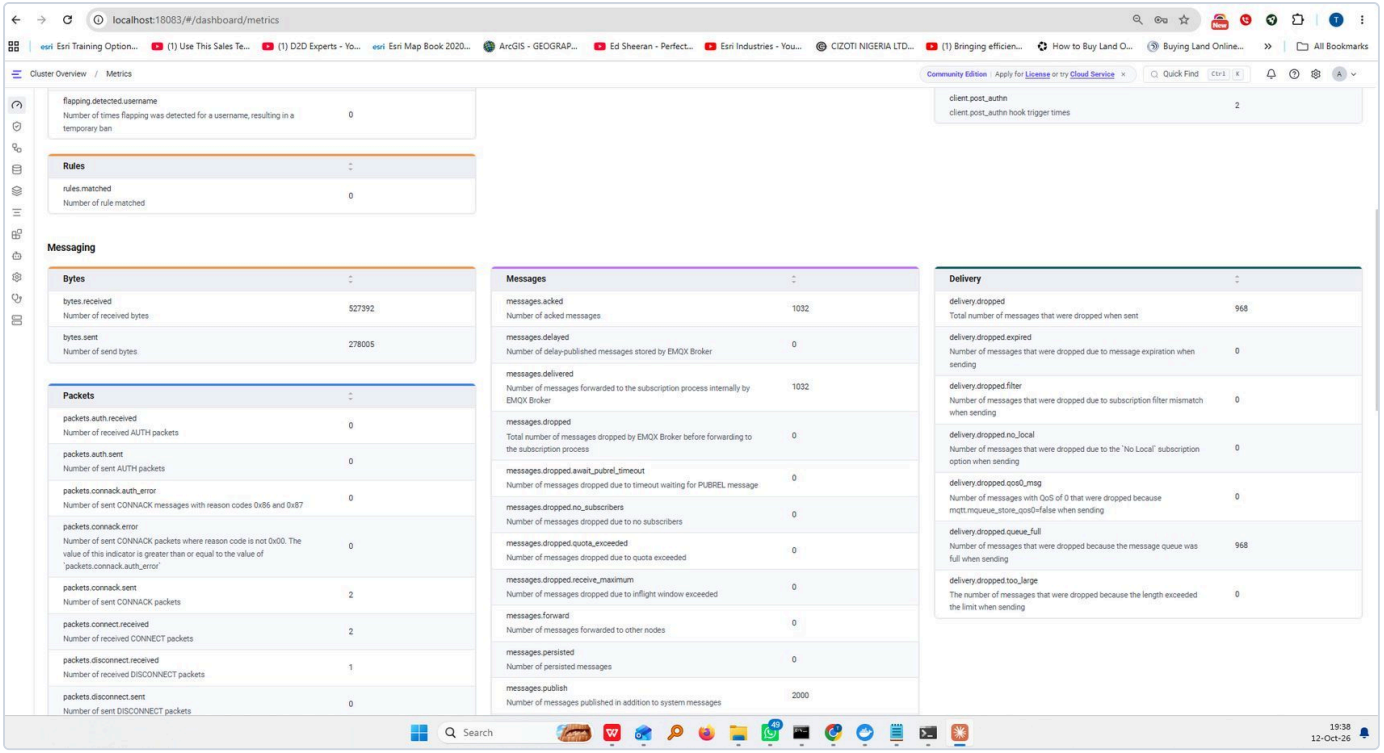
1.3 attempt 1 – publisher completion: 2000 taken from the generator, 2000 acknowledged by the broker



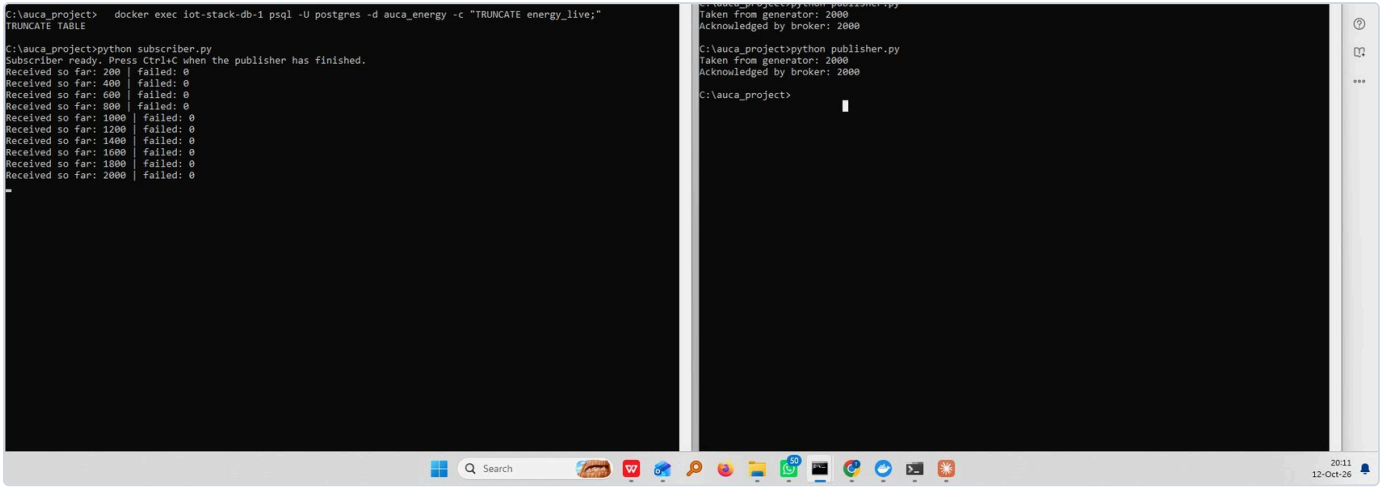
1.3 attempt 1 – subscriber received count: FINAL received 1032, failed inserts 0



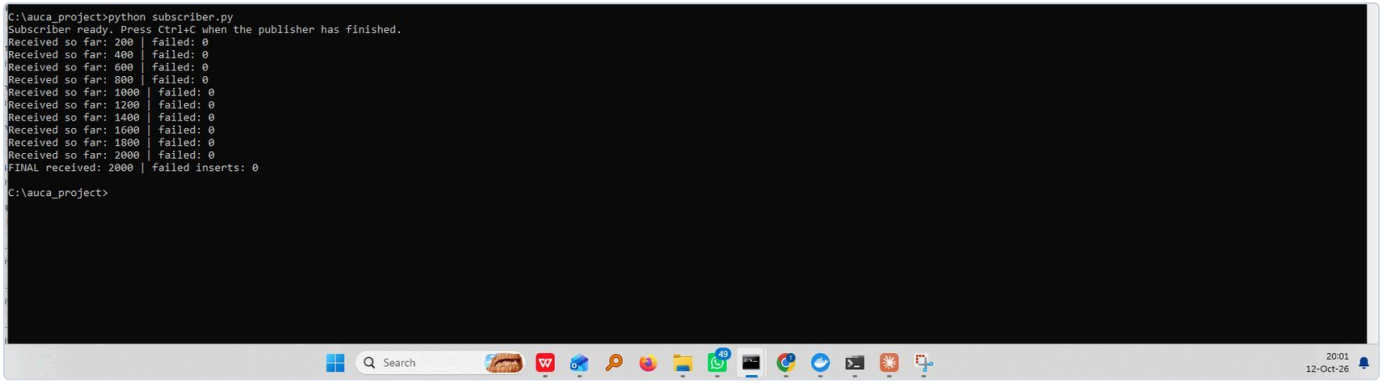
1.3 attempt 1 – database count: 1,032 rows in energy_live



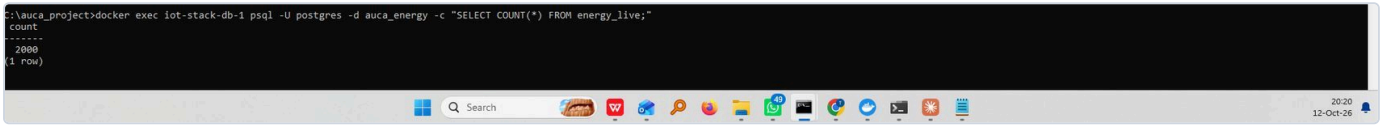
1.3 attempt 1 – EMQX metrics: 2000 published, 1032 acknowledged, 968 dropped (queue_full)



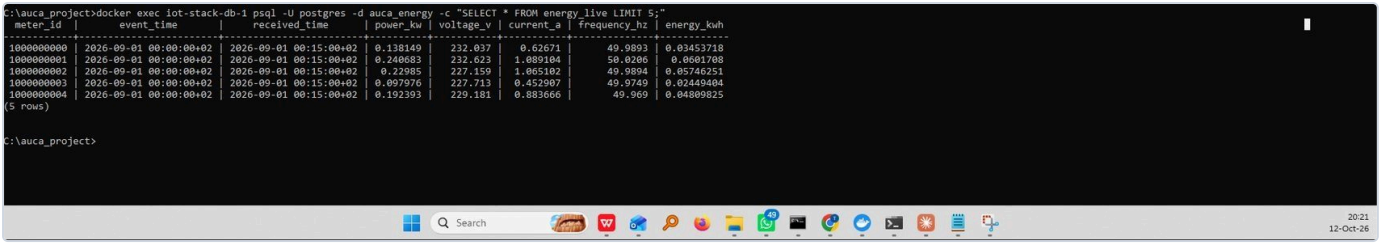
1.3 attempt 3 – publisher completion (taken 2000, acknowledged 2000) beside the running subscriber



1.3 attempt 3 – subscriber received count: FINAL received 2000, failed inserts 0



1.3 attempt 3 – database count: 2,000 rows in energy_live



1.3 attempt 3 – sample rows stored in energy_live

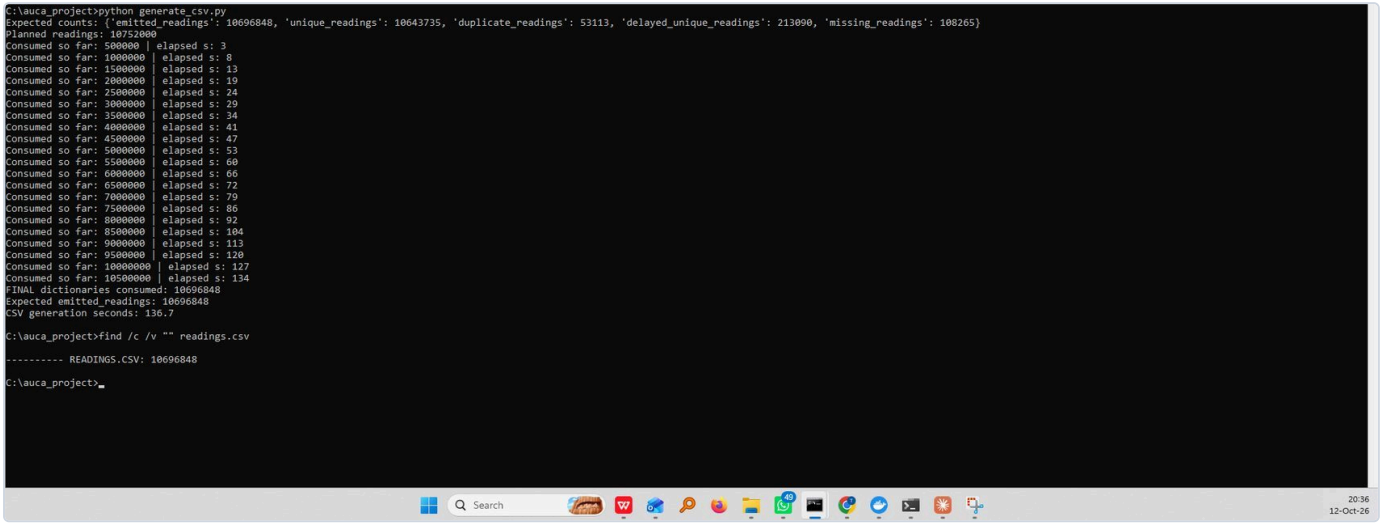
The README pilot run (Section 9.2) repeats this test with the final programs and shows the four counts together.

2 Capture and prepare the historical dataset

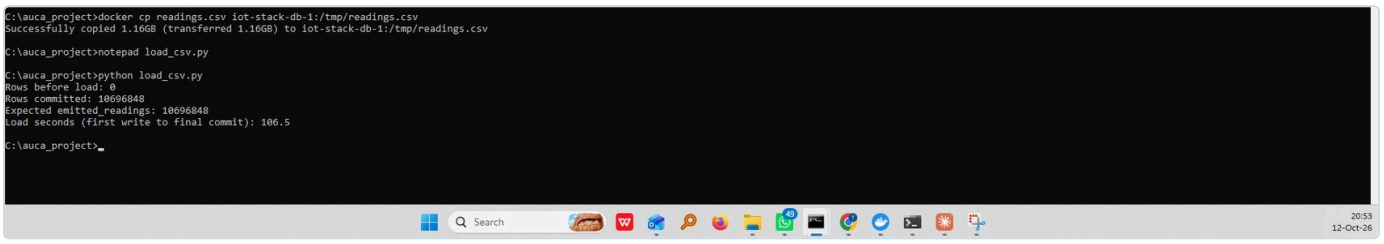
2.1 Capture and load the complete stream

Chosen ingestion route: CSV bulk load. `generate_csv.py` calls `dataset_info()`, starts a fresh `iter_readings()` iterator and streams every dictionary to `readings.csv` row by row (progress is printed every 500,000 readings). The 1.16 GB file is copied into the database container with `docker cp` and `load_csv.py` loads it with a single server-side `COPY` into the empty `energy_readings`, including duplicates. The 2,000 pilot rows stay in `energy_live`.

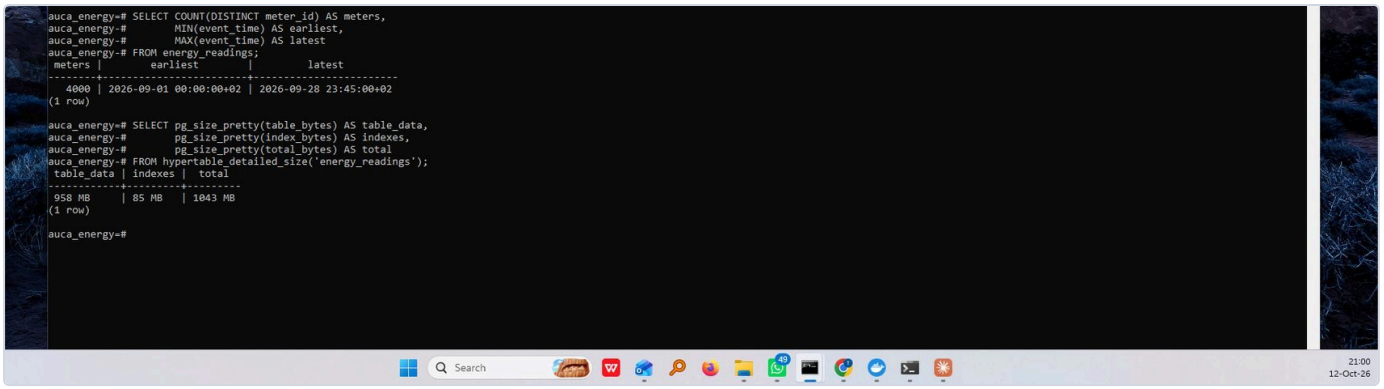
Quantity	Value
planned_readings (dataset_info)	10,752,000 (4,000 meters × 28 days × 96)
expected_counts.emitted_readings	10,696,848
Dictionaries consumed from the iterator	**10,696,848**
Data rows in readings.csv	**10,696,848** (`find /c /v ""`)
Rows committed in energy_readings	**10,696,848** (rows before load: 0)
All three agree with emitted_readings	Yes
CSV generation time	136.7 s
Database load time (first write to final commit)	106.5 s
Total programme time (136.7 + 106.5)	243.2 s — the 1.16 GB `docker cp` between the two steps was not timed separately; the screenshots of the two steps are 17 minutes apart (20:36 and 20:53 CAT) including manual steps
Distinct meters	4,000
Earliest / latest event_time	2026-09-01 00:00:00+02 / 2026-09-28 23:45:00+02
Size of energy_readings (all chunks) at the end of 2.1	table data 958 MB, indexes 85 MB, total 1,043 MB



2.1 – CSV generation: progress, final dictionaries consumed (10,696,848), generation seconds (136.7) and CSV data rows (10,696,848)



2.1 – transfer into the container and load: 10,696,848 rows committed, load seconds 106.5



2.1 – database checks: distinct meters, earliest and latest event_time, table / index / total size

The intermediate CSV can be regenerated at any time with `generate_csv.py`; the programs that create it are kept in the repository.

2.2 Investigate data quality

All counts below were queried on `energy_readings` before cleaning. A unique reading is one (meter_id, event_time) pair. Delayed readings are unique pairs with `received_time > event_time + 15 minutes` (30 to 180 minutes after event_time in this dataset).

Entry	Expected	Measured	Match
planned_readings	10,752,000	10,752,000	Yes
emitted_readings	10,696,848	10,696,848	Yes
unique_readings	10,643,735	10,643,735	Yes
duplicate_readings (extra copies)	53,113	53,113	Yes
delayed_unique_readings	213,090	213,090	Yes
missing_readings	108,265	108,265	Yes

Check / percentage	Calculation	Result
Unique + extra copies = emitted	10,643,735 + 53,113	10,696,848 = emitted ✓
Planned − unique = missing	10,752,000 − 10,643,735	108,265 = missing ✓
Missing percentage	100 × 108,265 / 10,752,000	1.0069 %
Duplicate percentage	100 × 53,113 / 10,643,735	0.4990 %
Delayed percentage	100 × 213,090 / 10,643,735	2.0020 %
Completeness	100 × 10,643,735 / 10,752,000	98.9931 %

The five meters with the most missing readings

Rank	meter_id	Missing readings (of 2,688 planned)
1	1000003622	47
2	1000000106	45
3	1000000366	45
4	1000001259	44
5	1000000189	43

> **Observation.** Every measured count equals the reference table exactly. About 1.0% of planned readings are missing, 0.5% of observed pairs have an extra copy and 2.0% of unique readings arrive late. Missing readings are spread thinly over the meters (the worst meter lacks 47 of 2,688 readings, 1.7%).

```
auca_energy=# SELECT
auca_energy-#   COUNT(*) AS emitted_rows,
auca_energy-#   COUNT(DISTINCT (meter_id, event_time)) AS unique_pairs,
auca_energy-#   COUNT(*) - COUNT(DISTINCT (meter_id, event_time)) AS extra_copies,
auca_energy-#   10752000 - COUNT(DISTINCT (meter_id, event_time)) AS missing_pairs
auca_energy-# FROM energy_readings;
 emitted_rows | unique_pairs | extra_copies | missing_pairs
--------------+--------------+--------------+---------------
     10696848 |     10643735 |        53113 |        108265
(1 row)

auca_energy=#
```

2.2 – diagnostic queries and counts: emitted, unique, extra copies

```
auca_energy=# SELECT COUNT(*) AS delayed_unique
auca_energy-# FROM (
auca_energy(#   SELECT DISTINCT meter_id, event_time
auca_energy(#   FROM energy_readings
auca_energy(#   WHERE received_time > event_time + INTERVAL '15 minutes'
auca_energy(# ) t;
 delayed_unique
----------------
         213090
(1 row)
```

2.2 – delayed unique readings

```
auca_energy=# WITH u AS (
auca_energy(#   SELECT COUNT(*) AS emitted,
auca_energy(#          COUNT(DISTINCT (meter_id, event_time)) AS uniq
auca_energy(#   FROM energy_readings
auca_energy(# ), d AS (
auca_energy(#   SELECT COUNT(*) AS delayed FROM (
auca_energy(#     SELECT DISTINCT meter_id, event_time
auca_energy(#     FROM energy_readings
auca_energy(#     WHERE received_time > event_time + INTERVAL '15 minutes') t
auca_energy(# )
auca_energy-# SELECT u.emitted,
auca_energy-#        u.uniq AS unique_pairs,
auca_energy-#        u.emitted - u.uniq AS extra_copies,
auca_energy-#        10752000 - u.uniq AS missing_pairs,
auca_energy-#        d.delayed AS delayed_unique,
auca_energy-#        ROUND(100.0 * (10752000 - u.uniq) / 10752000, 4) AS missing_pct,
auca_energy-#        ROUND(100.0 * (u.emitted - u.uniq) / u.uniq, 4) AS duplicate_pct,
auca_energy-#        ROUND(100.0 * d.delayed / u.uniq, 4) AS delayed_pct,
auca_energy-#        ROUND(100.0 * u.uniq / 10752000, 4) AS completeness_pct
auca_energy-# FROM u, d;
 emitted  | unique_pairs | extra_copies | missing_pairs | delayed_unique | missing_pct | duplicate_pct | delayed_pct | completeness_pct
----------+--------------+--------------+---------------+----------------+-------------+---------------+-------------+------------------
 10696848 |     10643735 |        53113 |        108265 |         213090 |      1.0069 |        0.4990 |      2.0020 |          98.9931
(1 row)

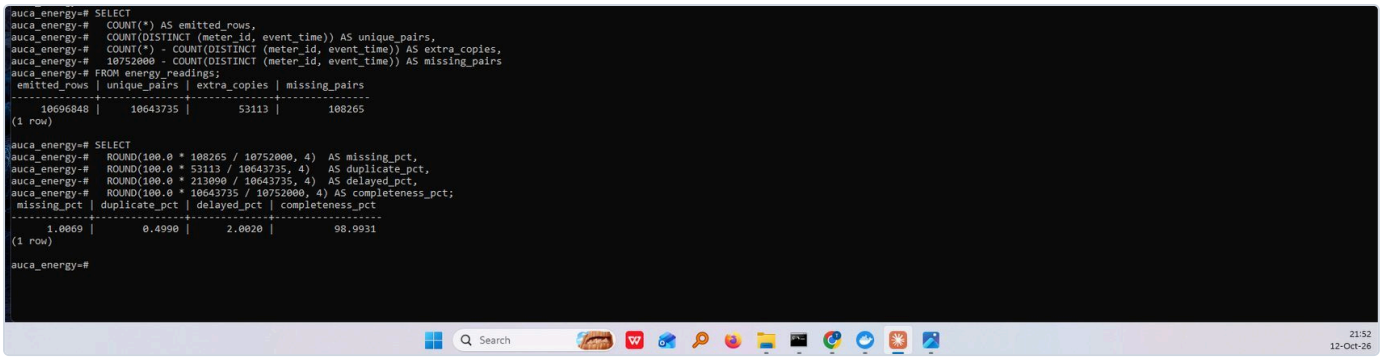
auca_energy=#
```

2.2 – counts compared with the reference and percentages

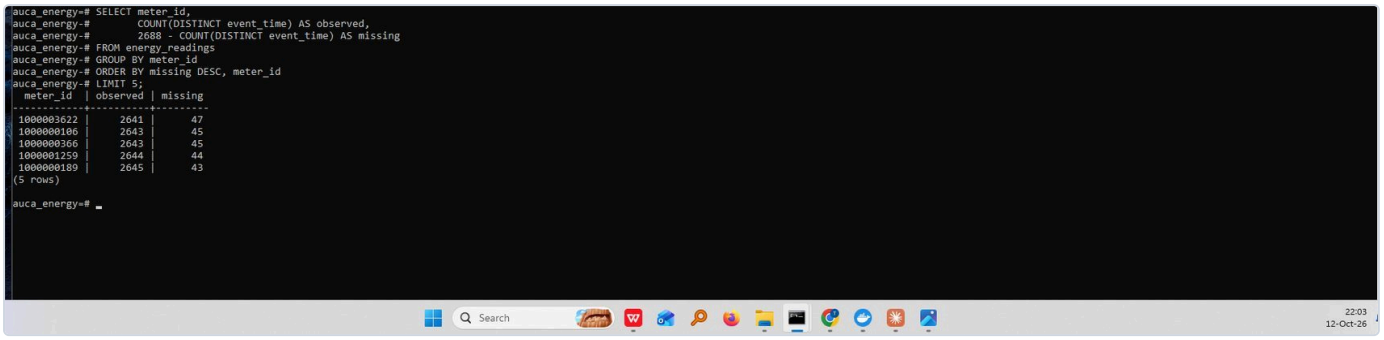
```
auca_energy=# SELECT 10643735 + 53113 AS unique_plus_extra,
auca_energy-#        10696848 AS emitted,
auca_energy-#        10752000 - 10643735 AS planned_minus_unique,
auca_energy-#        108265 AS missing;
 unique_plus_extra | emitted  | planned_minus_unique | missing
-------------------+----------+----------------------+---------
          10696848 | 10696848 |               108265 |  108265
(1 row)

auca_energy=#
```

2.2 – identity checks: unique + extra = emitted; planned − unique = missing



2.2 – missing, duplicate and delayed percentages and completeness



2.2 – the five meters with the most missing readings

2.3 Clean and preserve a consistent benchmark

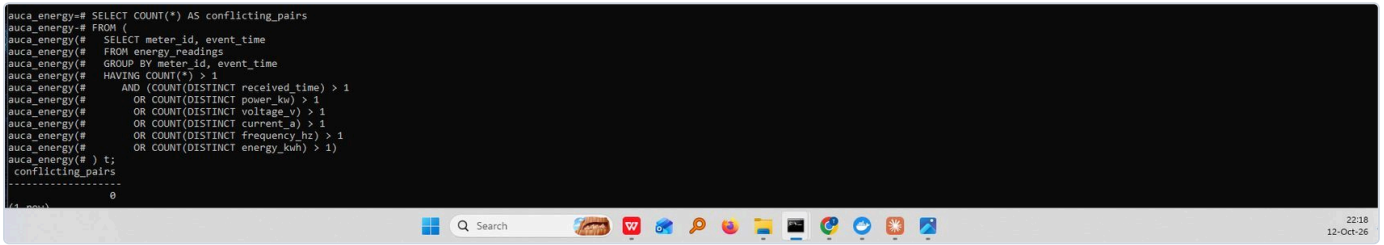
Conflict check first. We grouped rows by (meter_id, event_time) and counted pairs with more than one row whose received_time, power_kw, voltage_v, current_a, frequency_hz or energy_kwh differ. **Result: 0 conflicting pairs.** Of the 10,643,735 pairs, 53,113 have exactly one extra copy and every extra copy is identical in all six compared values (plus meter_id and event_time). Because all six values match, the extra copies are safely redundant.

Cleaning code (repeatable). `clean_duplicates.sql` runs in one transaction: it copies `SELECT DISTINCT` rows of all eight columns into a temporary table, truncates `energy_readings`, re-inserts the distinct rows and commits. Missing intervals are left as gaps. The cleaned table is the dataset for every later experiment.

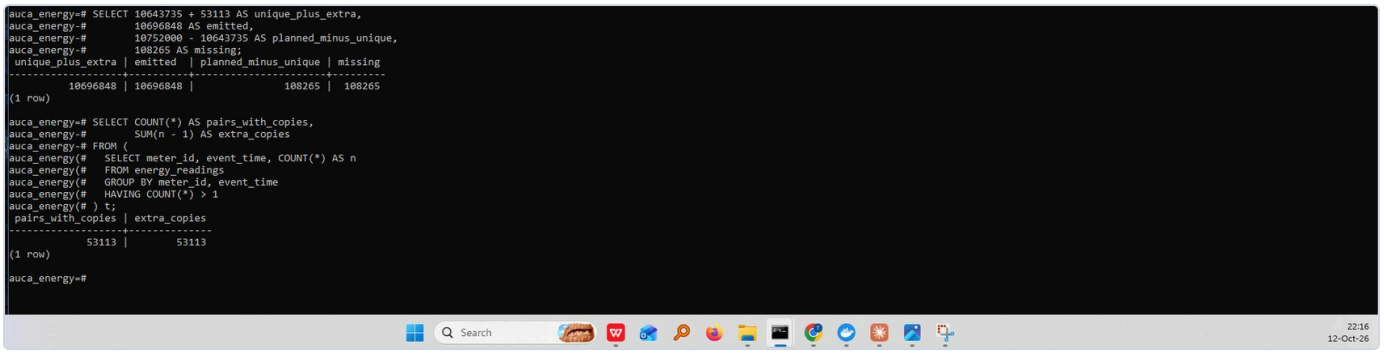
	Row count	SUM(energy_kwh)	Duplicate pairs
Before cleaning	10,696,848	3,872,863.8931792155	53,113 pairs with an extra copy
After cleaning	10,643,735	3,853,704.3468159684	0
Difference	−53,113	−19,159.5463632471 (the energy of the removed copies)	

The final row count equals `expected_counts.unique_readings` = 10,643,735 and no duplicate pair remains.

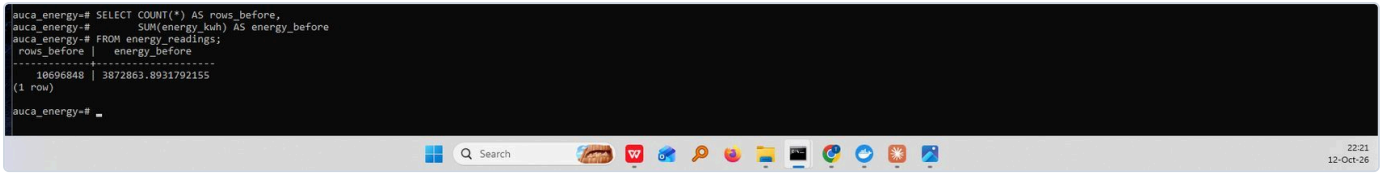
Why missing readings must not automatically become zero. A missing reading means we do not know the consumption in that interval; it does not mean the meter consumed nothing. Filling gaps with zero would understate energy, pull averages, minima and daily totals down, make a meter with a communication fault look like a meter with no demand, and distort peaks, anomaly rules and the forecast. Keeping the gap lets every query count observed readings honestly, lets completeness be reported separately (Section 5.3) and leaves a later decision (leave blank, interpolate or estimate) to an explicit, documented step.



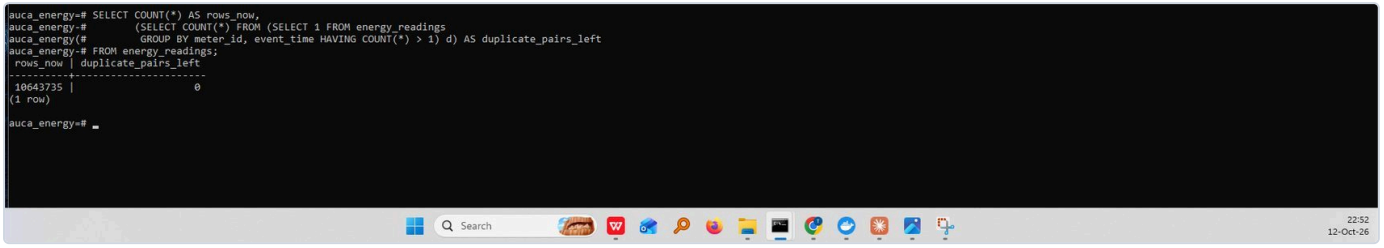
2.3 – conflict check: 0 pairs with differing values



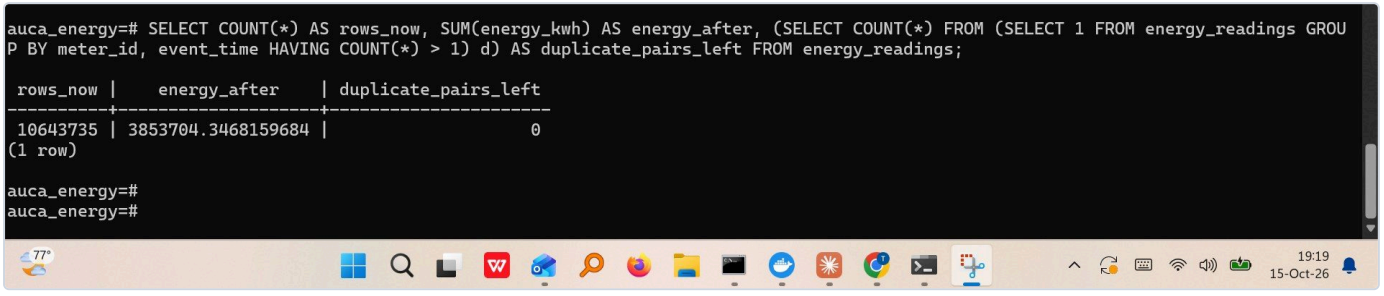
2.3 – duplicate pairs and extra copies before cleaning



2.3 – row count and summed energy before cleaning



2.3 – cleaning completion: row count after cleaning



2.3 – after cleaning: 10,643,735 rows, summed energy 3,853,704.35 kWh and 0 duplicate pairs left (15 October 2026, 19:19)

Recorded output of the after-cleaning check (psql, 15 October 2026 19:19 CAT):

```
auca_energy=# SELECT COUNT(*) AS rows_now, SUM(energy_kwh) AS energy_after, (SELECT COUNT(*) FROM (SELECT 1 FROM energy_r
eadings GROUP BY meter_id, event_time HAVING COUNT(*) > 1) d) AS duplicate_pairs_left FROM energy_readings;
 rows_now |    energy_after    | duplicate_pairs_left
----------+--------------------+----------------------
 10643735 | 3853704.3468159684 |                    0
```

> The energy sums in later checks differ from this value by about 10^{-9} kWh (for example 3,853,704.3468154166 in Section 3.1) only because floating-point addition order changes with the scan order; the difference is far below the 0.001 tolerance.

3 Compare chunk intervals

3.1 Create and verify three configurations

`create_hypertables.sql` creates `energy_readings_3h` (3-hour chunks) and `energy_readings_week` (7-day chunks) with the same eight columns and `event_time` partitioning; `energy_readings` stays the one-day hypertable. The cleaned rows were copied into both with `INSERT INTO … SELECT * FROM energy_readings`. All three tables carry the same two non-unique B-tree indexes: `(event_time DESC)`, created automatically by `create_hypertable`, and `(meter_id, event_time DESC)`, created explicitly. Index method and sort order are identical on all three. `ANALYZE` was run on all three tables after loading.

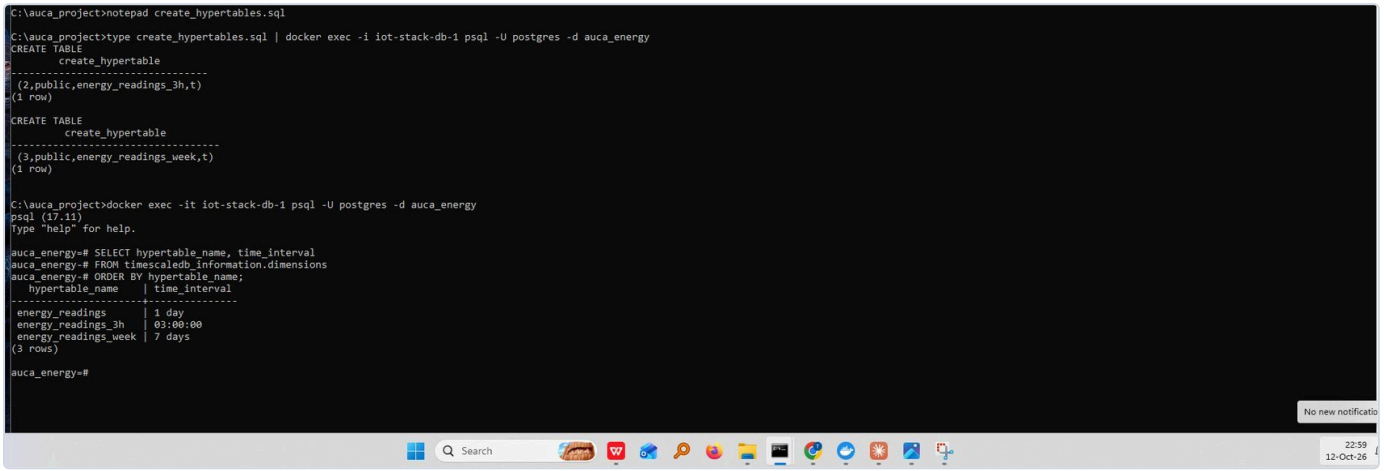
Table	Chunk interval	Chunks	First range start	Last range end
energy_readings	1 day	29	2026-08-31 02:00:00+02	2026-09-29 02:00:00+02
energy_readings_3h	3 hours	225	2026-08-31 23:00:00+02	2026-09-29 02:00:00+02
energy_readings_week	7 days	5	2026-08-27 02:00:00+02	2026-10-01 02:00:00+02

> **Observation.** Chunk ranges are aligned to UTC, not to CAT midnight: a one-day chunk runs from 02:00 CAT to 02:00 CAT the next day, three-hour chunks start at 02:00, 05:00, 08:00 … CAT, and week chunks start on a Thursday at 02:00 CAT. A CAT calendar day therefore overlaps two one-day chunks.

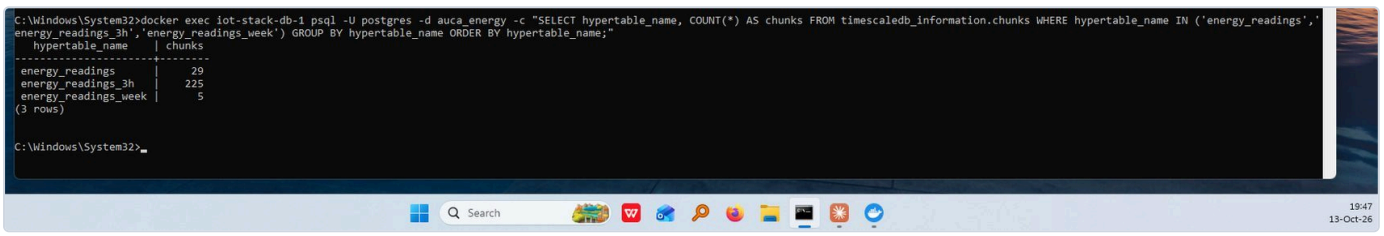
Matching data checks

Table	Rows	Meters	Earliest event_time	Latest event_time	SUM(power_kw)	SUM(energy_kwh)
energy_readings	10,643,735	4,000	2026-09-01 00:00:00+02	2026-09-28 23:45:00+02	15414817.387793025	3853704.3468154166
energy_readings_3h	10,643,735	4,000	2026-09-01 00:00:00+02	2026-09-28 23:45:00+02	15414817.387793	3853704.3468154133
energy_readings_week	10,643,735	4,000	2026-09-01 00:00:00+02	2026-09-28 23:45:00+02	15414817.387793005	3853704.3468154157

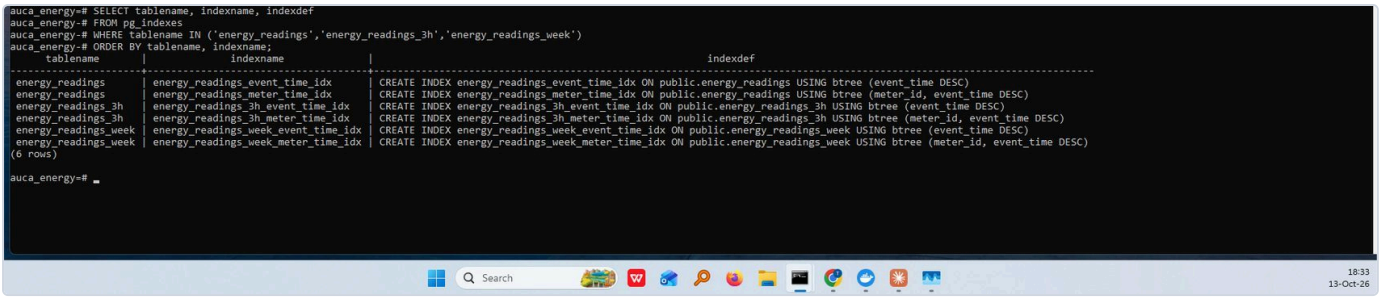
All three tables have the required 10,643,735 rows and 4,000 meters, identical date limits, and sums that agree within 3×10^{-8} (tolerance 0.001).



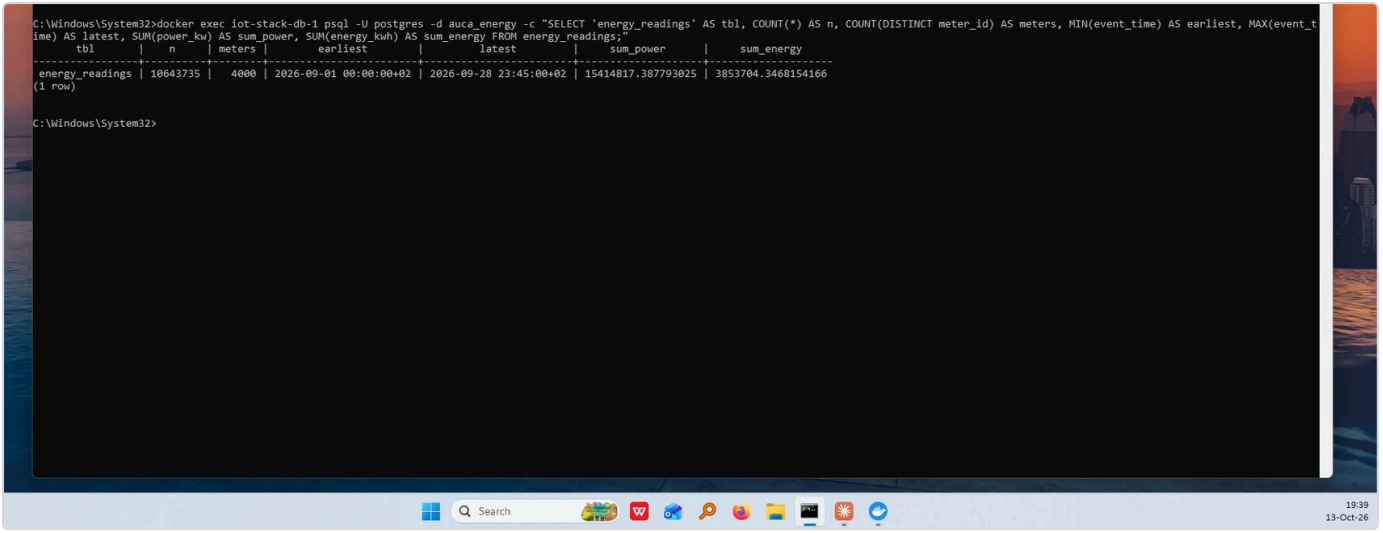
3.1 – chunk interval of each table



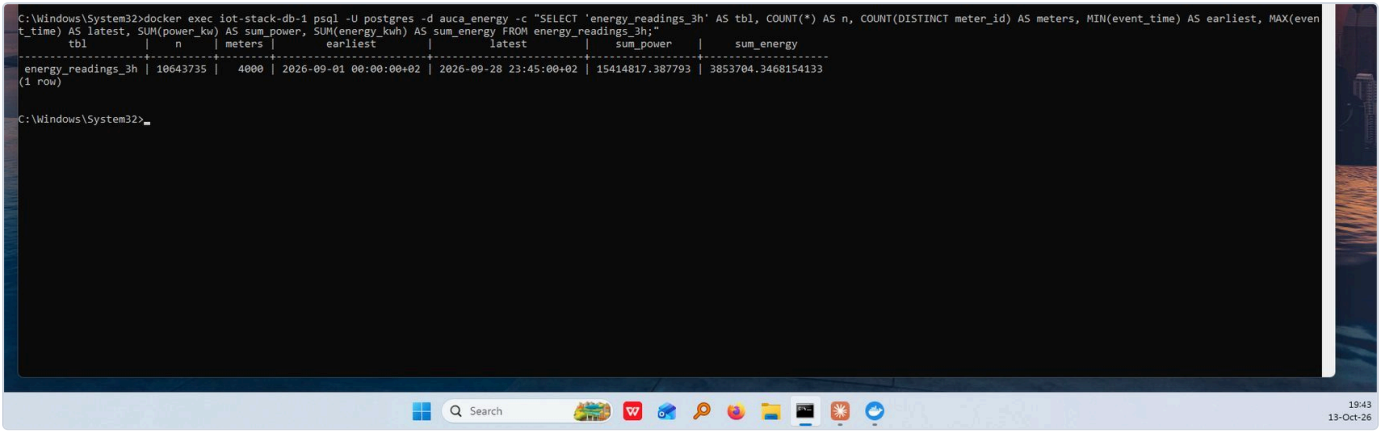
3.1 – number of chunks per table



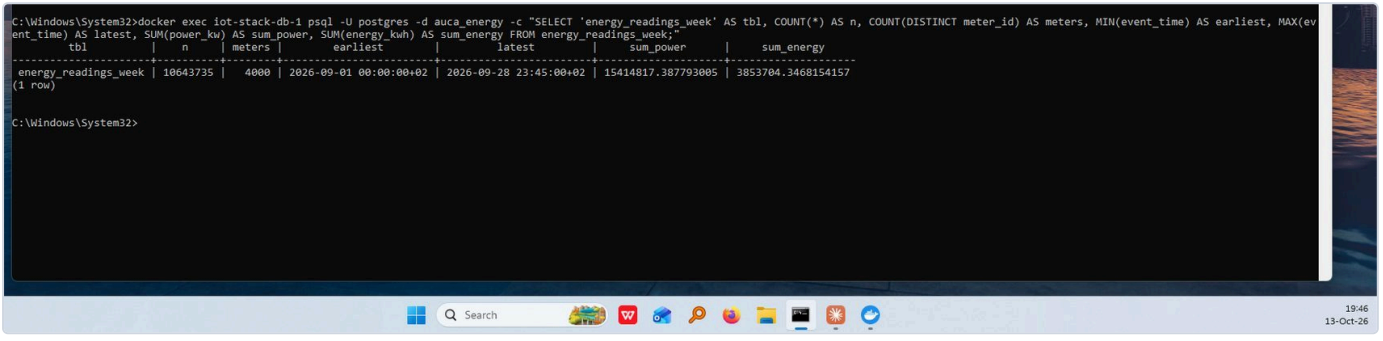
3.1 – matching indexes on the three tables



3.1 – data check, energy_readings (1 day)



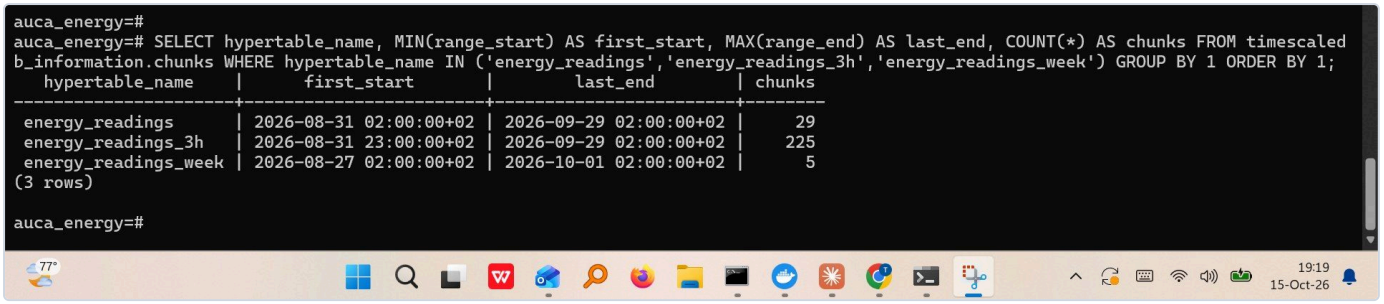
3.1 – data check, energy_readings_3h (3 hours)



3.1 – data check, energy_readings_week (7 days)

Recorded output for the chunk date ranges (psql, 15 October 2026 19:19 CAT):

```
auca_energy=# SELECT hypertable_name, MIN(range_start) AS first_start, MAX(range_end) AS last_end, COUNT(*) AS chunks FRO
M timescaledb_information.chunks WHERE hypertable_name IN ('energy_readings','energy_readings_3h','energy_readings_week')
GROUP BY 1 ORDER BY 1;
   hypertable_name    |      first_start       |        last_end        | chunks
----------------------+------------------------+------------------------+--------
 energy_readings      | 2026-08-31 02:00:00+02 | 2026-09-29 02:00:00+02 |     29
 energy_readings_3h   | 2026-08-31 23:00:00+02 | 2026-09-29 02:00:00+02 |    225
 energy_readings_week | 2026-08-27 02:00:00+02 | 2026-10-01 02:00:00+02 |      5
```



3.1 – chunk date ranges of the three tables: first start, last end and number of chunks (15 October 2026, 19:19)

3.2 Implement and time the five workloads

Client and timing tool. All SQL runs used one named client: the Python program `benchmark_chunks.py` (Python 3.13.14, psycopg2, one autocommit connection with time zone Africa/Kigali) on the machine **Josh**. Duration is measured with `time.perf_counter()` from just before `cursor.execute()` until after `fetchall()` has returned the last result row (statement submission to last result row received). Execution plans were taken separately in `psql` with `EXPLAIN (ANALYZE, BUFFERS)`.

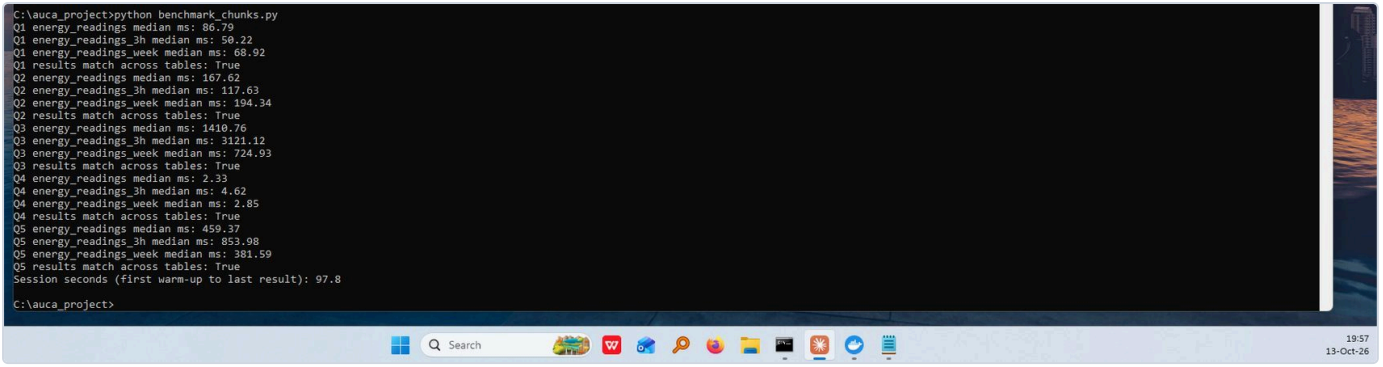
Procedure. For each query and each table: one warm-up execution, then five measured executions, run sequentially with no PostgreSQL restart and no cache clearing, so the results are **warm-cache measurements**. Only the table name changes between configurations. Results were compared across the three tables after each query (float tolerance 0.001) and matched for all five queries. Statistics were refreshed with `ANALYZE` after loading. No loaders, compression policies or continuous-aggregate jobs existed or were running during these runs (they were created later, in Sections 4 and 6) and competing applications were closed. Every run's CAT start and end time, duration and machine name are recorded in `benchmark_runs.csv` (75 rows); per-query medians, minima and maxima are in `benchmark_summary.csv`. **Session time** from the first warm-up submission to the last measured result: **97.8 s** (run on 13 October 2026, about 19:51 CAT).

The five queries (only the table name changes)

```
Q1  SELECT date_trunc('hour', event_time) AS hr, AVG(power_kw) FROM {t}
    WHERE event_time >= '2026-09-21 00:00:00+02' AND event_time < '2026-09-22 00:00:00+02' GROUP BY 1 ORDER BY 1
Q2  SELECT event_time, SUM(power_kw) AS total_kw FROM {t}
    WHERE event_time >= '2026-09-22 00:00:00+02' AND event_time < '2026-09-29 00:00:00+02'
    GROUP BY event_time ORDER BY total_kw DESC, event_time ASC LIMIT 10
Q3  SELECT meter_id, SUM(energy_kwh) FROM {t} GROUP BY meter_id ORDER BY meter_id
Q4  SELECT SUM(energy_kwh), AVG(power_kw), MAX(power_kw) FROM {t}
    WHERE meter_id = 1000000000 AND event_time >= '2026-09-22 00:00:00+02' AND event_time < '2026-09-29 00:00:00+02'
Q5  SELECT COUNT(*), SUM(energy_kwh), AVG(power_kw), MIN(power_kw), MAX(power_kw) FROM {t}
```

Warm-cache results: median (minimum–maximum) in milliseconds, five measured runs

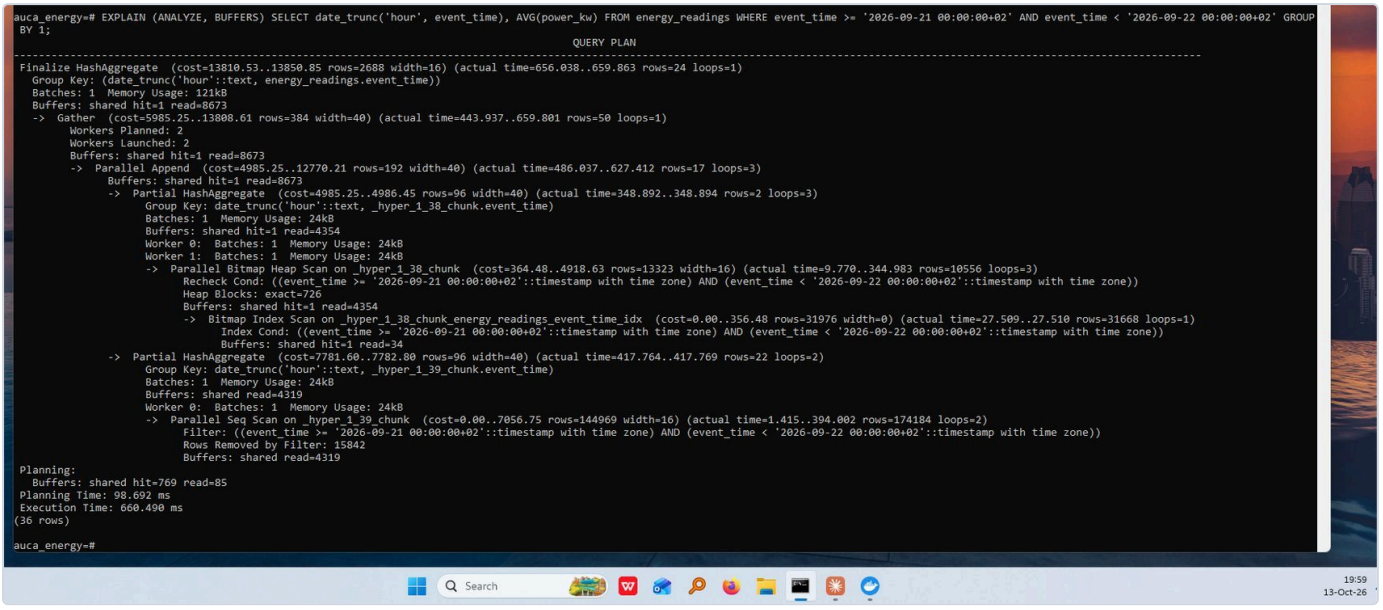
Query	energy_readings 1-day chunks	energy_readings_3h 3-hour chunks	energy_readings_week 7-day chunks	Fastest median
Q1	86.79 (74.22–127.60)	**50.22 (47.98–54.95)**	68.92 (66.61–89.90)	3 hours
Q2	167.62 (159.23–174.11)	**117.63 (115.43–132.51)**	194.34 (189.37–205.22)	3 hours
Q3	1,410.76 (657.96–3,655.04)	3,121.12 (2,083.34–9,930.13)	**724.93 (554.03–1,179.14)**	7 days
Q4	**2.33 (1.94–2.72)**	4.62 (4.17–5.53)	2.85 (1.78–3.88)	1 day
Q5	459.37 (369.43–1,221.61)	853.98 (558.28–1,730.39)	**381.59 (349.14–527.14)**	7 days



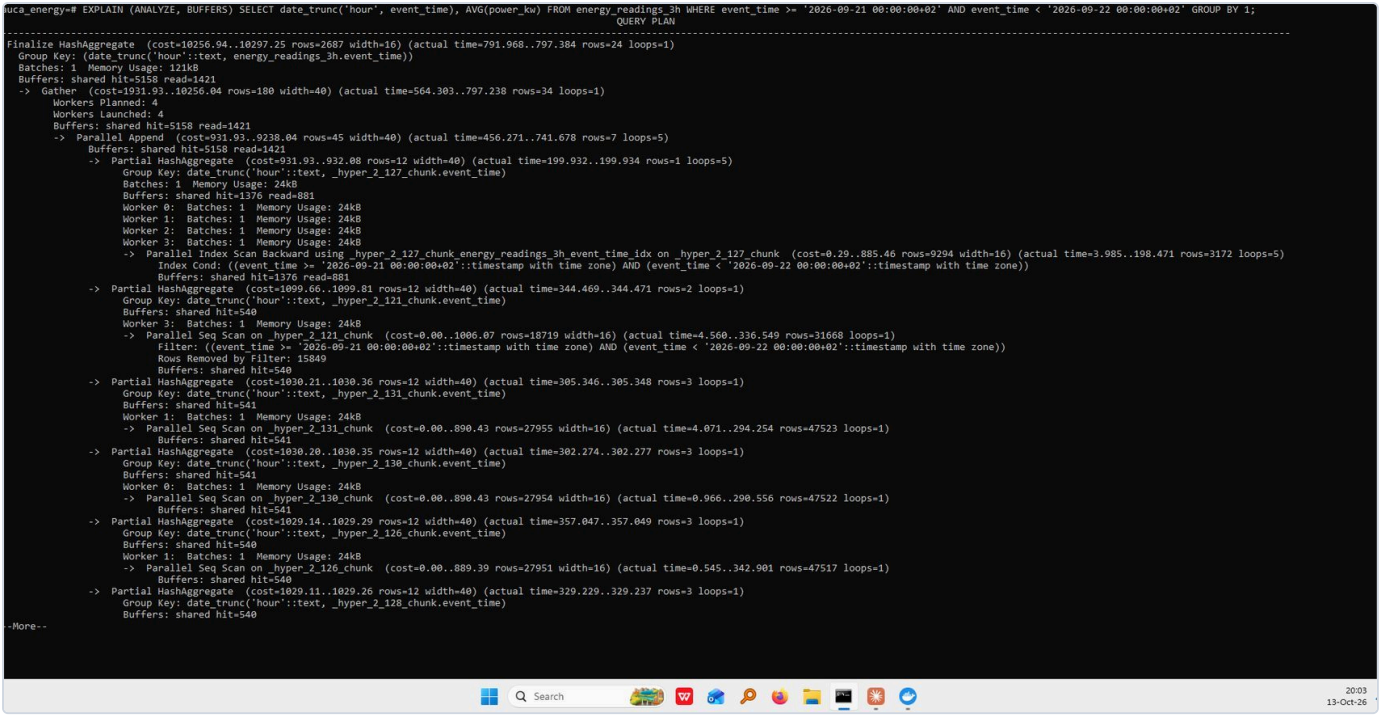
3.2 – completed timing comparison (medians, warm cache) and session time

Execution plans with actual timing and buffer statistics

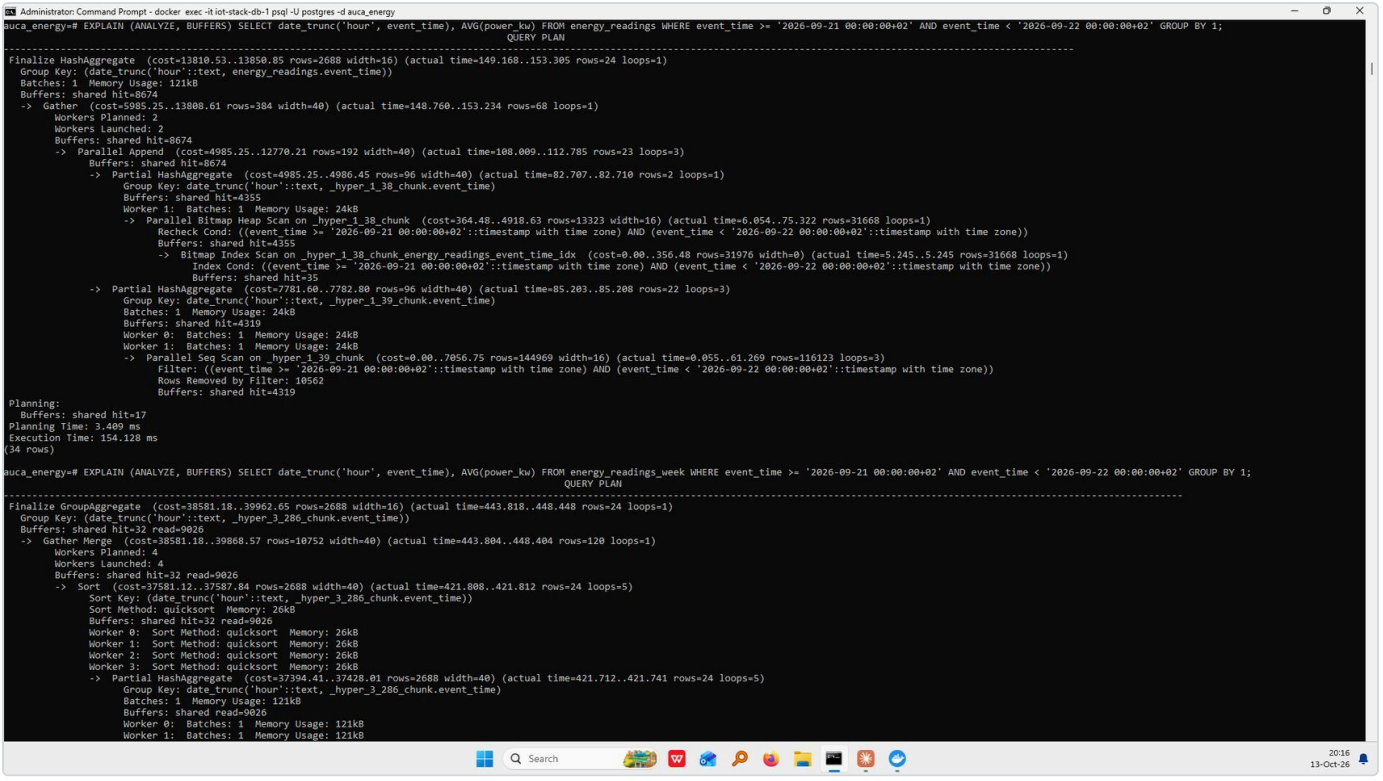
One `EXPLAIN (ANALYZE, BUFFERS)` of Q1 per configuration. These are single executions: the first execution of each plan was cold (reads from disk, e.g. 660 ms for the 1-day table) and a repeat was warm (154 ms), so plan times are not the medians above. The plans show the chunks scanned, parallel workers, scan types and buffers.



3.2 – Q1 plan on energy_readings (1 day): 2 chunks, 2 workers, 660 ms cold (shared read=8673)



3.2 – Q1 plan on energy_readings_3h (3 hours): Parallel Append over many small chunks, 4 workers (output continues past --More--)



3.2 – Q1 plans: energy_readings (1 day) repeated warm (154 ms, shared hit=8674) and energy_readings_week (7 days): one chunk, 4 workers, sort per worker

3.3 Explain the chunk results

Chunks overlapped by each time filter, calculated from the ranges in 3.1: Q1 (21 Sep CAT day) touches **2** one-day chunks (48 h of data), **9** three-hour chunks (27 h) and **1** week chunk (168 h); Q2 and Q4 (22–28 Sep) touch **8** one-day chunks, **57** three-hour chunks and **2** week chunks; Q3 and Q5 have no time filter and touch all **29**, **225** and **5** chunks. The Q1 plans confirm 2 chunks for the one-day table and a single chunk for the week table.

Query	Fastest	Median (ms)	Second	Slowest	Slowest → fastest saving
Q1	energy_readings_3h	50.22	energy_readings_week 68.92 (1.37×)	energy_readings 86.79 (1.73×)	42.1%
Q2	energy_readings_3h	117.63	energy_readings 167.62 (1.42×)	energy_readings_week 194.34 (1.65×)	39.5%
Q3	energy_readings_week	724.93	energy_readings 1,410.76 (1.95×)	energy_readings_3h 3,121.12 (4.31×)	76.8%
Q4	energy_readings	2.33	energy_readings_week 2.85 (1.22×)	energy_readings_3h 4.62 (1.98×)	49.6%
Q5	energy_readings_week	381.59	energy_readings 459.37 (1.20×)	energy_readings_3h 853.98 (2.24×)	55.3%

Q1 – hourly average on 21 September

Observation. 3-hour chunks are fastest: 50.22 ms against 68.92 ms (week, 1.37×) and 86.79 ms (1 day, 1.73×); the 3-hour median is 42.1% lower than the 1-day median.

Explanation. The 3-hour table reads the least surplus data for a 24-hour request (27 h in 9 chunks, against 48 h and 168 h) and its many small chunks give the planner more independent chunks to scan in parallel (the plan shows 4 workers launched against 2 for the 1-day table). The week table scans a single 7-day chunk and needs a sort in every worker (plan: Sort, GroupAggregate, Gather Merge), so it does more work per useful row.

Q2 – top ten 15-minute totals, 22–28 September

Observation. 3-hour chunks are fastest: 117.63 ms against 167.62 ms (1 day, 1.42×) and 194.34 ms (week, 1.65×).

Explanation. The filter selects 7 days. The 3-hour table touches 57 chunks that cover almost exactly the requested 7.1 days, while the week table must work through two chunks that cover 14 days and the 1-day table 8 chunks. Reading closer to the requested range and having more chunks to spread over workers is consistent with the faster result (we did not capture a Q2 plan, so this is an inference from the chunk counts and the Q1 plans).

Q3 – total energy per meter, full period

Observation. 7-day chunks are fastest: 724.93 ms against 1,410.76 ms (1 day, 1.95×) and 3,121.12 ms (3 hours, 4.31×). Run-to-run spread is large for the 1-day and 3-hour tables (657.96–3,655.04 ms and 2,083.34–9,930.13 ms).

Explanation. The query has no time filter, so every chunk is read and every chunk produces its own partial result for up to 4,000 meter groups that must then be merged. With 225 chunks that is up to 900,000 partial groups, with 29 chunks 116,000 and with 5 chunks 20,000. Fewer, larger chunks therefore mean less per-chunk overhead and less merging.

Q4 – one meter, 22–28 September

Observation. The 1-day table is fastest at 2.33 ms, then week at 2.85 ms (1.22×) and 3 hours at 4.62 ms (1.98×). The ranges of the 1-day and week tables overlap (1.94–2.72 ms and 1.78–3.88 ms), so these two cannot be separated reliably; the 3-hour table is clearly slower.

Explanation. The (meter_id, event_time) index makes the work tiny, so a large share of the time is per-chunk overhead: 57 chunks for the 3-hour table against 8 and 2 for the others.

Q5 – full scan and aggregates

Observation. The week table is fastest at 381.59 ms, then 1 day at 459.37 ms (1.20×) and 3 hours at 853.98 ms (2.24×). The 1-day and week ranges overlap (369.43–1,221.61 ms and 349.14–527.14 ms).

Explanation. A full scan reads the same 10.6 million rows in every layout; the difference comes from the number of chunks (225 against 29 and 5) and the cost of starting, planning and combining partial aggregates for each of them.

> **Overall.** Narrow time windows favour smaller chunks (Q1, Q2), full-history scans favour fewer chunks (Q3, Q5), and a single-meter index lookup is almost indifferent except that 225 chunks cost extra (Q4). No chunk interval is best for all five workloads.

4 Compare compression strategies

4.1 Create the two compressed configurations

Chosen chunk interval: 1 day. In 3.2 it was the fastest for the single-meter query (Q4), second-fastest for Q2, Q3 and Q5, and slowest only for Q1; it keeps the chunk count moderate (29 against 225 for three hours, which would produce 225 small compressed chunks), and it lets data be compressed soon after it is 48 hours old (a 7-day chunk would stay uncompressed for up to 9 days).

Configurations (`create_compressed.sql`): `energy_time_compressed` and `energy_meter_compressed` are empty one-day hypertables with the same eight columns and the same indexes as before (`event_time DESC` and `(meter_id, event_time DESC)`). `energy_time_compressed`: `timescaledb.compress`, `compress_orderby = 'event_time DESC'`, no segmentation. `energy_meter_compressed`: the same ordering plus `compress_segmentby = 'meter_id'` (TimescaleDB 2.30.2 compression / columnstore settings).

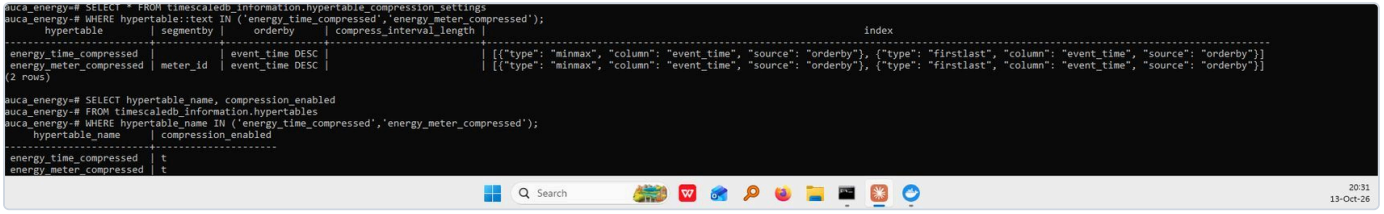
Policies. A compression policy was added to each table with `compress_after = 48 hours` (jobs 1000 and 1001) and **paused** (`scheduled = f`) before any data was loaded; they were resumed after the measurements in 4.3 (screenshot in 4.3).

Cutoff. Experiment start: **2026-10-13 20:20:26.902065+02**. Cutoff = start − 48 h = **2026-10-11 20:20:26.902065+02**, used for both tables. All 29 chunks end on or before 2026-09-29 02:00, before the cutoff, so all 29 were compressed.

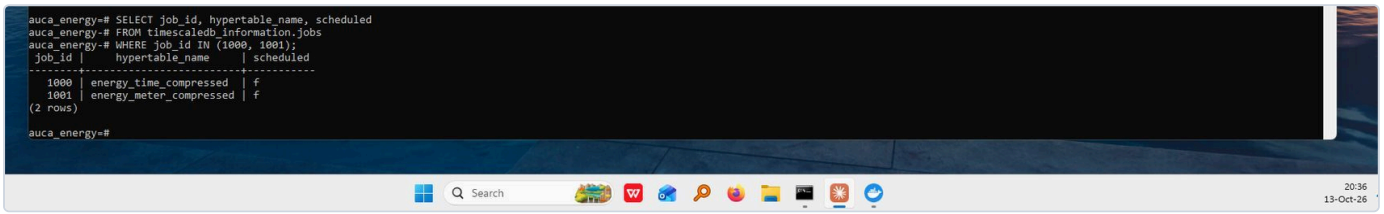
Loading and compression (`load_compress.py`): the cleaned data was copied into each table one CAT day at a time with `INSERT … SELECT`; after a chunk was fully loaded and its end preceded the cutoff it was compressed with `compress_chunk()`. Loading and compression were timed separately (per-chunk values in `compress_timings.csv`).

Table	Load seconds (sum of 28 daily loads)	Chunks compressed	Compression seconds (sum)	Per chunk min / mean / max (s)
energy_time_compressed	90.45	29	36.23	0.13 / 1.25 / 2.59
energy_meter_compressed	110.15	29	60.60	0.93 / 2.09 / 3.39

Data checks. Both compressed tables hold 10,643,735 rows and 4,000 meters between 2026-09-01 00:00:00+02 and 2026-09-28 23:45:00+02; both have 29 chunks, 29 of them compressed, with the same 29 time ranges (a join of the two chunk lists on range_start and range_end returns 29 pairs).



4.1 – compression settings (order by event_time DESC; segment by meter_id for the second table)



4.1 – compression policies added and paused (scheduled = f)

```
auca_energy=# SELECT now() AS experiment_start, now() - INTERVAL '48 hours' AS cutoff;
      experiment_start       |            cutoff
-----------------------------+-------------------------------
 2026-10-13 20:20:26.902065+02 | 2026-10-11 20:20:26.902065+02
(1 row)

auca_energy=# SELECT proname FROM pg_proc
auca_energy-# WHERE proname IN ('convert_to_columnstore','compress_chunk','add_columnstore_policy','add_compression_policy')
auca_energy-# ORDER BY proname;
         proname
-------------------------
 add_columnstore_policy
 add_compression_policy
 compress_chunk
 convert_to_columnstore
```

4.1 – recorded experiment start and the 48-hour cutoff

```
Administrator: Command Prompt
  compressed energy_meter_compressed _timescaledb_internal._hyper_5_306_chunk 2.03 s
loaded CAT day 2026-09-10
  compressed energy_time_compressed _timescaledb_internal._hyper_4_307_chunk 1.72 s
  compressed energy_meter_compressed _timescaledb_internal._hyper_5_308_chunk 3.39 s
loaded CAT day 2026-09-11
  compressed energy_time_compressed _timescaledb_internal._hyper_4_309_chunk 2.24 s
  compressed energy_meter_compressed _timescaledb_internal._hyper_5_310_chunk 2.0 s
loaded CAT day 2026-09-12
  compressed energy_time_compressed _timescaledb_internal._hyper_4_311_chunk 2.03 s
  compressed energy_meter_compressed _timescaledb_internal._hyper_5_312_chunk 2.24 s
loaded CAT day 2026-09-13
  compressed energy_time_compressed _timescaledb_internal._hyper_4_313_chunk 2.59 s
  compressed energy_meter_compressed _timescaledb_internal._hyper_5_314_chunk 2.07 s
loaded CAT day 2026-09-14
  compressed energy_time_compressed _timescaledb_internal._hyper_4_315_chunk 1.34 s
  compressed energy_meter_compressed _timescaledb_internal._hyper_5_316_chunk 2.09 s
loaded CAT day 2026-09-15
  compressed energy_time_compressed _timescaledb_internal._hyper_4_317_chunk 0.96 s
  compressed energy_meter_compressed _timescaledb_internal._hyper_5_318_chunk 2.01 s
loaded CAT day 2026-09-16
  compressed energy_time_compressed _timescaledb_internal._hyper_4_319_chunk 1.26 s
  compressed energy_meter_compressed _timescaledb_internal._hyper_5_320_chunk 1.95 s
loaded CAT day 2026-09-17
  compressed energy_time_compressed _timescaledb_internal._hyper_4_321_chunk 1.33 s
  compressed energy_meter_compressed _timescaledb_internal._hyper_5_322_chunk 1.91 s
loaded CAT day 2026-09-18
  compressed energy_time_compressed _timescaledb_internal._hyper_4_323_chunk 1.14 s
  compressed energy_meter_compressed _timescaledb_internal._hyper_5_324_chunk 2.02 s
loaded CAT day 2026-09-19
  compressed energy_time_compressed _timescaledb_internal._hyper_4_325_chunk 1.13 s
  compressed energy_meter_compressed _timescaledb_internal._hyper_5_326_chunk 2.0 s
loaded CAT day 2026-09-20
  compressed energy_time_compressed _timescaledb_internal._hyper_4_327_chunk 1.12 s
  compressed energy_meter_compressed _timescaledb_internal._hyper_5_328_chunk 2.0 s
loaded CAT day 2026-09-21
  compressed energy_time_compressed _timescaledb_internal._hyper_4_329_chunk 1.57 s
  compressed energy_meter_compressed _timescaledb_internal._hyper_5_330_chunk 2.36 s
loaded CAT day 2026-09-22
  compressed energy_time_compressed _timescaledb_internal._hyper_4_331_chunk 1.02 s
  compressed energy_meter_compressed _timescaledb_internal._hyper_5_332_chunk 2.06 s
loaded CAT day 2026-09-23
  compressed energy_time_compressed _timescaledb_internal._hyper_4_333_chunk 1.14 s
  compressed energy_meter_compressed _timescaledb_internal._hyper_5_334_chunk 2.02 s
loaded CAT day 2026-09-24
  compressed energy_time_compressed _timescaledb_internal._hyper_4_335_chunk 1.0 s
  compressed energy_meter_compressed _timescaledb_internal._hyper_5_336_chunk 1.95 s
loaded CAT day 2026-09-25
  compressed energy_time_compressed _timescaledb_internal._hyper_4_337_chunk 1.1 s
  compressed energy_meter_compressed _timescaledb_internal._hyper_5_338_chunk 1.89 s
loaded CAT day 2026-09-26
  compressed energy_time_compressed _timescaledb_internal._hyper_4_339_chunk 1.94 s
  compressed energy_meter_compressed _timescaledb_internal._hyper_5_340_chunk 1.98 s
loaded CAT day 2026-09-27
  compressed energy_time_compressed _timescaledb_internal._hyper_4_341_chunk 0.94 s
  compressed energy_meter_compressed _timescaledb_internal._hyper_5_342_chunk 2.14 s
loaded CAT day 2026-09-28
  compressed energy_time_compressed _timescaledb_internal._hyper_4_343_chunk 0.92 s
  compressed energy_meter_compressed _timescaledb_internal._hyper_5_344_chunk 1.95 s
  compressed energy_time_compressed _timescaledb_internal._hyper_4_345_chunk 0.58 s
  compressed energy_meter_compressed _timescaledb_internal._hyper_5_346_chunk 2.15 s
Total compression seconds energy_time_compressed 36.23 chunks: 29
Total compression seconds energy_meter_compressed 60.6 chunks: 29
```

4.1 – load_compress.py: each chunk compressed after loading with its own timing; totals 36.23 s and 60.6 s, 29 chunks each

```
auca_energy=# SELECT 'energy_meter_compressed' AS tbl, COUNT(*) AS n, COUNT(DISTINCT meter_id) AS meters, MIN(event_time) AS earliest, MAX(event_time) AS latest, SUM(power_kw) AS sum_power, SUM(energy_kwh) A
S sum_energy FROM energy_meter_compressed;
          tbl           |    n     | meters |       earliest        |        latest         |     sum_power      |     sum_energy
------------------------+----------+--------+-----------------------+-----------------------+--------------------+--------------------
 energy_meter_compressed | 10643735 |   4000 | 2026-09-01 00:00:00+02 | 2026-09-28 23:45:00+02 | 15414817.387792999 | 3853704.3468154133
(1 row)

auca_energy=# SELECT hypertable_name, COUNT(*) AS chunks, COUNT(*) FILTER (WHERE is_compressed) AS compressed
auca_energy-# FROM timescaledb_information.chunks
auca_energy-# WHERE hypertable_name IN ('energy_time_compressed','energy_meter_compressed')
auca_energy-# GROUP BY hypertable_name;
     hypertable_name     | chunks | compressed
-------------------------+--------+------------
 energy_meter_compressed |     29 |         29
 energy_time_compressed  |     29 |         29
(2 rows)

auca_energy=# ANALYZE energy_time_compressed;
ANALYZE
auca_energy=# ANALYZE energy_meter_compressed;
ANALYZE
auca_energy=#
```

4.1 – data check and compressed-chunk list for energy_meter_compressed

Recorded output (psql, 15 October 2026 19:21–19:24 CAT) for the table list, indexes, compressed-chunk lists, identical ranges and the data check of energy_time_compressed:

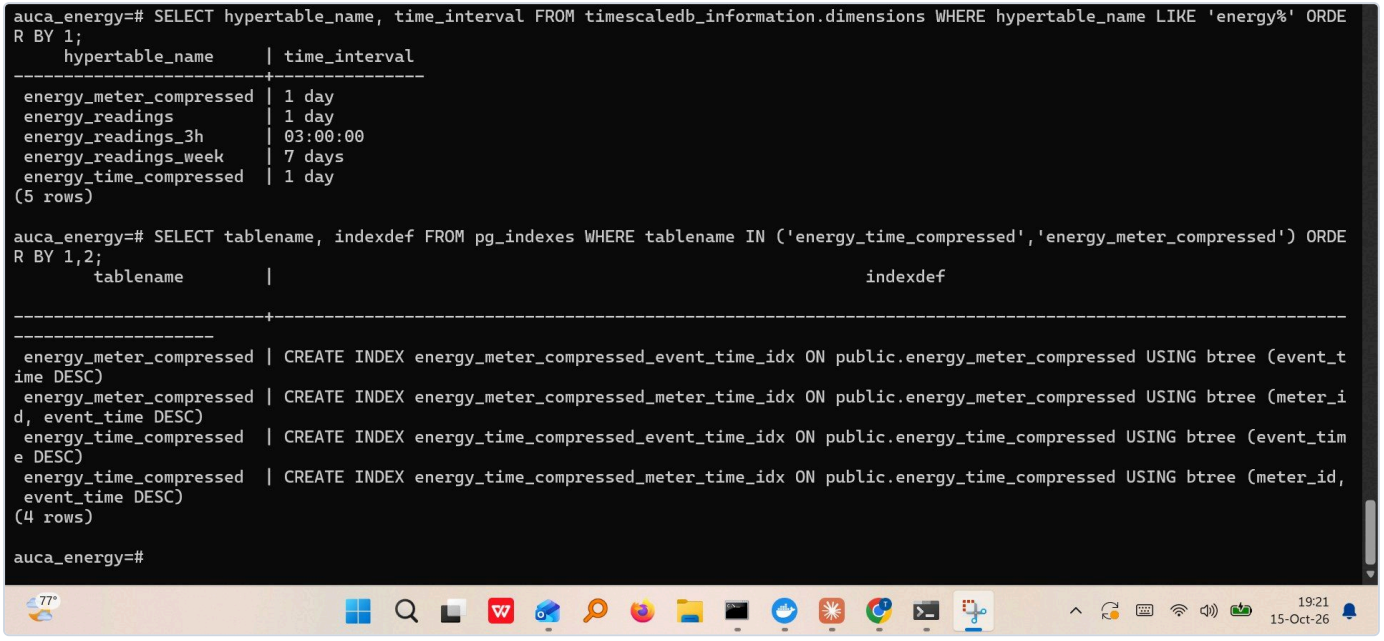
```
auca_energy=# SELECT hypertable_name, time_interval FROM timescaledb_information.dimensions WHERE hypertable_name LIKE 'e
nergy%' ORDER BY 1;
     hypertable_name     | time_interval
 energy_meter_compressed | 1 day
 energy_readings         | 1 day
 energy_readings_3h      | 03:00:00
 energy_readings_week    | 7 days
 energy_time_compressed  | 1 day

auca_energy=# SELECT tablename, indexdef FROM pg_indexes WHERE tablename IN ('energy_time_compressed','energy_meter_compr
essed') ORDER BY 1,2;
 energy_meter_compressed | ... (event_time DESC)           energy_meter_compressed | ... (meter_id, event_time DESC)
 energy_time_compressed  | ... (event_time DESC)           energy_time_compressed  | ... (meter_id, event_time DESC)

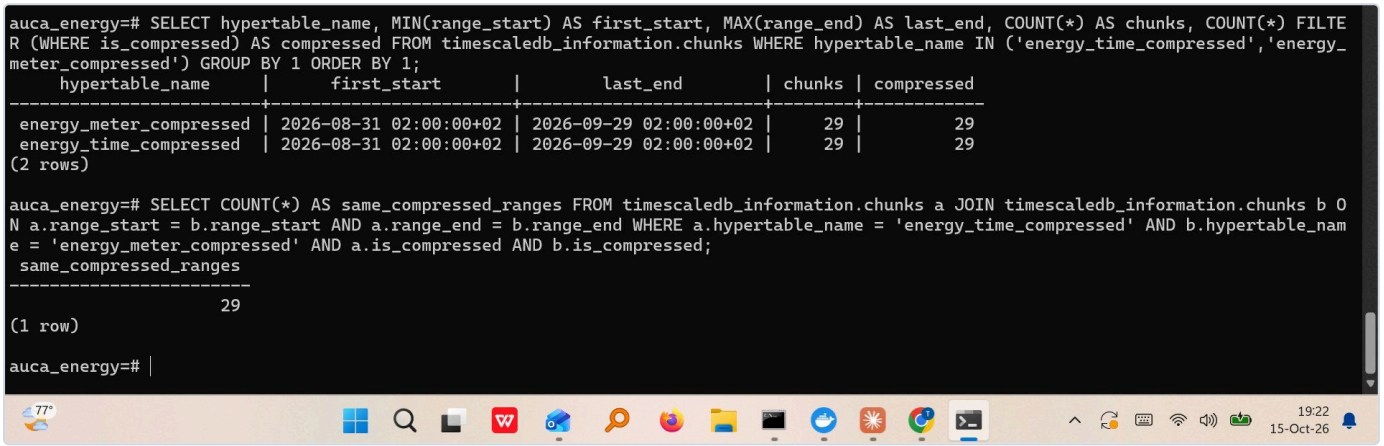
     hypertable_name     |      first_start       |        last_end        | chunks | compressed
 energy_meter_compressed | 2026-08-31 02:00:00+02 | 2026-09-29 02:00:00+02 |     29 |         29
 energy_time_compressed  | 2026-08-31 02:00:00+02 | 2026-09-29 02:00:00+02 |     29 |         29

 same_compressed_ranges
                    29

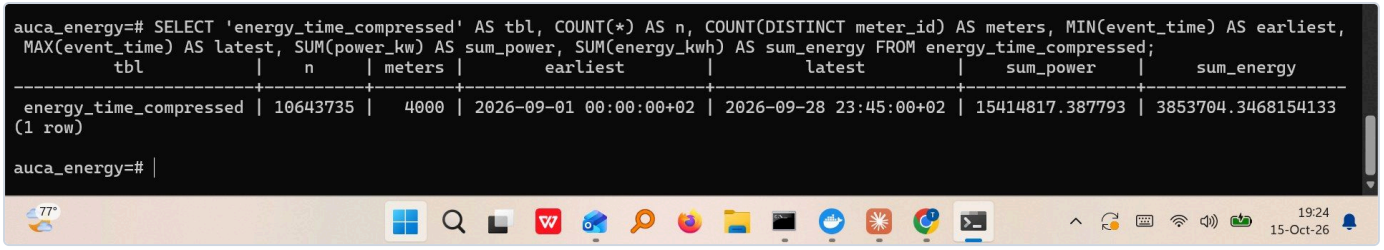
          tbl           |    n     | meters |        earliest        |         latest         |    sum_power    |     sum
_energy
 energy_time_compressed | 10643735 |   4000 | 2026-09-01 00:00:00+02 | 2026-09-28 23:45:00+02 | 15414817.387793 | 385370
4.3468154133
```



4.1 – the compressed hypertables: chunk intervals (1 day) and the two btree indexes on each table (15 October 2026, 19:21)



4.1 – 29 of 29 chunks compressed in both tables, and the join showing 29 identical compressed ranges (15 October 2026, 19:22)



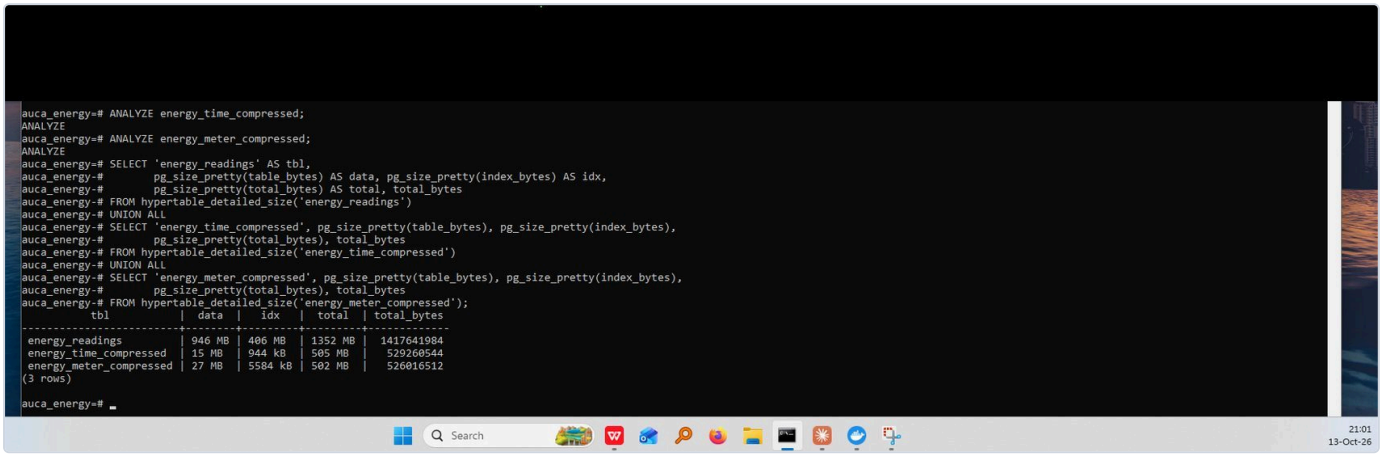
4.1 – data check of energy_time_compressed: 10,643,735 rows, 4,000 meters, matching energy sum (15 October 2026, 19:24)

4.2 Measure storage and compression cost

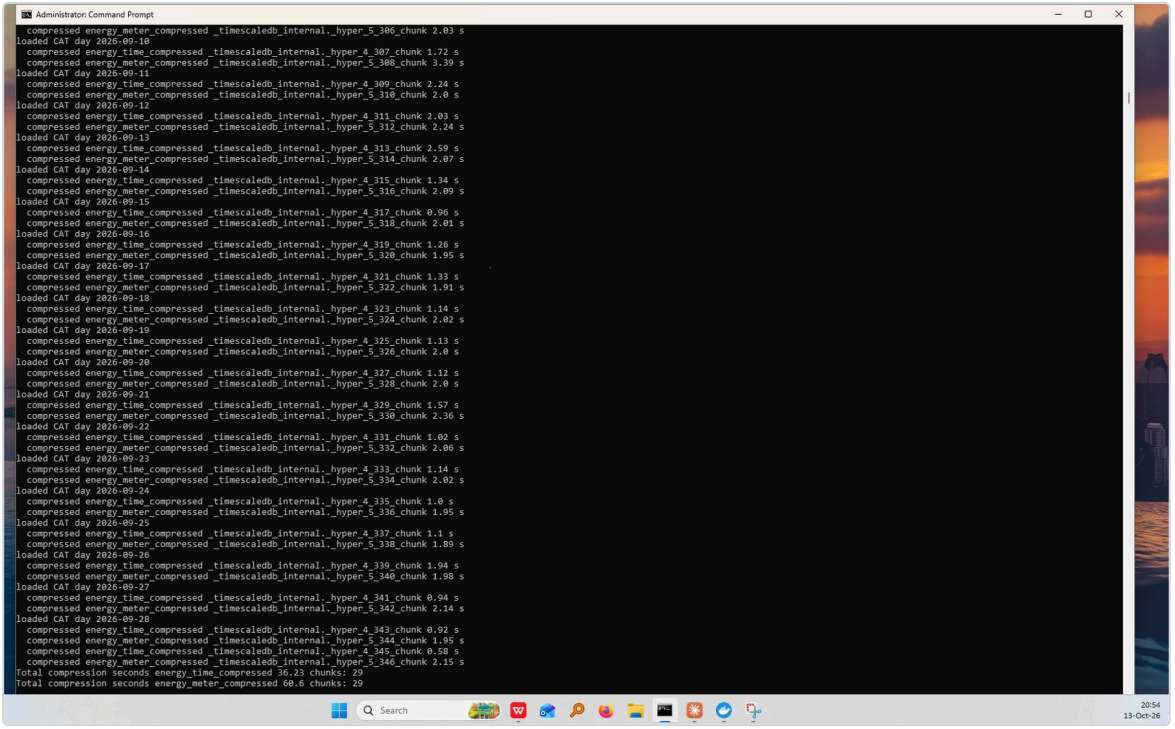
Table	Data	Index	Total (all chunks)	Compression ratio	Saved	Compression time (sum)	Compressed chunks	Rows in compressed chunks
energy_readings (baseline, uncompressed)	946 MB	406 MB	1,352 MB (1,417,641,984 B)	–	–	–	0 / 29	0%
energy_time_compressed	15 MB	944 kB	505 MB (529,260,544 B)	2.68×	62.7%	36.23 s	29 / 29	100%
energy_meter_compressed	27 MB	5,584 kB	502 MB (526,016,512 B)	2.70×	62.9%	60.60 s	29 / 29	100%

Compression ratio = baseline total / compressed total: **2.68×** (time-ordered, no segmentation) and **2.70×** (segmented by meter_id). Percentage saved = 100 × (baseline − compressed) / baseline = **62.7%** and **62.9%**. Compression cost = the sum of the 29 measured chunk-compression times, excluding loading: **36.23 s** and **60.60 s**. All 29 of 29 chunks are compressed in both tables, and the rows held in compressed chunks are 10,643,735 / 10,643,735 = **100%** of the cleaned rows. The baseline is the uncompressed one-day table `energy_readings`, measured with all chunks and both indexes (the 406 MB of indexes is large because there are two B-tree indexes on 10.6 million rows).

> The Data and Index columns of `hypertable_detailed_size` do not include TOAST storage, which holds most of the compressed column batches for the two compressed tables; `total_bytes` does include it, so the ratios use the totals.



4.2 – storage statistics: data, index and total bytes of the baseline and both compressed tables



4.2 – compression timings per chunk and the two totals (36.23 s and 60.6 s), as in 4.1

4.3 Repeat the query comparison

Q1 to Q5 were run on both compressed tables with the same procedure and correctness check as 3.2 (one warm-up, five measured runs, results compared with the baseline: all five queries returned **True**), in a separate session on 13 October 2026 at about 21:08 CAT (`benchmark_compressed.py`, session time **28.1 s**). The uncompressed baseline (`energy_readings`, 1 day) was re-measured inside this same session so improvements compare like with like; its medians are lower than the 3.2 values (for example Q3 470.80 ms against 1,410.76 ms), so the two sessions must not be mixed in the improvement calculation. A likely reason, which we did not test directly, is cache pressure: the three uncompressed tables of 3.2 (about 4 GB together) exceeded the roughly 3.7 GiB of memory available to Docker and evicted each other, whereas the baseline plus the two compressed tables of 4.3 (about 2.4 GB) fit.

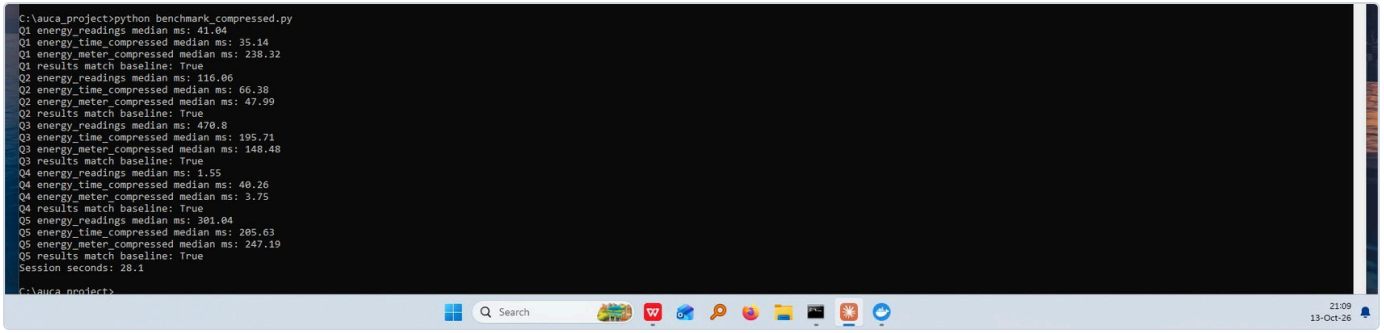
All five table configurations in one comparison (median ms)

Query	1 day (3.2 session)	3 hours (3.2 session)	7 days (3.2 session)	1 day uncompressed (4.3 session baseline)	energy_time_ compressed	energy_meter_ compressed
Q1	86.79	50.22	68.92	41.04	35.14	238.32
Q2	167.62	117.63	194.34	116.06	66.38	47.99
Q3	1,410.76	3,121.12	724.93	470.80	195.71	148.48
Q4	2.33	4.62	2.85	1.55	40.26	3.75
Q5	459.37	853.98	381.59	301.04	205.63	247.19

Compressed-table timings with minimum and maximum (ms) and percentage improvement over the baseline

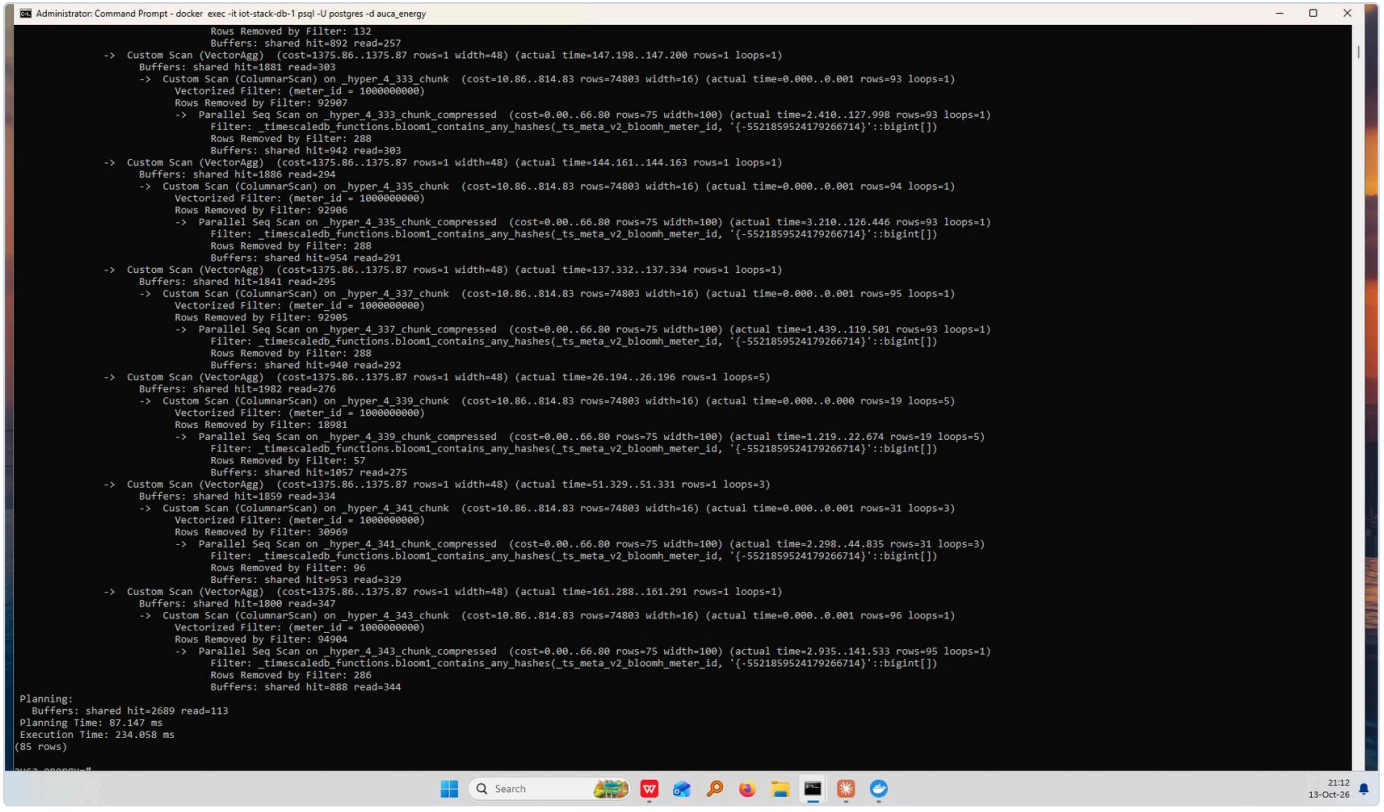
Query	Baseline median (min–max)	Time-compressed median (min–max)	Improvement	Meter-compressed median (min–max)	Improvement
Q1	41.04 (40.25–49.91)	35.14 (34.69–45.61)	**+14.4%**	238.32 (190.59–339.19)	**-480.7%**
Q2	116.06 (99.28–132.81)	66.38 (60.30–71.70)	**+42.8%**	47.99 (47.71–49.30)	**+58.7%**
Q3	470.80 (431.00–1,804.26)	195.71 (177.16–255.61)	**+58.4%**	148.48 (137.64–184.10)	**+68.5%**
Q4	1.55 (1.40–2.15)	40.26 (38.04–55.37)	**-2,497.4%**	3.75 (2.73–4.30)	**-141.9%**
Q5	301.04 (289.41–734.35)	205.63 (196.11–256.59)	**+31.7%**	247.19 (217.96–284.94)	**+17.9%**

Improvement = 100 × (baseline median − compressed median) / baseline median; positive means faster, negative means slower.

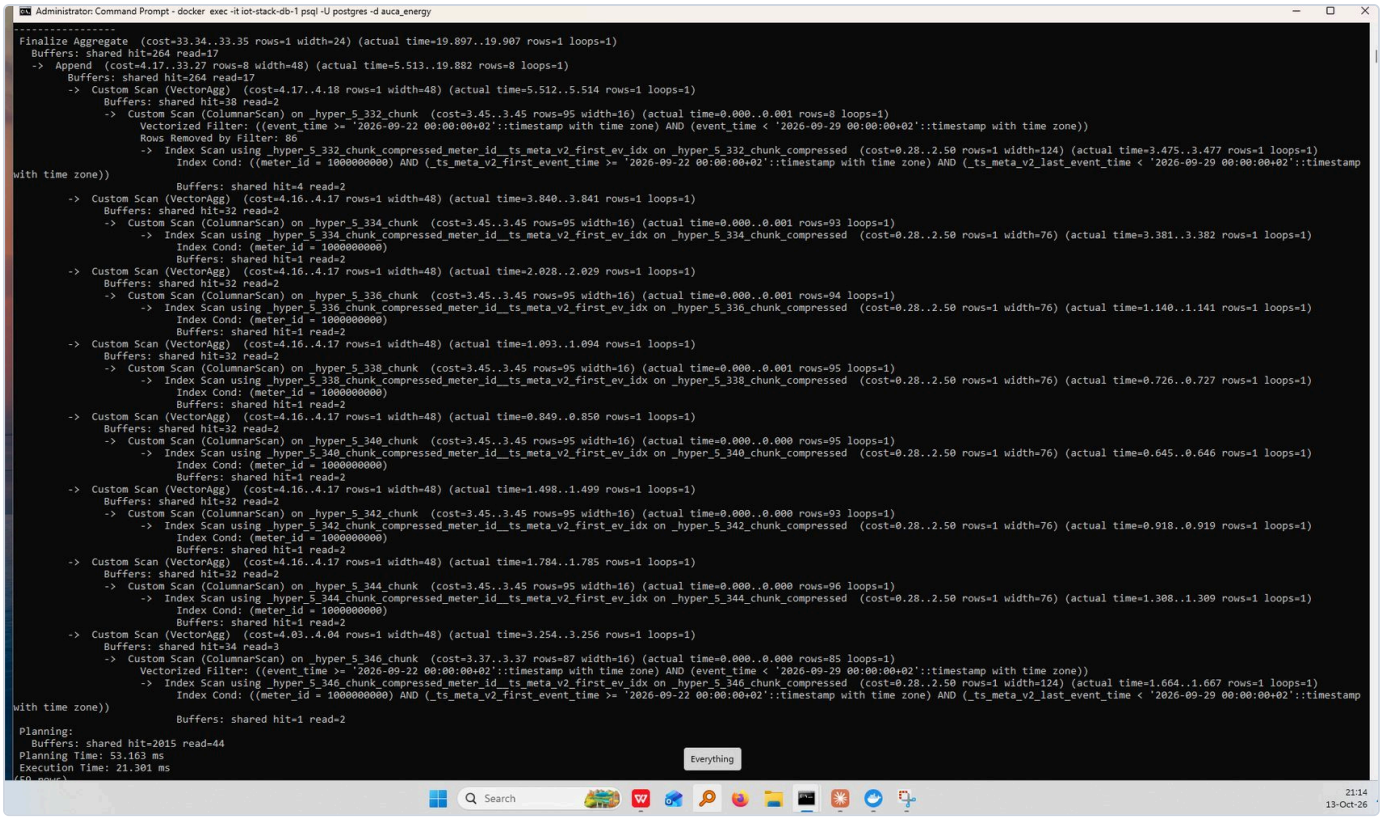


4.3 – completed comparison of the baseline and both compressed tables, results match, session 28.1 s

Actual execution plans (Q4). The screenshots show the end of `EXPLAIN (ANALYZE, BUFFERS)` of Q4 on each compressed table (first, cold executions; the window title shows the psql session). The time-ordered table has to read every compressed batch of every chunk: a bloom-filter on meter_id lets about a quarter of the batches through (e.g. 93 batches read, 288 skipped per chunk), these batches are decompressed and about 92,900 rows per chunk are removed by the vectorized filter, giving 234 ms in this cold run. The meter-segmented table uses an index scan on the segment column and decompresses a single batch (rows = 1, about 95 readings) per chunk, giving 21 ms in this cold run.



4.3 – Q4 plan on energy_time_compressed (no segmentation): bloom filter, many batches decompressed, execution 234.058 ms



4.3 – Q4 plan on energy_meter_compressed (segmented by meter_id): index scan, one batch per chunk, execution 21.301 ms

Policies resumed after measurement (compress_after 48 hours, jobs 1000 and 1001 scheduled again, 13 October 2026 21:20 CAT):



4.3 – both compression policies resumed and enabled

4.4 Select a storage configuration

Effect of meter segmentation

> **Observation. Single-meter workload (Q4):** segmentation by meter_id cut the median from 40.26 ms (time-ordered) to 3.75 ms, 10.7× faster; both compressed tables are still slower than the uncompressed indexed baseline (1.55 ms), by 2.4× for segmented and 26× for time-ordered. **Workloads across all meters:** segmentation was faster than time-ordering for Q2 (47.99 vs 66.38 ms) and Q3 (148.48 vs 195.71 ms), slower for Q5 (247.19 vs 205.63 ms) and much slower for Q1 (238.32 vs 35.14 ms, 6.8×). Storage is almost identical (502 MB vs 505 MB).

> **Explanation.** With segmentation every batch holds one meter's readings, so a meter filter finds its batches through an index (one batch per chunk in the Q4 plan). But a one-day chunk holds only about 96 readings per meter, so each batch is tiny (about 95 rows); a query that needs one day of every meter (Q1) must open about 4,000 small batches per chunk and pays a per-batch cost, and this also explains why the segmented table saves almost no extra space. Time-ordered batches of about 1,000 rows each mix meters, which suits all-meter, time-filtered scans (Q1) but forces a single-meter query to read and filter every batch (Q4). These reasons are inferred from the plans and sizes; we did not count batches directly.

Recommendation

Chosen configuration: one-day chunks with compression segmented by meter_id (order by event_time DESC), applied automatically by the policy to chunks older than 48 hours (recent data stays uncompressed and fast to write). **Evidence:** total size falls from 1,352 MB to 502 MB (2.70×, 62.9% saved), and against the same-session baseline the segmented table improves Q2 by 58.7% and Q3 by 68.5% and keeps the single-meter query at 3.75 ms (versus 40.26 ms without segmentation). **Trade-off:** compression took 60.6 s against 36.2 s for the unsegmented table (+67%, about 24 s once), and the segmented table is much slower for one-day all-meter queries such as Q1 (238.32 ms vs 41.04 ms), which is why such hourly/daily summaries should be served from the continuous aggregates of Section 6 instead of from compressed raw data.

5 Investigate the utility data

Sections 5 to 8 use the cleaned `energy_readings` (10,643,735 rows) and `meters`, grouped by `event_time` (CAT), except the separate test in 6.2. The queries are in `analysis_queries.sql`; the charts come from `chart_5_1.py`.

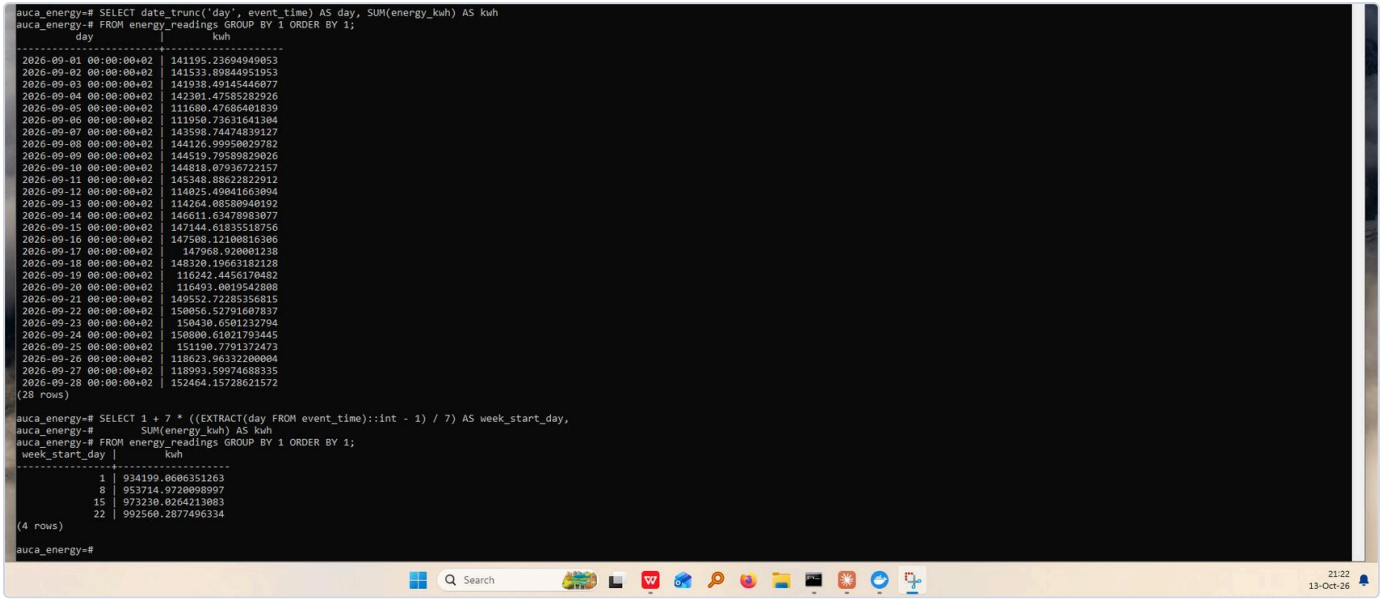
5.1 Identify demand patterns

Total network energy (kWh) per CAT day and per seven-day period

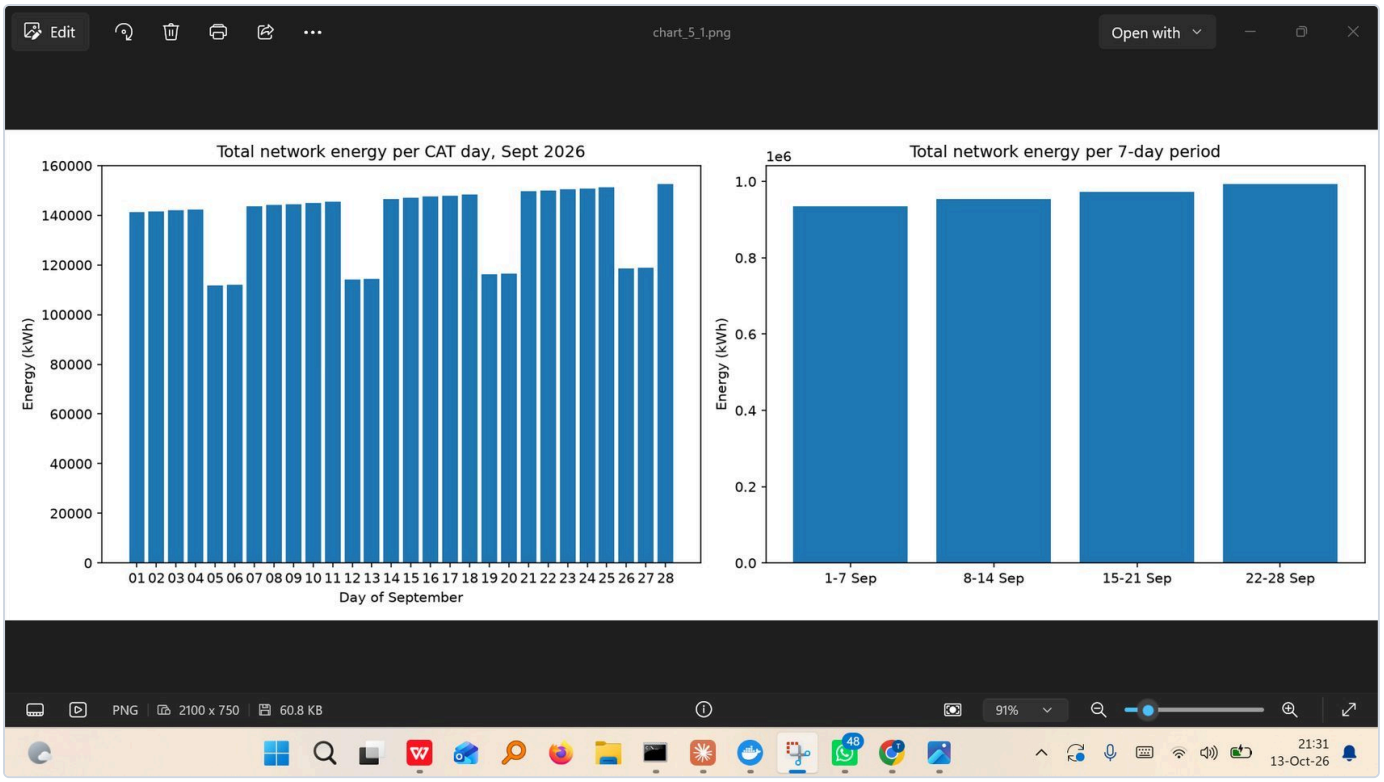
Period	Day 1	Day 2	Day 3	Day 4	Day 5	Day 6	Day 7	Seven-day total (kWh)
1–7 Sep	141,195	141,534	141,938	142,301	*111,680*	*111,951*	143,599	**934,199.06**
8–14 Sep	144,127	144,520	144,818	145,349	*114,025*	*114,264*	146,612	**953,714.97**
15–21 Sep	147,145	147,508	147,969	148,320	*116,242*	*116,493*	149,553	**973,230.03**
22–28 Sep	150,057	150,431	150,801	151,191	*118,624*	*118,994*	152,464	**992,560.29**

Italic values are Saturdays and Sundays (5, 6, 12, 13, 19, 20, 26 and 27 September). Each row's days are the calendar days 1–7, 8–14, 15–21 and 22–28 September. Energy is the sum of `energy_kwh`.

> **Observation.** Weekdays use 141,195–152,464 kWh per day (mean 146,572), weekends 111,680–118,994 kWh (mean 115,284), about 21% lower. The seven-day totals rise every week: 2.09%, 2.05% and 1.99% week on week, and the weekday level climbs steadily from 141,195 kWh (1 Sep) to 152,464 kWh (28 Sep, +8.0%).



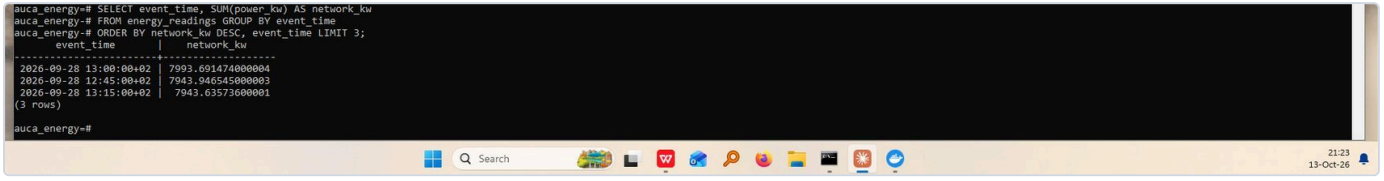
5.1 – executed queries: daily totals (28 rows) and the four seven-day totals



5.1 – charts: total network energy per CAT day and per seven-day period (chart_5_1.py)

Three highest 15-minute network demand intervals

Rank	event_time (interval start, CAT)	Network demand (kW)	Equivalent energy in the interval (kWh)
1	2026-09-28 13:00	7,993.69	1,998.42
2	2026-09-28 12:45	7,943.95	1,985.99
3	2026-09-28 13:15	7,943.64	1,985.91



5.1 – executed query for the three highest 15-minute network demand intervals (sum of power_kw across meters per event_time)

> **Observation.** The three highest intervals are consecutive quarter-hours (12:45, 13:00 and 13:15) on the last day of the data, Monday 28 September. The hourly profiles in the dashboards and the forecast plot (Region 1) show a daily trough at night and a midday peak at about 13:00 on weekdays.

> **Explanation.** The midday peak together with the 21% weekday/weekend gap suggests demand driven by commercial and industrial activity during working hours, and the upward weekly trend explains why the highest peaks fall at the end of the period. **Operational implications:** generation, reserve margin and network capacity must be sized for the roughly 13:00 weekday peak and re-checked as the trend grows; demand response or tariffs that shift load away from 12:45–13:15 reduce the peak, and weekend and night periods are the low-load windows for maintenance.

kW versus kWh. kW (`power_kw`) is a rate, the demand at a moment or its average over an interval; it determines the capacity the network must supply. kWh (`energy_kwh`) is the energy used over time, kW × hours (here kW × 0.25 h per reading), and determines fuel, purchase cost and billing. The 7,993.69 kW peak lasted one 15-minute interval and corresponds to only about 1,998 kWh, whereas a whole day uses about 150,000 kWh. A network can have a modest daily energy total and still need a large capacity because of a short peak.

5.2 Compare regions and customer types

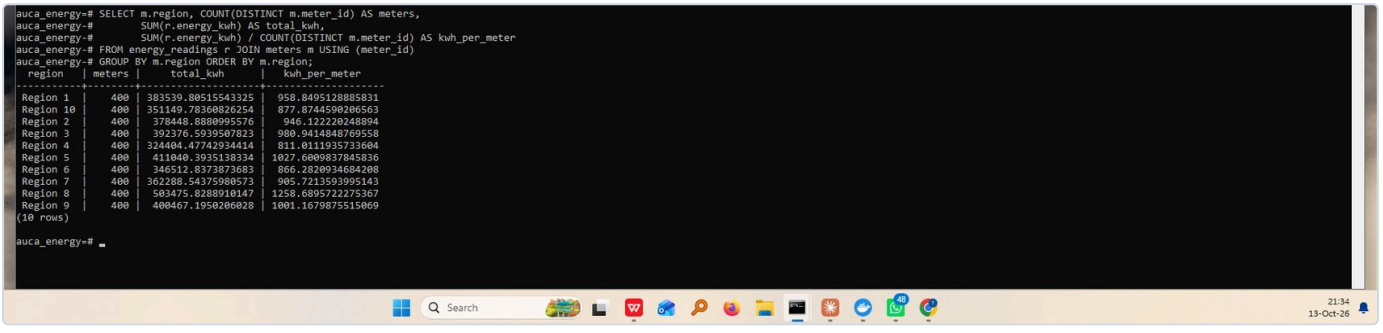
Readings were joined to `meters` on `meter_id`. Energy per meter = total energy over the 28 days divided by the group's registered meter count.

Region	Meters	Total energy (kWh)	Energy per meter (kWh)	Commercial / industrial / residential meters	Per-meter energy expected from the customer mix (kWh)
Region 1	400	383,539.81	958.85	116 / 37 / 247	935.94
Region 2	400	378,448.89	946.12	98 / 39 / 263	939.01
Region 3	400	392,376.59	980.94	94 / 46 / 260	1,024.21
Region 4	400	324,404.48	811.01	94 / 29 / 277	805.04
Region 5	400	411,040.39	1,027.60	94 / 48 / 258	1,050.00
Region 6	400	346,512.84	866.28	105 / 32 / 263	857.60
Region 7	400	362,288.54	905.72	103 / 35 / 262	893.75
Region 8	400	503,475.83	1,258.69	105 / 61 / 234	1,231.49
Region 9	400	400,467.20	1,001.17	96 / 45 / 259	1,013.84
Region 10	400	351,149.78	877.87	105 / 34 / 261	883.38

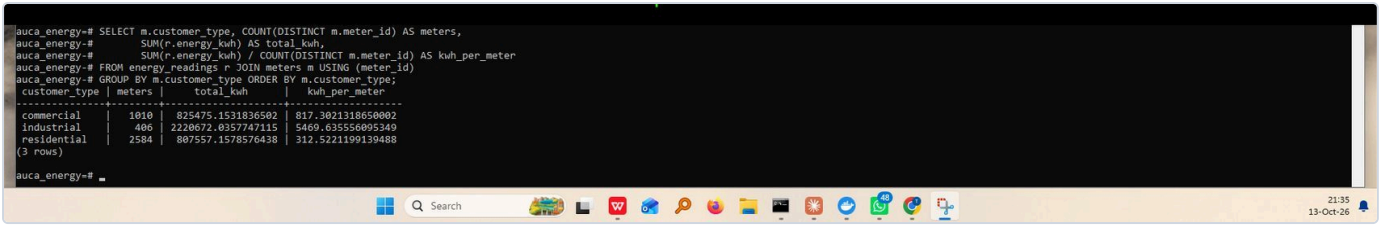
Customer type	Meters	Total energy (kWh)	Energy per meter (kWh)	Share of meters	Share of energy
commercial	1,010	825,475.15	817.30	25.2%	21.4%
industrial	406	2,220,672.04	5,469.64	10.2%	57.6%
residential	2,584	807,557.16	312.52	64.6%	21.0%

Observation. Every region has 400 meters, so per-meter values are the totals divided by 400 and the ranking is unchanged: Region 8 is highest at 1,258.69 kWh per meter and Region 4 lowest at 811.01 kWh per meter (55% higher). By customer type, an industrial meter uses 5,470 kWh, 17.5× a residential meter (313) and 6.7× a commercial meter (817); the 406 industrial meters (10.2% of meters) use 57.6% of all energy.

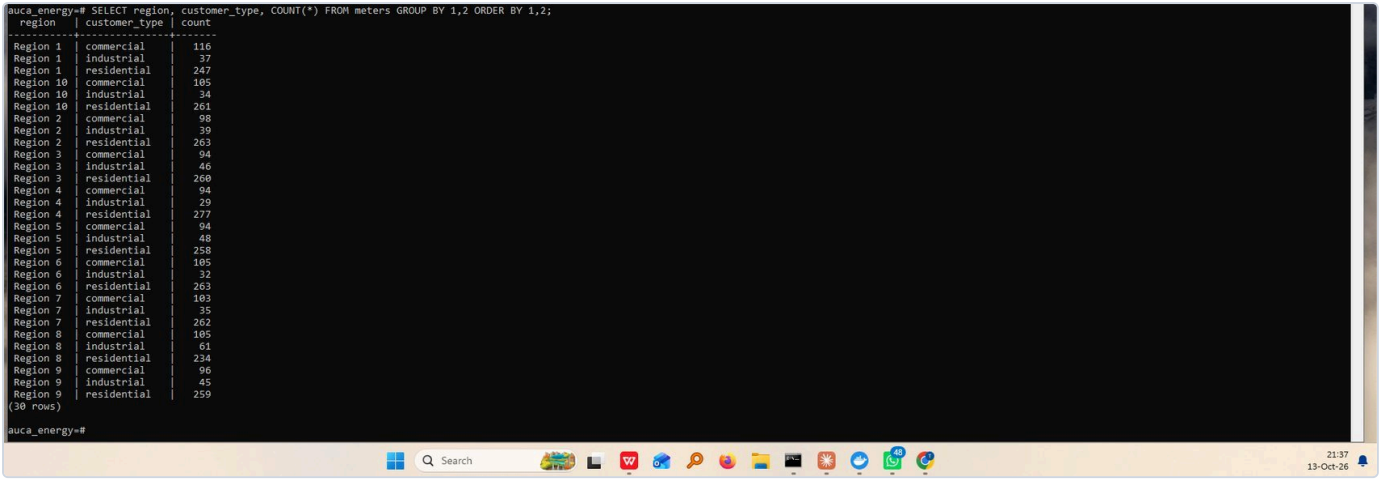
Explanation. The differences between regions remain after normalising by meter count, and they follow the customer mix: Region 8 has the most industrial meters (61) and Region 4 the fewest (29); the correlation between industrial meters per region and energy per meter is r = 0.97. Predicting each region's energy per meter only from its mix and the network-wide per-meter value of each customer type gives values within 4.4% of the measured ones, so little regional difference is left once the mix is accounted for. **Implications:** compare regions by customer mix, not by meter count alone; load forecasting, network reinforcement and tariffs should be set per customer segment, and the small number of industrial meters (especially in Region 8) deserve individual monitoring because they dominate energy and peak demand.



5.2 – energy totals, meter counts and energy per meter by region



5.2 – energy totals, meter counts and energy per meter by customer type



5.2 – meter counts by region and customer type (30 rows)

5.3 Assess reporting quality

Region	Observed unique readings	Planned (meters × 28 × 96)	Completeness (%)
Region 1	1,064,513	1,075,200	99.0060
Region 2	1,064,419	1,075,200	98.9973
Region 3	1,064,327	1,075,200	98.9887
Region 4	1,064,441	1,075,200	98.9993
Region 5	1,064,290	1,075,200	98.9853
Region 6	1,064,118	1,075,200	98.9693
Region 7	1,064,237	1,075,200	98.9804
Region 8	1,064,587	1,075,200	99.0129
Region 9	1,064,388	1,075,200	98.9944
Region 10	1,064,415	1,075,200	98.9969
Whole network	**10,643,735**	**10,752,000**	**98.9931**

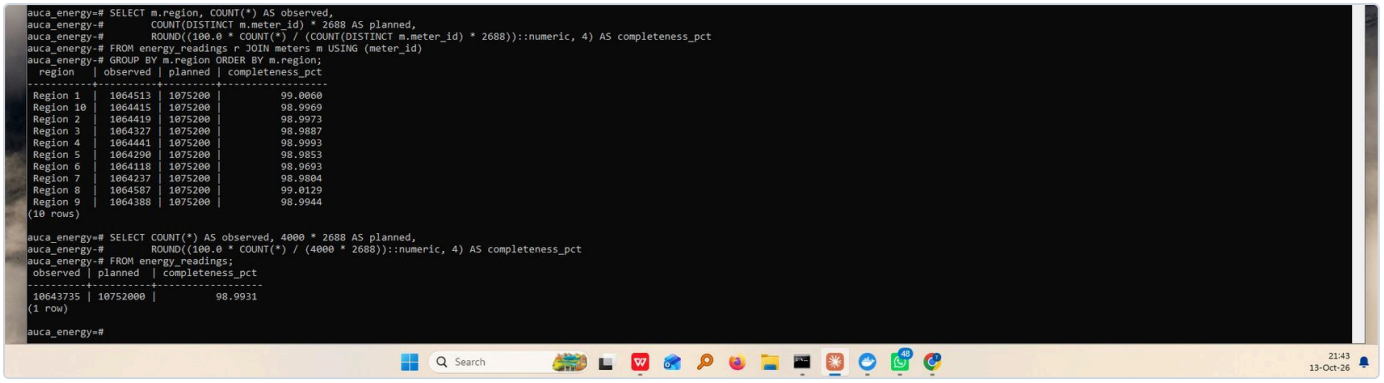
Planned per region = 400 meters × 28 days × 96 = 1,075,200; whole network = 4,000 × 2,688 = 10,752,000.

> **Observation.** Completeness is 98.9931% for the network and between 98.9693% (Region 6) and 99.0129% (Region 8) for the regions: a narrow range of 0.04 percentage points, so no region is reported much worse than the others.

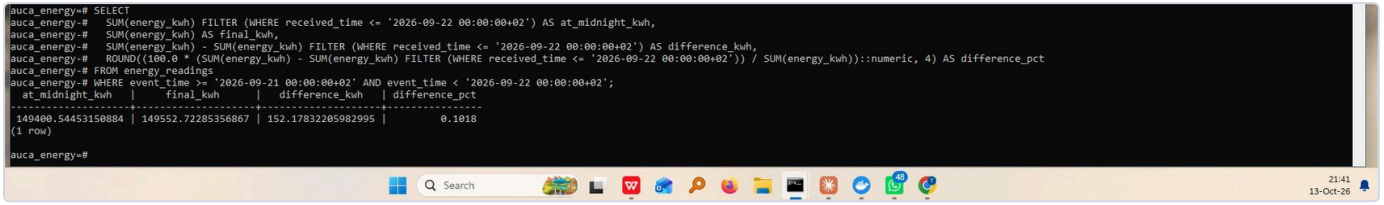
Energy on 21 September: preliminary and final totals

Total for readings with event_time on 21 September	kWh
Using only readings received at or before 00:00 on 22 September	149,400.5445
Using all arrivals (final total)	149,552.7229
Difference	152.1783 kWh = **0.1018%** of the final total

> **Explanation. Delayed readings** (2.0% of readings arrive 30–180 minutes late) only matter for a daily total when they cross midnight: a late reading from the last three hours of 21 September arrives after 00:00 on 22 September, so the midnight total is slightly low (0.10%). As a rough check, if delays are spread evenly the share crossing midnight is about 105 min / 1,440 min ≈ 7% of the delayed readings, 2.0% × 7% ≈ 0.15% of readings, the same order as the measured 0.10%. **Missing readings** (about 1.0%) never arrive, so they are absent from both totals; the final 149,552.72 kWh is itself an under-count, by roughly 1% (about 1,500 kWh) if missing intervals resemble observed ones. Delays are corrected by waiting; missing data are not, and must be reported alongside the total.



5.3 – completeness per region and for the whole network



5.3 – 21 September total at midnight and final total, with the difference in kWh and per cent

5.4 Investigate abnormal consumption

Rule (`anomaly_detect.py`). For every meter, every 15-minute slot of the day (96 slots) and day type (weekday or weekend), compute the **median** `power_kw` over the 28 days. A reading is flagged when `power_kw > 2.0 × median` or `power_kw < 0.25 × median`. Each flag stores meter_id, event_time, power_kw and the median. The rule was applied to the cleaned readings before the labels were loaded; `anomaly_events()` was used only to evaluate it.

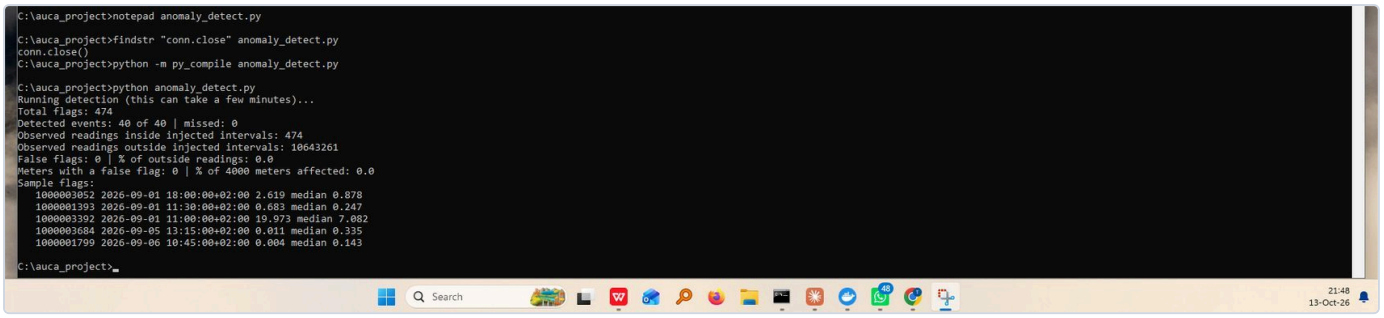
Reasons for the choices. The median is robust: the injected events are a tiny fraction of the readings, so they hardly move it, unlike a mean. Comparing a reading with the same meter, the same time of day and the same day type removes the daily load shape (night versus midday), the weekend drop of about 21% and the large differences between residential, commercial and industrial meters. The 2.0× and 0.25× thresholds are deliberately wide, so ordinary variation and the 2% weekly growth do not trigger flags, while a doubling or a collapse of load does.

Measure	Result
Flags raised	474
Injected events detected (at least one flag with the event's meter_id inside [start_time, end_time))	**40 of 40**
Injected events missed	**0**
Observed readings inside injected intervals	474 (all 474 are flagged)
Observed readings outside injected intervals	10,643,261
False flags (flagged readings outside every injected interval of that meter)	**0** = 0.0% of the 10,643,261 readings outside injected intervals
Meters with at least one false flag	0 → **0.0%** of 4,000 meters affected by false flags

meter_id	event_time	power_kw	meter's median for that slot	ratio
1000003052	2026-09-01 18:00	2.619	0.878	3.0×
1000001393	2026-09-01 11:30	0.683	0.247	2.8×
1000003392	2026-09-01 11:00	19.973	7.082	2.8×
1000003684	2026-09-05 13:15	0.011	0.335	0.03×
1000001799	2026-09-06 10:45	0.004	0.143	0.03×

The sample shows both kinds of event: surges to about three times the normal level and collapses to a few per cent of it.

> **Explanation. No event was missed and there were no false alarms**, and every observed reading inside an injected interval was flagged. The injected events are large (about 2.8× or about 0.03× the normal value), well outside the thresholds, and the normal readings stay inside 0.25–2.0× their slot median. A missed event would arise from a subtle deviation (for example +50%) or from an event whose readings are all missing or late; false alarms would arise from genuine but unusual behaviour (holidays, new customers, a changed schedule), which this data set does not contain. **Limitations:** the medians are computed from all 28 days (including future data), so the rule is an offline analysis; a live system would need a rolling baseline, and the result says little about smaller real faults.



5.4 – executed detection: total flags, detected and missed events, false flags, percentages and sample flags

5.5 Management recommendation

Operational actions.

1. Plan for the weekday midday peak. The three highest 15-minute network demands were 7,993.69 kW (28 Sep 13:00), 7,943.95 kW (12:45) and 7,943.64 kW (13:15). Weekdays used 141,195–152,464 kWh per day against 111,680–118,994 kWh at weekends, and weekly totals grew about 2% each week, so reserve and capacity plans should be re-checked monthly.

2. Manage data completeness. Reporting was 98.99% complete (1.01% missing; 2.00% of readings arrived 30–180 minutes late). A midnight daily total (21 Sep: 149,400.54 kWh) was 0.10% (152.18 kWh) below the final value, so operations can use it, but billing should wait for the final figure. Send field crews first to the meters with most gaps, for example 1000003622 (47 missing readings).

3. Use the anomaly rule (flag a reading above 2× or below 0.25× the meter's median for that time slot and day type) as a daily screening list. It found all 40 injected events with no false flags, but those events are large; confirm it on real faults before automating action.

Storage and reporting configuration.

Keep one-day chunks and compress chunks older than 48 hours with meter-segmented compression. Storage fell from 1,352 MB to 502 MB (2.70×, 62.9% saved) for a one-off compression cost of 60.6 s. Compressed raw data is slower for one-day all-meter queries (Q1: 238.32 ms against 41.04 ms), so serve dashboards from the continuous aggregates. In our tests the five Aggregate-dashboard queries took a median 16.23 ms against 828.08 ms for the Raw dashboard (51× faster), and a full refresh in a real browser took 1,758.0 ms against 3,204.8 ms (1.8× faster), for 18 MB of extra storage and 86.7 s of one-off refresh work, which is worthwhile. Keep the three-day refresh policy so late arrivals are absorbed, and refresh older periods manually.

Forecast model.

Adopt the Ridge regression model for day-ahead hourly regional planning, keeping the previous-day forecast as a fallback. On the 1,680 test hours its MAE was 4.538 kWh against 42.708 kWh for the previous-day baseline (89% lower) and its RMSE 5.773 kWh against 79.403 kWh. At each region's peak hour the model's absolute error was 2.35–18.20 kWh against up to 270.34 kWh for the baseline. Retrain it regularly, because consumption is trending upward, and expect lower accuracy when readings are missing or late.

> Word count: 384 (limit 500).

6 Create and investigate continuous aggregates

6.1 Create the three aggregate views

`create_caggs.sql` creates three initially unpopulated continuous aggregates over `energy_readings` with `WITH NO DATA` and `timescaledb.materialized_only = true` (real-time aggregation off). Each stores its bucket and grouping key, `SUM(energy_kwh)`, `SUM(power_kw)` and the reading count (`readings`). The two regional views use an inner join of `energy_readings` and `meters` on `meter_id`; metadata was not changed.

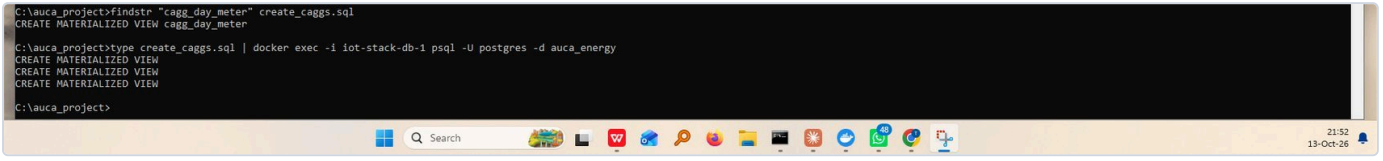
View	Bucket and grouping	Source
cagg_15min_region	`time_bucket('15 minutes', event_time)`, region	energy_readings JOIN meters
cagg_hour_region	`time_bucket('1 hour', event_time, 'Africa/Kigali')`, region	energy_readings JOIN meters
cagg_day_meter	`time_bucket('1 day', event_time, 'Africa/Kigali')`, meter_id	energy_readings

Initial manual refresh (1–29 September 2026, before any policy) and storage

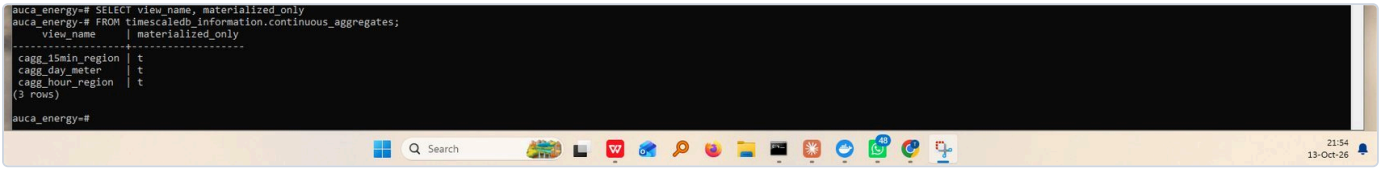
View	Refresh duration (s)	Rows	Materialized data	Index	Total size
cagg_15min_region	39.742	26,880	2152 kB	2088 kB	4280 kB
cagg_hour_region	33.109	6,720	640 kB	608 kB	1288 kB
cagg_day_meter	13.808	112,000	7616 kB	5272 kB	13 MB
All three	**86.659**	**145,600**			**≈ 18.0 MB**

Each view was refreshed manually over the whole generated period with `CALL refresh_continuous_aggregate(view, '2026-09-01 00:00:00+02', '2026-09-29 00:00:00+02')` (13 October 2026, about 21:59 CAT) and its size was measured with `hypertable_detailed_size` on the materialization hypertable. Policies were added only after these measurements: `start_offset = 3 days`, `end_offset = 1 hour`, `schedule_interval = 15 minutes` (jobs 1002, 1003 and 1004). Because the offsets are relative to the database clock (13 October onwards), the policies alone would not populate the older September data.

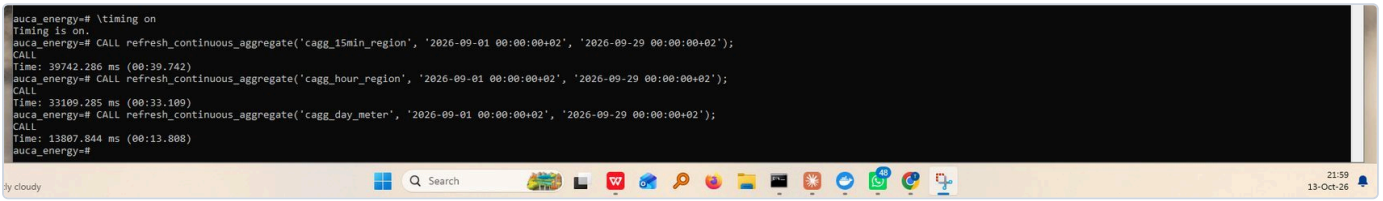
> **Observation.** The three views hold 145,600 rows in about 18 MB, against 1,352 MB for the raw table (about 1.3%), and took 86.7 s to build.



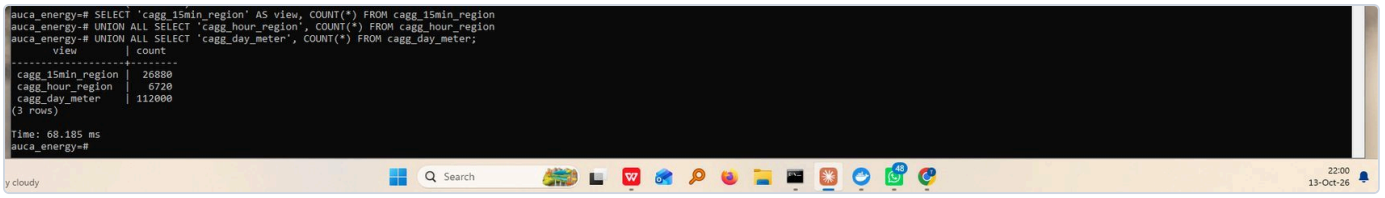
6.1 – creation of the three continuous aggregates (WITH NO DATA)



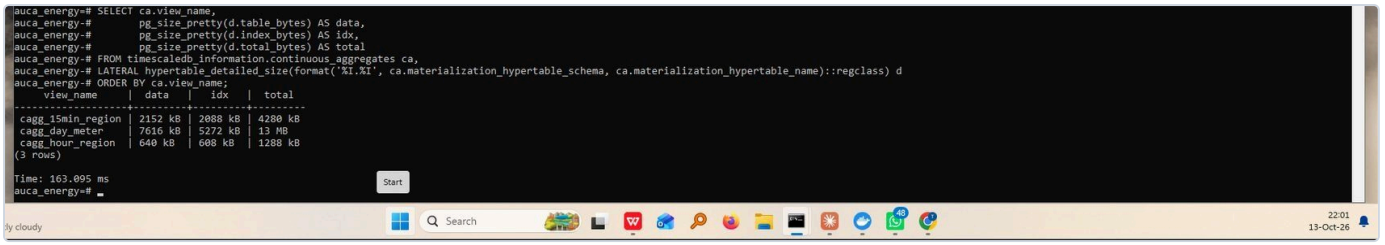
6.1 – view definitions: materialized_only (real-time aggregation off)



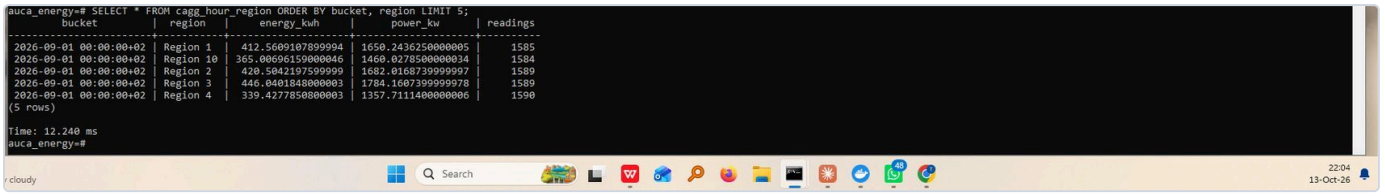
6.1 – initial manual refresh and its duration for each view



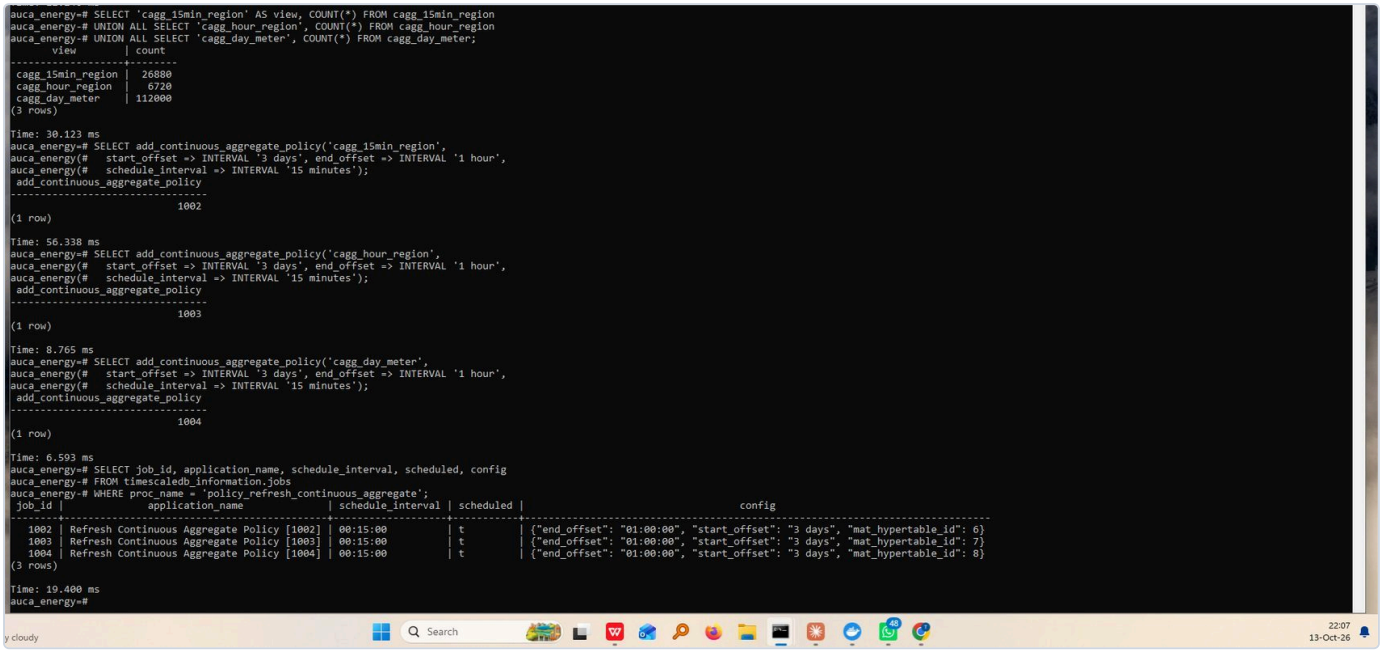
6.1 – populated row counts (26,880 / 6,720 / 112,000)



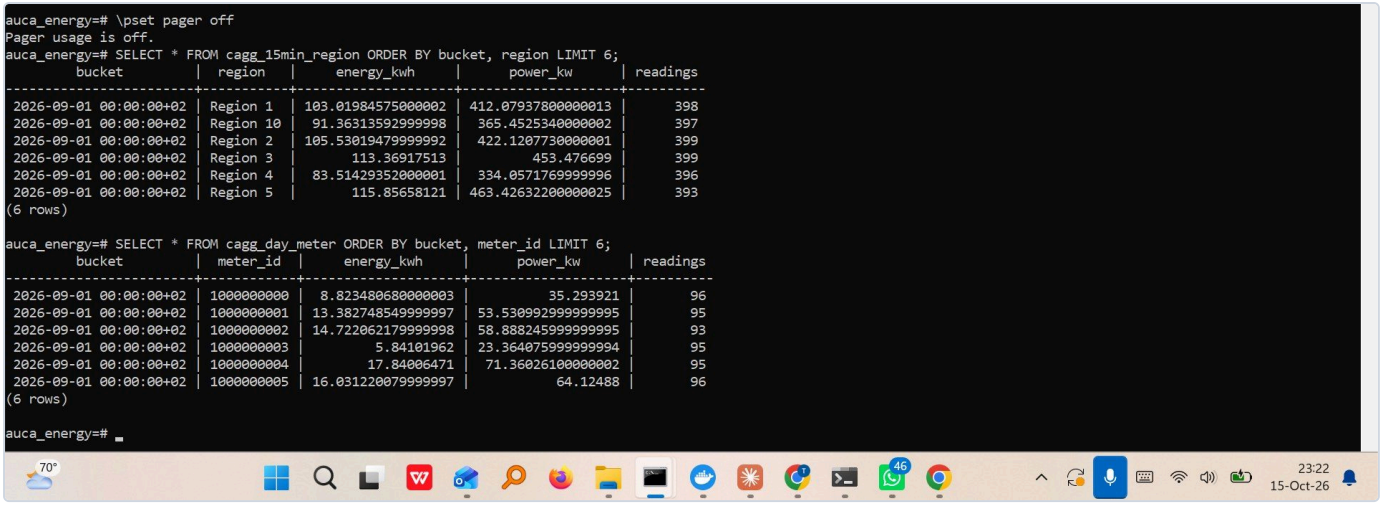
6.1 – materialized data, index and total size of each view



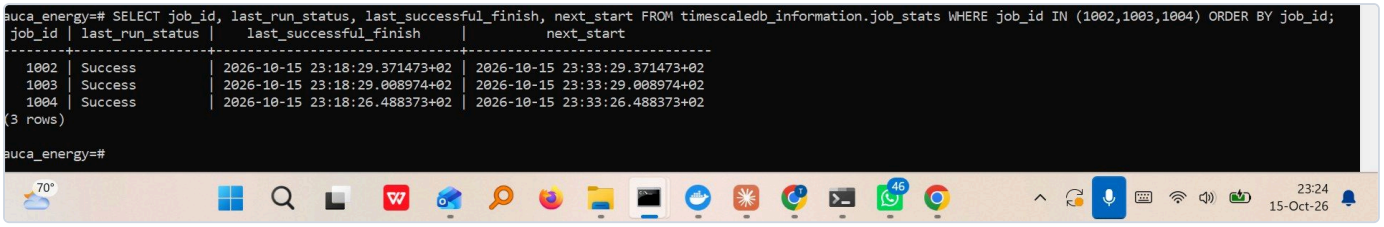
6.1 – sample populated rows (hourly regional view)



6.1 – policy settings (3-day start offset, 1-hour end offset, 15-minute schedule)



6.1 – sample populated rows of cagg_15min_region and cagg_day_meter (15 October 2026, 23:22)



6.1 – background jobs 1002–1004: last run Success, last finish 23:18, next start 23:33, i.e. every 15 minutes (15 October 2026, 23:24)

6.2 Test freshness and delayed arrivals

Test objects. A separate hypertable `fresh_test(event_time TIMESTAMPTZ NOT NULL, energy_kwh DOUBLE PRECISION)` partitioned by `event_time` and an hourly continuous aggregate `fresh_test_hourly` (sum of energy and reading count, `WITH NO DATA`, `materialized_only = true`, no policy, so automatic refresh stays off).

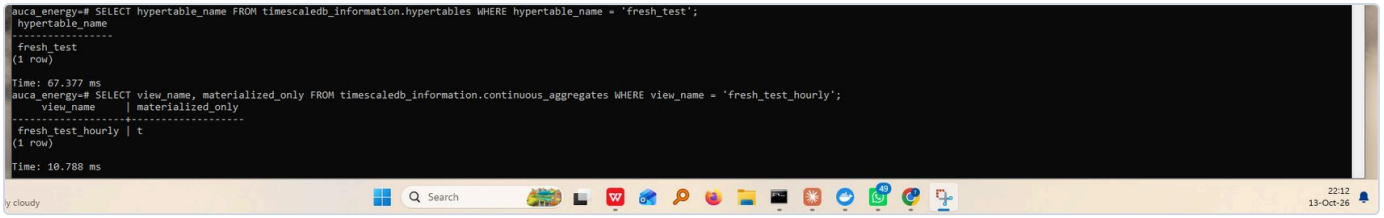
Quantity	Recorded value
Actual test time (CAT)	13 October 2026, about 22:12 (first command) to 22:19 (last command)
H = start of the most recent completed CAT hour	2026-10-13 21:00:00+02
H − 48 hours (the older hour)	2026-10-11 21:00:00+02
Insert 1: 1 kWh	event_time 2026-10-11 21:10 (older hour)
Insert 2: 0.5 kWh	event_time 2026-10-11 21:25 (older hour)
Insert 3: 2 kWh	event_time 2026-10-13 21:10 (hour H)

Result at each stage

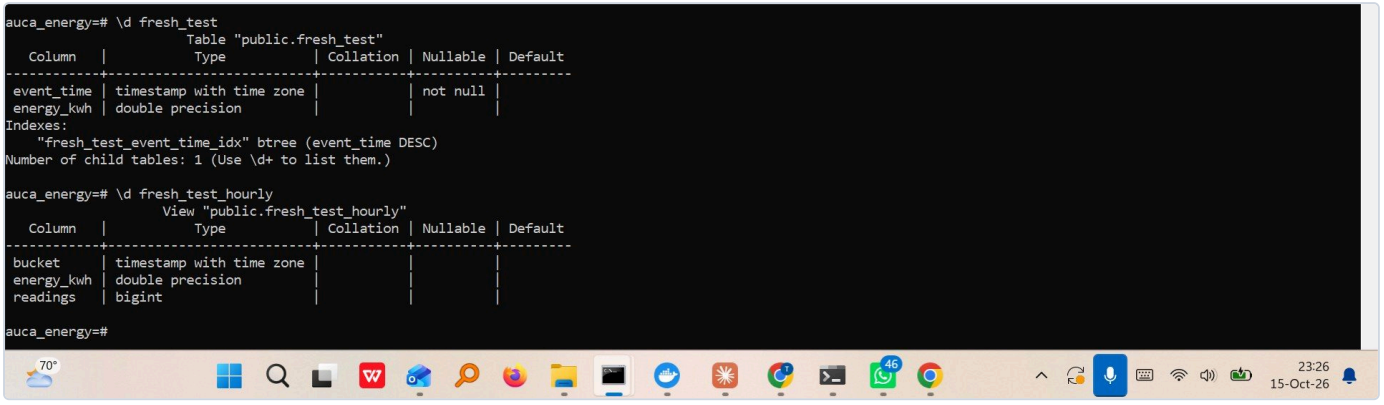
Stage	Raw table: older hour	Raw table: hour H	Aggregate: older hour	Aggregate: hour H
1. Insert 1 kWh into the older hour, manually refresh only that hour (real-time off)	1 kWh, 1 row	–	1 kWh, 1 row	no bucket
2. Insert 0.5 kWh (older hour) and 2 kWh (hour H); before refreshing, real-time off	1.5 kWh, 2 rows	2 kWh, 1 row	1 kWh, 1 row	no bucket
3. Real-time aggregation on, still before refresh	1.5 kWh, 2 rows	2 kWh, 1 row	1 kWh, 1 row	2 kWh, 1 row
4. After manual refresh of both complete hours	1.5 kWh, 2 rows	2 kWh, 1 row	**1.5 kWh, 2 rows**	**2 kWh, 1 row**

> **Explanation. Stage 1.** The refresh materialises the older hour only, so the aggregate holds that hour and nothing else; hour H has no bucket (an absent bucket is not a zero reading: nothing has been materialised for it). **Stage 2.** The new rows are in the raw table (1.5 kWh and 2 rows in the older hour, 2 kWh in H) but a materialized-only view is a stored snapshot: it still shows 1 kWh/1 row for the older hour and no bucket for H until it is refreshed. **Stage 3.** Real-time aggregation adds a live query over raw rows *newer than the materialization watermark* (the end of the refreshed older hour is already behind it, so new rows there are not included, and H is after it). H therefore appears (2 kWh, 1 row) but the older hour is still 1 kWh/1 row: real-time aggregation shows new data, not late data in an already materialised bucket. **Stage 4.** A manual refresh of both complete hours re-reads the raw rows and the aggregate agrees with the raw table: 1.5 kWh and 2 rows, and 2 kWh and 1 row.

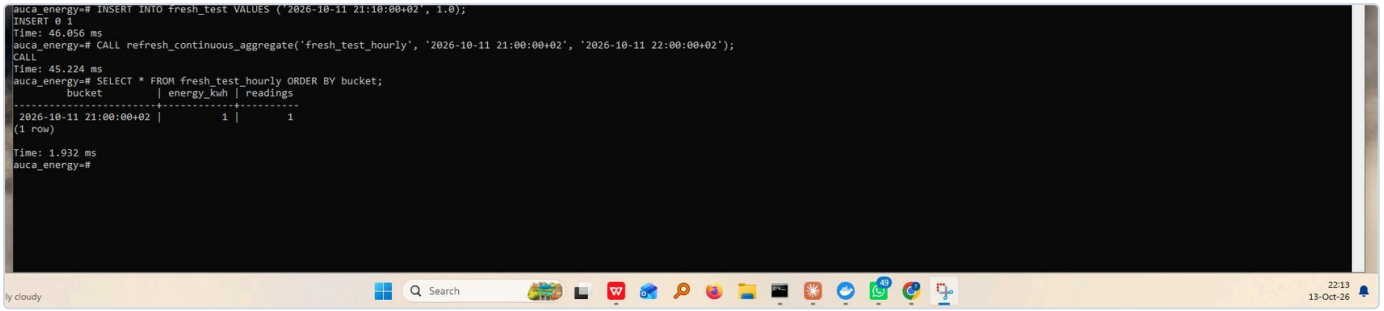
Which hours the 6.1 policy could refresh at the test time. The policy window is [now − 3 days, now − 1 hour], i.e. from about 10 October 22:12 to 13 October 21:12 at the test. The older hour (11 October 21:00–22:00) lies inside it, so the policy would pick up the late 0.5 kWh insert at its next 15-minute run. Hour H (13 October 21:00–22:00) ends after 21:12, so it is not in the window until about 23:00; real-time aggregation or a manual refresh is needed to see it earlier. Late data older than three days (for example September) is never refreshed by the policy and needs a manual refresh. (The 6.1 policies belong to the three views over `energy_readings`; `fresh_test_hourly` has no policy.)



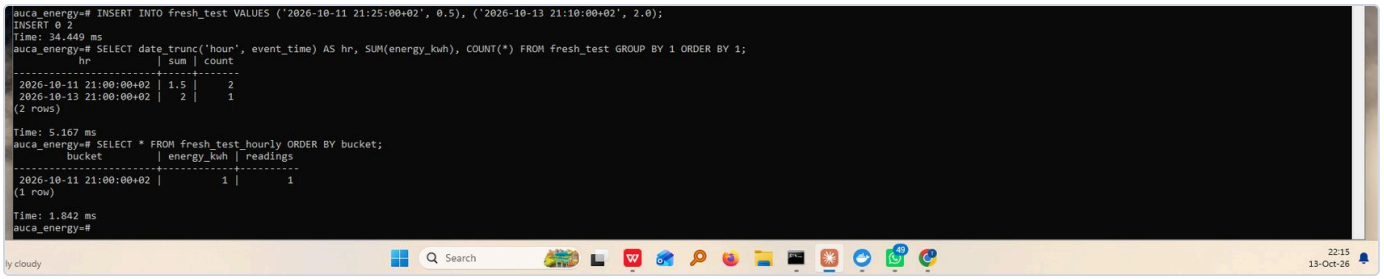
6.2 – test hypertable fresh_test and view fresh_test_hourly (materialized_only = t)



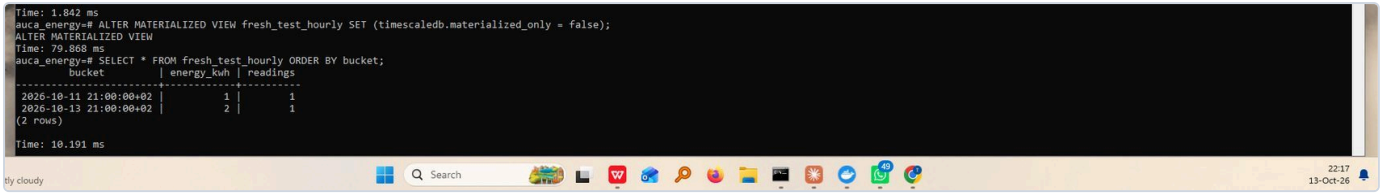
6.2 – \d fresh_test (hypertable, event_time and energy_kwh) and \d fresh_test_hourly (view with bucket, energy_kwh and readings) (15 October 2026, 23:26)



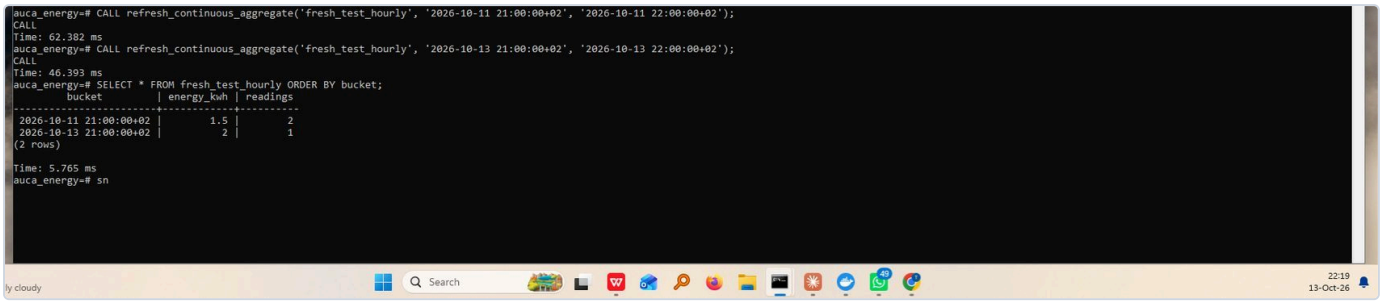
6.2 – stage 1: insert of 1 kWh into the older hour and refresh of that hour; raw and aggregate result



6.2 – stage 2: after the two new inserts, before refresh: raw table versus aggregate (real-time off)



6.2 – stage 3: real-time aggregation enabled, before refresh



6.2 – stage 4: after manual refresh of both hours: 1.5 kWh / 2 rows and 2 kWh / 1 row

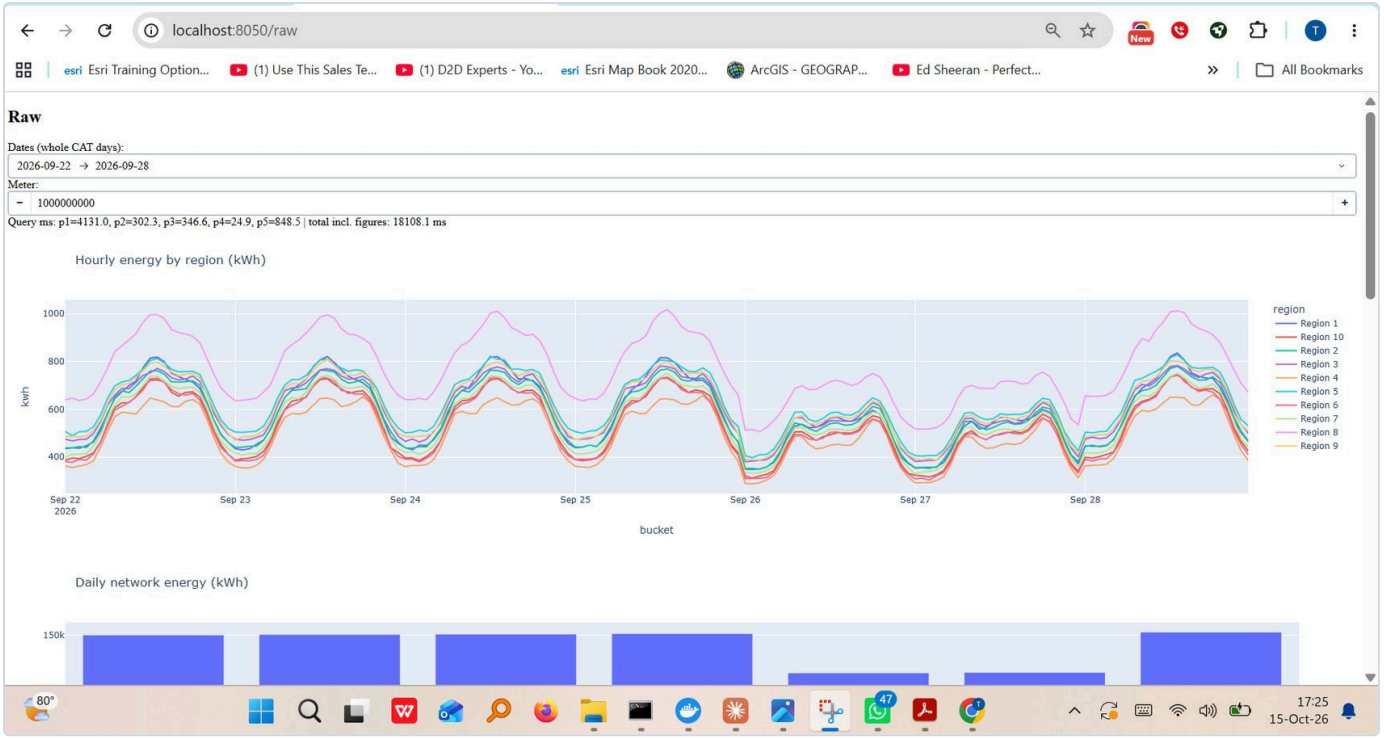
7 Build and compare dashboards

7.1 Build the Raw dashboard

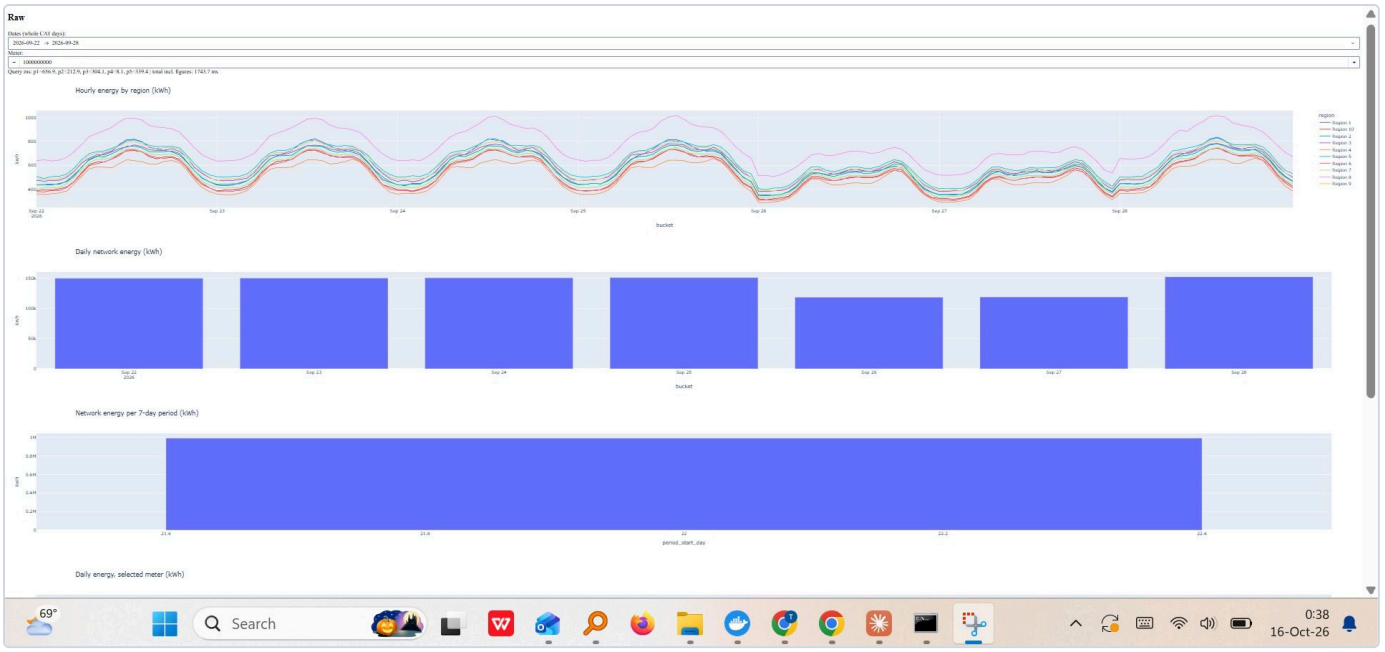
Tool. Plotly Dash (`dash_app.py` , port 8050). One application serves two pages with the same layout: `localhost:8050/raw` (title **Raw**) and `localhost:8050/aggregate` (title **Aggregate**). All panel queries are in `dq.py` (the dictionary `RAW` for this page, `AGG` for 7.2); the dashboard and the checker `dash_check.py` use the same functions, so what is compared and timed is exactly what the dashboard runs. There is no query-result cache in the code: every refresh runs all five queries again.

Filters. A date-range picker for whole CAT days (selectable 1–28 September; the end day is included, so 22–28 September means 00:00 on 22 September to 00:00 on 29 September) and a numeric input for the selected meter (default 1000000000). Every panel is limited to the selected range.

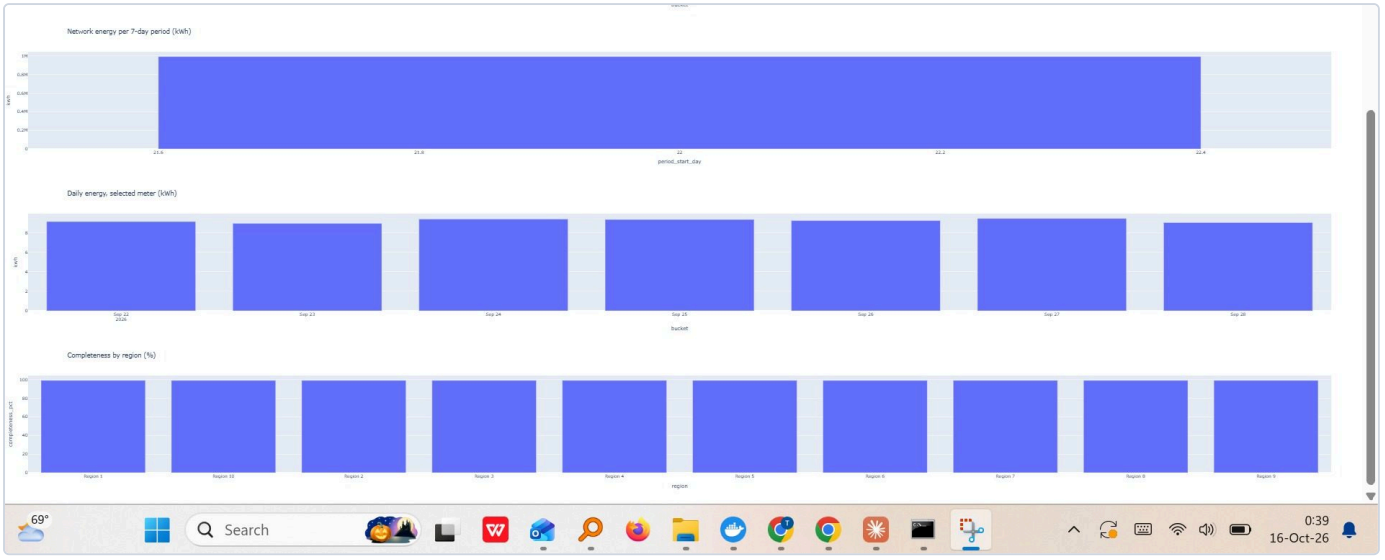
Panel	Title and unit	Raw query reads
p1	Hourly energy by region (kWh), line chart	energy_readings JOIN meters, grouped by CAT hour and region
p2	Daily network energy (kWh), bar chart	energy_readings, grouped by CAT day
p3	Network energy per seven-day period (kWh), bar chart	energy_readings, grouped by the 1–7, 8–14, 15–21, 22–28 periods
p4	Daily energy of the selected meter (kWh), bar chart	energy_readings filtered on meter_id, grouped by CAT day
p5	Completeness by region (%), bar chart	energy_readings JOIN meters; observed pairs / (registered meters × days × 96)



7.1 – the running Raw dashboard (localhost:8050/raw), 22–28 September, meter 1000000000: the first panels (the page continues below)



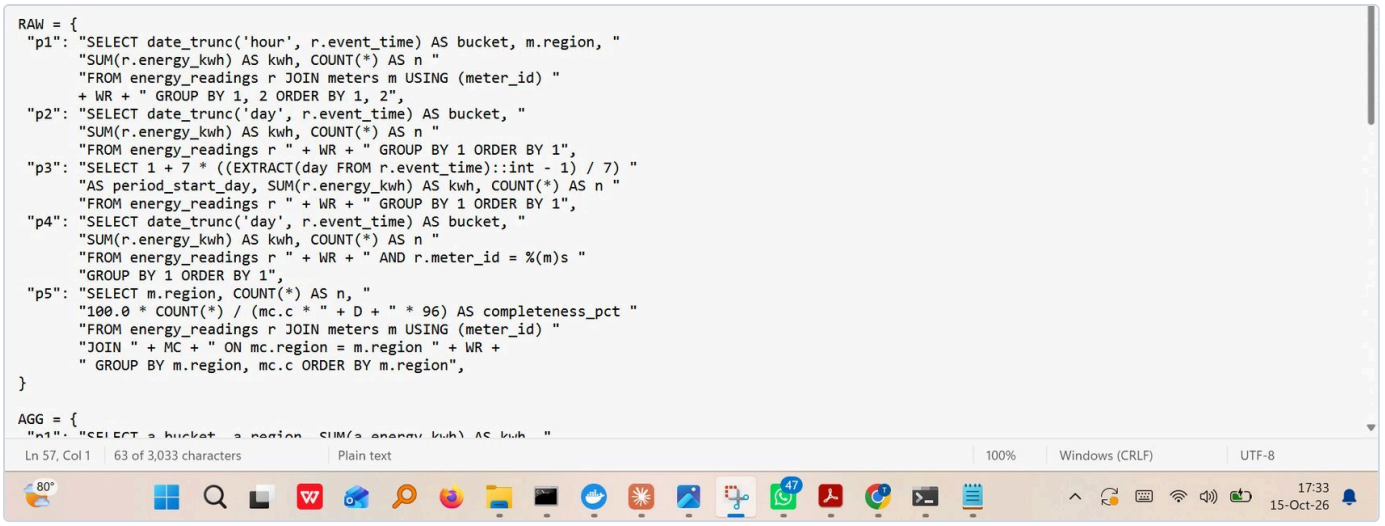
7.1 – the Raw dashboard, top of the page (16 October 2026, 00:38 CAT): filters, the timing line and panels 1–3 (22–28 September, meter 1000000000)



7.1 – the Raw dashboard scrolled down (00:39 CAT): panels 3, 4 and 5, so all five panels are shown across the two pictures. The seven-day-period chart (panel 3) has one bar because the selected week is a single period starting on day 22; the axis shows 21.6–22.4 only because Plotly treats the day number as a continuous axis



7.1 – the Raw timing line enlarged: query time per panel and the total including figure creation (1,743.7 ms)



7.1 – the panel queries behind the Raw dashboard

Retained: `dash_app.py`, `dq.py` (all panel queries) and `dash_check.py` in the repository.

7.2 Build the Aggregate dashboard

Panel	Aggregate query reads	How buckets are combined
p1 hourly regional energy	**cagg_hour_region**	SUM(energy_kwh) by bucket and region
p2 daily network energy	**cagg_15min_region**	SUM(energy_kwh) of the 15-minute buckets, grouped by CAT day
p3 seven-day periods	**cagg_15min_region**	SUM(energy_kwh) grouped by the same period expression
p4 selected-meter daily energy	**cagg_day_meter**	the stored daily sum for the meter
p5 completeness by region	**cagg_15min_region** and meters	SUM(readings) / (registered meters per region × days × 96); meters is queried only for registered counts

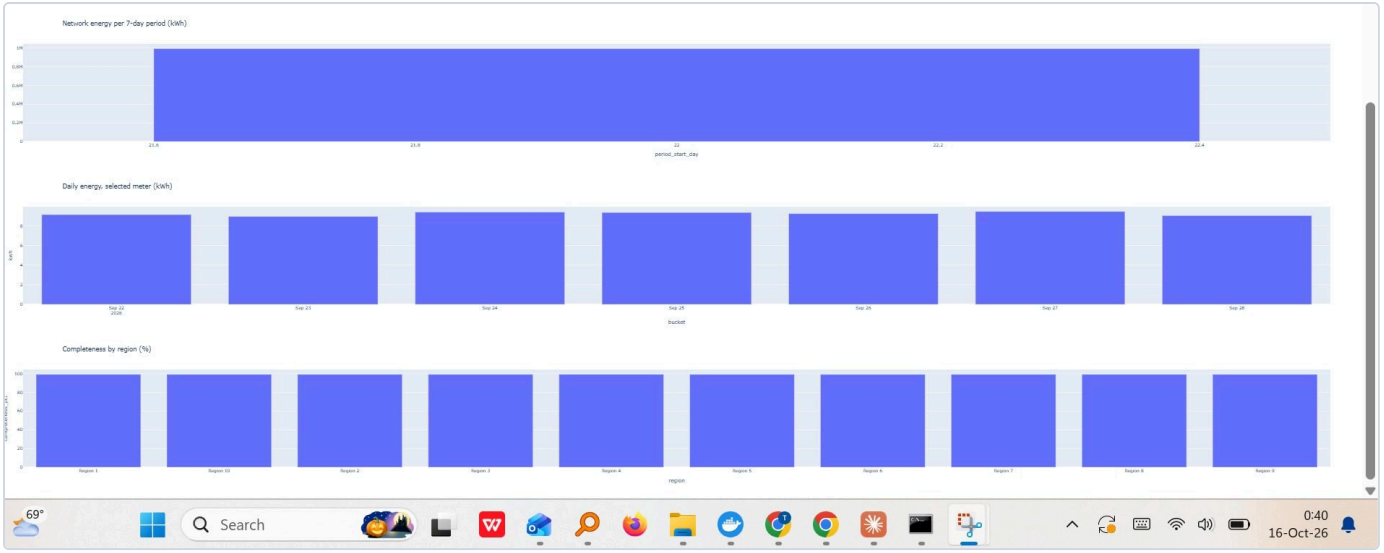
The Aggregate page has the same five panels, layout, filters and date range as the Raw page. Stored energy sums and reading counts are summed when buckets are combined; planned readings come from the registered meter counts and the selected period. No panel reads `energy_readings`. (The optional pilot-message panel was not built.)



7.2 – the running Aggregate dashboard (localhost:8050/aggregate): the first panels (the page continues below)



7.2 – the Aggregate dashboard, top of the page (16 October 2026, 00:40 CAT): same layout, filters and values as the Raw page



7.2 – the Aggregate dashboard scrolled down (00:40 CAT): panels 3, 4 and 5



7.2 – the Aggregate timing line enlarged: query time per panel and the total including figure creation (748.0 ms)

```
AGG = {
 "p1": "SELECT a.bucket, a.region, SUM(a.energy_kwh) AS kwh, "
       "SUM(a.readings) AS n FROM cagg_hour_region a " + WA +
       " GROUP BY 1, 2 ORDER BY 1, 2",
 "p2": "SELECT date_trunc('day', a.bucket) AS bucket, "
       "SUM(a.energy_kwh) AS kwh, SUM(a.readings) AS n "
       "FROM cagg_15min_region a " + WA + " GROUP BY 1 ORDER BY 1",
 "p3": "SELECT 1 + 7 * ((EXTRACT(day FROM a.bucket)::int - 1) / 7) "
       "AS period_start_day, SUM(a.energy_kwh) AS kwh, "
       "SUM(a.readings) AS n FROM cagg_15min_region a " + WA +
       " GROUP BY 1 ORDER BY 1",
 "p4": "SELECT a.bucket, a.energy_kwh AS kwh, a.readings AS n "
       "FROM cagg_day_meter a " + WA + " AND a.meter_id = %(m)s ORDER BY 1",
 "p5": "SELECT a.region, SUM(a.readings) AS n, "
       "100.0 * SUM(a.readings) / (mc.c * " + D + " * 96) "
       "AS completeness_pct FROM cagg_15min_region a "
       "JOIN " + MC + " ON mc.region = a.region " + WA +
       " GROUP BY a.region, mc.c ORDER BY a.region",
}
MODES = {"raw": RAW, "agg": AGG}

def connect():
    c = psycopg2.connect(host="localhost", port=5432, dbname="auca_energy",
                         user="postgres", password="[redacted]")
    c.autocommit = True
    c.cursor().execute("SET timezone TO 'Africa/Kigali'")
    return c

def frame(conn, mode, panel, p):
    cur = conn.cursor()
    t0 = time.perf_counter()
    cur.execute(MODES[mode][panel], p)
    rows = cur.fetchall()
```

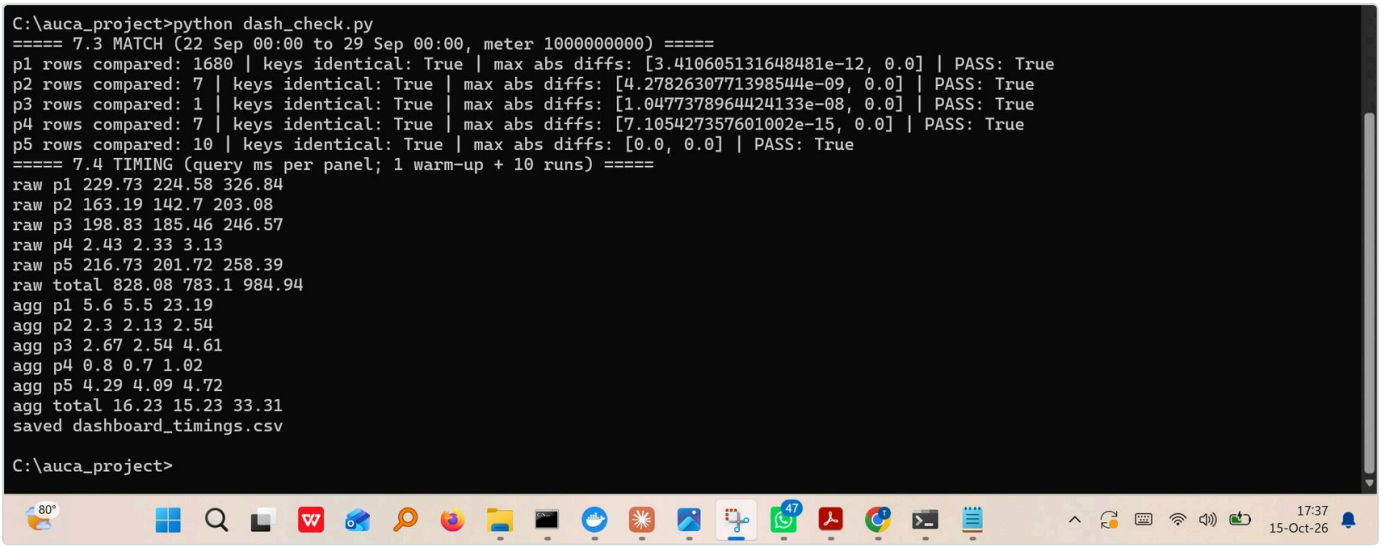

7.2 – the panel queries behind the Aggregate dashboard (the database password is hidden)

7.3 Verify matching results

Both dashboards were set to 00:00 on 22 September through 00:00 on 29 September (exclusive), all regions, meter 1000000000. `dash_check.py` runs the numerical data behind every panel in both modes, matches rows by key (bucket and region, bucket, period, bucket, region) and applies the required tolerances: identical keys and reading counts, energy difference ≤ 0.001 kWh and completeness difference ≤ 0.000001 percentage points.

Panel	Rows compared	Keys identical	Max |energy difference| (kWh)	Max |reading-count difference|	Result
p1 hourly energy by region	1,680	Yes	3.41×10^{-12}	0	**PASS**
p2 daily network energy	7	Yes	4.28×10^{-9}	0	**PASS**
p3 seven-day period	1	Yes	1.05×10^{-8}	0	**PASS**
p4 selected-meter daily energy	7	Yes	7.1×10^{-15}	0	**PASS**
p5 completeness by region	10	Yes	0.0 (completeness, pp)	0	**PASS**

All five panels agree: the keys and reading counts are identical and the energy differences are at the level of floating-point rounding (at most 1.05×10^{-8} kWh, against the 0.001 kWh limit). No correction was needed before timing. The p3 panel has one row because the selected range lies entirely in the 22–28 September period.



7.3 and 7.4 – dash_check.py output (17:37 CAT, 15 October): numerical checks for all five panels (top) and the timing results (bottom)

7.4 Measure performance and judge the tradeoff

Method. With the 7.3 filters and dates, `dash_check.py` performs **one warm-up refresh and ten measured refreshes per mode** (a refresh = the five panel queries run one after another on the same Python process, psycopg2 connection and machine). Query durations are measured with `time.perf_counter()` around `cursor.execute()` and `fetchall()` (tool: `dq.frame()`); the dashboard code has no result cache. The measured "dashboard" value is the sum of the five query durations for one refresh. Results are in `dashboard_timings.csv`. The assignment background jobs (compression and continuous-aggregate policies, jobs 1000–1004) were paused during the timing and resumed afterwards (screenshot below: all five jobs scheduled = t at 17:40 CAT, three minutes after the 17:37 timing run).

Query time per refresh (ms): median (minimum–maximum), ten measured refreshes

Panel	Raw	Aggregate	Raw / Aggregate (medians)
p1 hourly energy by region	229.73 (224.58–326.84)	5.60 (5.50–23.19)	41.0×
p2 daily network energy	163.19 (142.70–203.08)	2.30 (2.13–2.54)	71.0×
p3 seven-day period	198.83 (185.46–246.57)	2.67 (2.54–4.61)	74.5×
p4 selected-meter daily energy	2.43 (2.33–3.13)	0.80 (0.70–1.02)	3.0×
p5 completeness by region	216.73 (201.72–258.39)	4.29 (4.09–4.72)	50.5×
Whole dashboard (sum of five)	**828.08 (783.10–984.94)**	**16.23 (15.23–33.31)**	**51.0×**

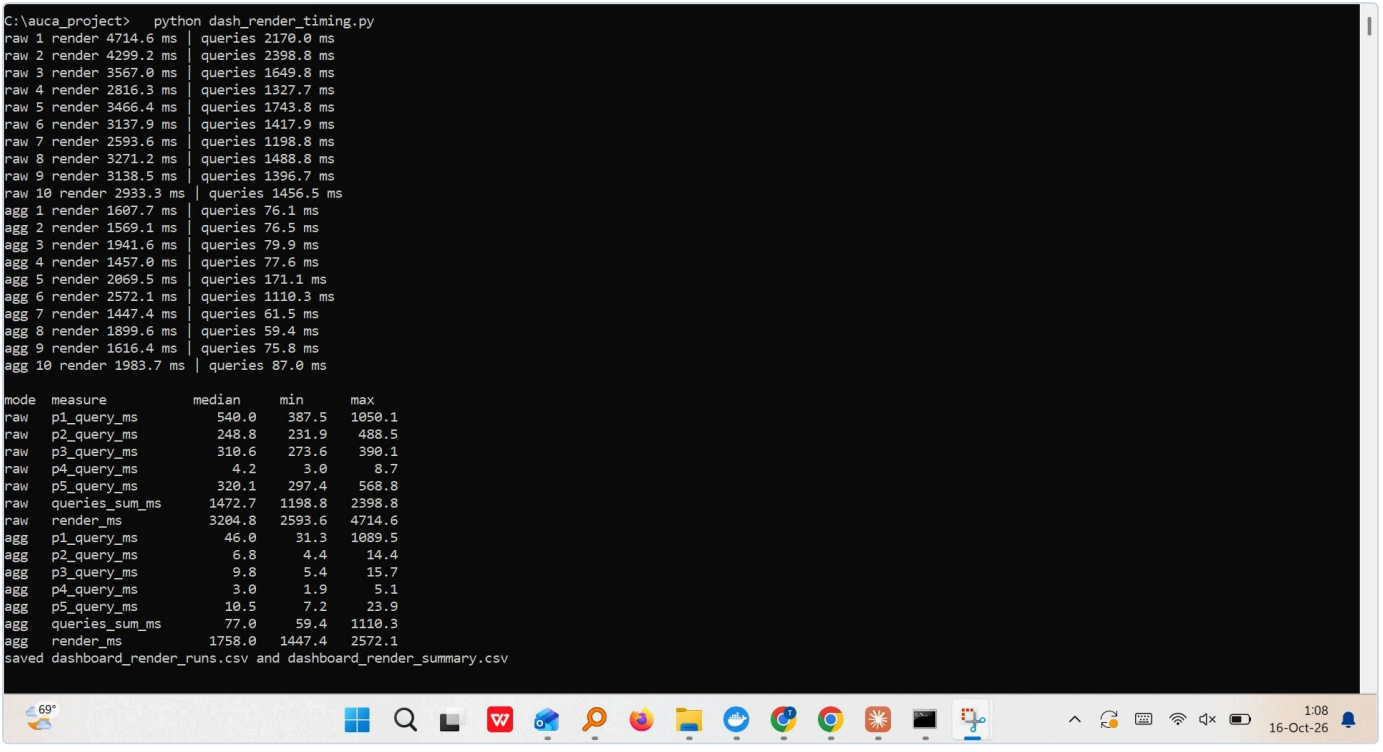
> **Observation.** The dashboard's median query time fell from 828.08 ms to 16.23 ms, 51× faster (98.0% less). Even the slowest aggregate refresh (33.31 ms) was 24× faster than the fastest raw refresh (783.10 ms). The raw panels that scan readings (p1, p2, p3, p5) cost 160–230 ms each; panel p4 is already fast on raw data (2.43 ms, one meter with an index) so it gains least in absolute terms.

End-to-end refresh in a real browser (request until all five panels are drawn)

Method. `dash_render_timing.py` drives Google Chrome (through Playwright) against the running dashboard with the same filters (22–28 September, meter 1000000000). For each page it makes **one warm-up request and ten measured requests** in the same browser. The clock starts when the page is requested and stops when the server has answered, the five chart titles are drawn in the browser and the dashboard's timing line is shown; the per-panel query durations are the ones the dashboard itself prints. The measured requests were made between 01:06:03 and 01:06:59 CAT on 16 October (the start and end time of every run is in the CSV), on the same laptop as the database; jobs 1000–1004 were paused at 00:52 CAT and switched on again after the run (screenshot below). Results are in `dashboard_render_runs.csv` and `dashboard_render_summary.csv`.

Browser run (ms): median (minimum–maximum), ten measured refreshes

Measure	Raw	Aggregate	Raw / Aggregate (medians)
p1 hourly energy by region	540.0 (387.5–1,050.1)	46.0 (31.3–1,089.5)	11.7×
p2 daily network energy	248.8 (231.9–488.5)	6.8 (4.4–14.4)	36.6×
p3 seven-day period	310.6 (273.6–390.1)	9.8 (5.4–15.7)	31.7×
p4 selected-meter daily energy	4.2 (3.0–8.7)	3.0 (1.9–5.1)	1.4×
p5 completeness by region	320.1 (297.4–568.8)	10.5 (7.2–23.9)	30.5×
Sum of the five queries	**1,472.7 (1,198.8–2,398.8)**	**77.0 (59.4–1,110.3)**	**19.1×**
Request until all five panels are drawn	**3,204.8 (2,593.6–4,714.6)**	**1,758.0 (1,447.4–2,572.1)**	**1.8×**



7.4 – output of dash_render_timing.py (run at 01:06 CAT, screenshot taken at 01:08 CAT, 16 October): the ten runs per page and the medians, minimums and maximums

> **Observation.** Measured in the browser, a full refresh fell from a median **3,204.8 ms to 1,758.0 ms**, 1.8× faster (1,446.8 ms, 45% less), while the query time alone fell 19× (1,472.7 ms to 77.0 ms). The time saved on screen (1,446.8 ms) is close to the query time saved (1,395.7 ms); what is left, roughly 1.7 s on Raw and 1.5 s on Aggregate (request, building the figures, sending them and drawing them), is paid by both pages and is why the ratio on screen is much smaller than the ratio for the queries. The two ranges almost touch: the slowest Aggregate refresh (2,572.1 ms) was only just faster than the fastest Raw refresh (2,593.6 ms).

Explanation. Two points that need care. (1) The query times in this run are higher than in the dash_check.py benchmark (Raw 1,472.7 ms against 828.08 ms; Aggregate 77.0 ms against 16.23 ms), most likely because here the queries run inside the dashboard server while the browser is drawing on the same laptop; we did not isolate the cause, so the two sets of numbers are reported separately and not mixed. (2) Aggregate run 6 had one slow query phase (queries 1,110.3 ms, p1 = 1,089.5 ms) and a refresh of 2,572.1 ms; we did not establish why. It does not move the medians, but it is the reason the range is shown.

First load of the pages. The dashboard also shows its own timing line on page load. On a single cold first load of each page (17:25–17:26 CAT, 15 October) the Raw page showed query times of p1 = 4,131.0 ms, p2 = 302.3 ms, p3 = 346.6 ms, p4 = 24.9 ms and p5 = 848.5 ms and **18,108.1 ms** in total including figure creation, whereas the Aggregate page took **529.8 ms** in total. These are single cold loads and are shown for illustration only; they are not part of the ten-run statistics.

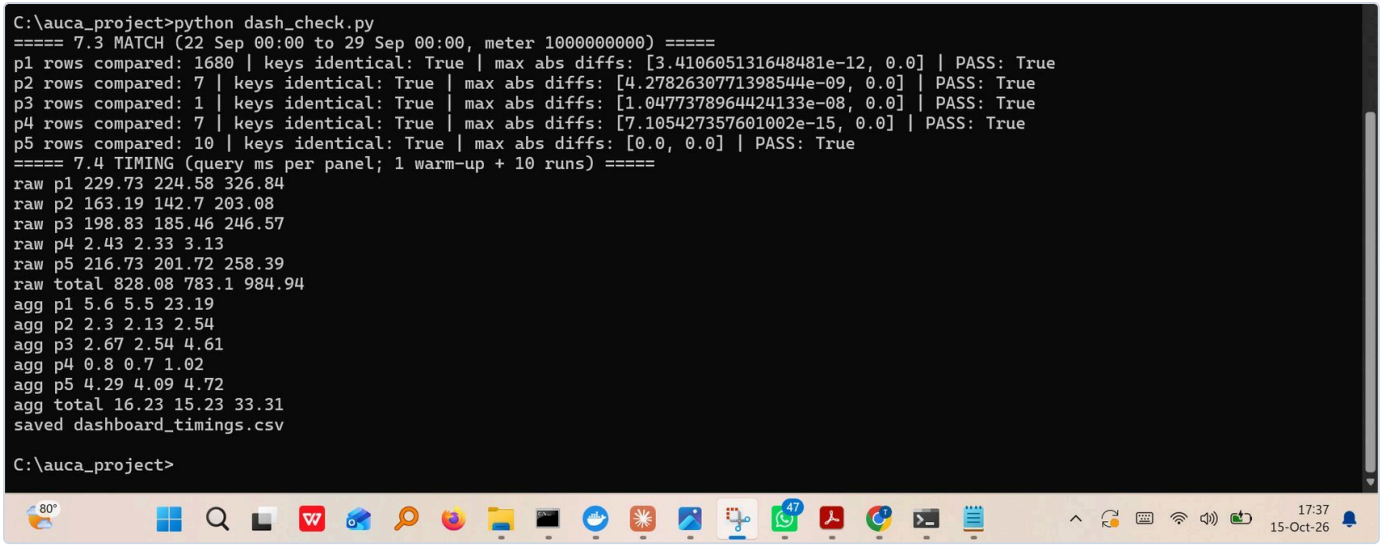
Repeat loads. Both pages were loaded again at 00:38–00:40 CAT on 16 October (screenshots and enlarged timing lines in 7.1 and 7.2): Raw took **1,743.7 ms** and Aggregate **748.0 ms** in total including figure creation, which is the dashboard's own timing line and does not include the browser's drawing time. An earlier pair of single loads at 23:35–23:36 CAT on 15 October gave 15,904.4 ms (Raw) and 11,585.0 ms (Aggregate), with Aggregate panels p2, p3 and p5 as slow as the Raw ones; that was not reproduced on the repeat and we did not establish its cause, so it is not used. Single loads depend on cache state and on what else the machine is doing, which is why the report relies on the ten-run measurements.

Storage and refresh cost of the aggregates (from 6.1)

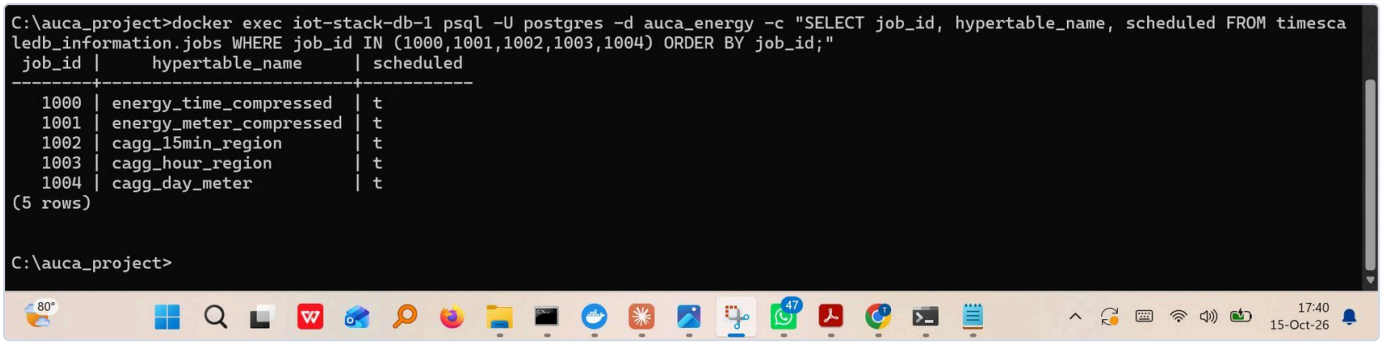
View	Initial refresh (s)	Total size
cagg_15min_region	39.742	4,280 kB
cagg_hour_region	33.109	1,288 kB
cagg_day_meter	13.808	13 MB (12,888 kB)
All three	**86.659**	**≈ 18.0 MB** (1.3% of the 1,352 MB raw table)

Explanation. The speed-up justifies the extra storage and refresh work, although it is smaller on screen than in the database. A dashboard refresh as the user sees it fell from 3,204.8 ms to 1,758.0 ms (1.8×, about 1.4 s saved per refresh, 45% less); the five queries alone are 51× faster in the controlled benchmark (812 ms saved) and 19× faster in the browser run. The price is only about 18 MB (1.3% of the raw table) and a one-off 86.7 s build, plus the 15-minute policy refresh of the last three days in the background. A dashboard viewed by many users, refreshing every few seconds, would otherwise load the database with about 0.8 to 1.5 s of scans per refresh, and the Aggregate dashboard removes almost all of that load. The costs are real but small: aggregates are slightly stale until refreshed (6.2), data older than the policy window must be refreshed manually, and queries that need detail not stored in the aggregates still have to go to the raw table. The identical results in 7.3 show that nothing is lost for these five panels.

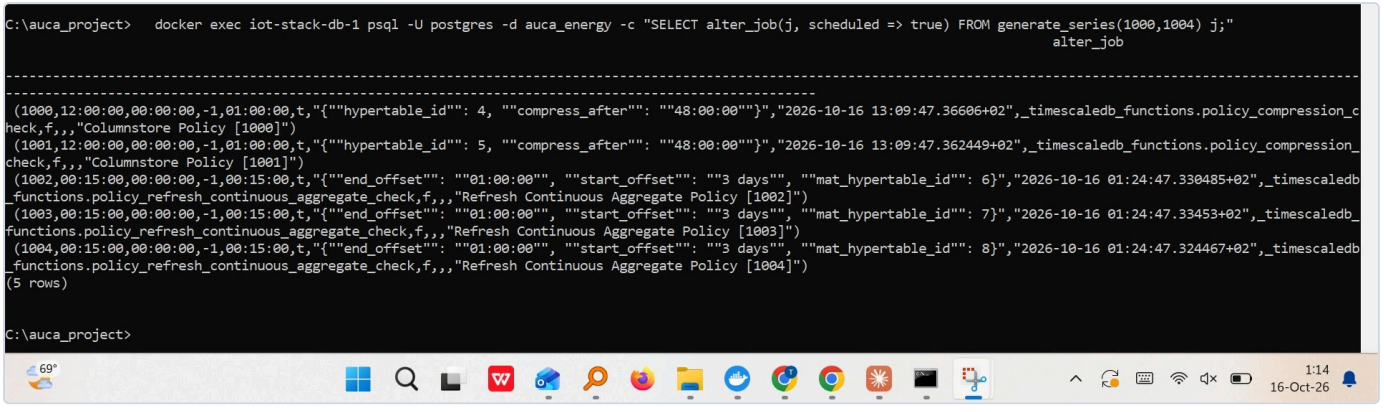
Measurement limits. Both measurements ran on one laptop that also hosts the database, the dashboard server and the browser, with the background jobs paused and ten measured runs per page; a different machine, a network between the parts or many users at the same time would change the absolute numbers. The browser run requests a fresh page each time, so the page's script files are cached after the warm-up, as they would be for a returning user. The single slow Aggregate run (run 6) is kept in the data.



7.4 – completed timing results (the lower half of the dash_check.py output) with 1 warm-up and 10 measured runs per mode



7.4 – paused jobs resumed after the measurement: jobs 1000–1004, scheduled = t (17:40 CAT)



7.4 – jobs 1000–1004 switched on again after the browser run: the sixth value of every row (scheduled) is t (command repeated for this screenshot at 01:14 CAT, 16 October)

8 Forecast the next day

The task is an offline forecast of observed hourly regional consumption (kWh). Completed historical hourly totals are treated as final; the remaining missing readings (about 1%) are a limitation discussed in 8.4. Code: `fc1.py` (dataset), `fc2.py` (baseline, model, evaluation).

8.1 Prepare chronological data and features

Dataset. One row per region and CAT hour: `SUM(energy_kwh)` from the cleaned readings, plus the reading count. The result has 10 regions × 672 hours = **6,720 rows** with **no absent region-hours**. Each region-hour should contain 400 meters × 4 readings = 1,600 readings; the observed counts range from **1,562 to 1,597**, i.e. up to 2.4% of readings are missing in an hour. The observed totals are kept as they are and used as the targets (missing readings are not filled with zero, see 2.3), so there were no missing predictor values in the validation and test periods.

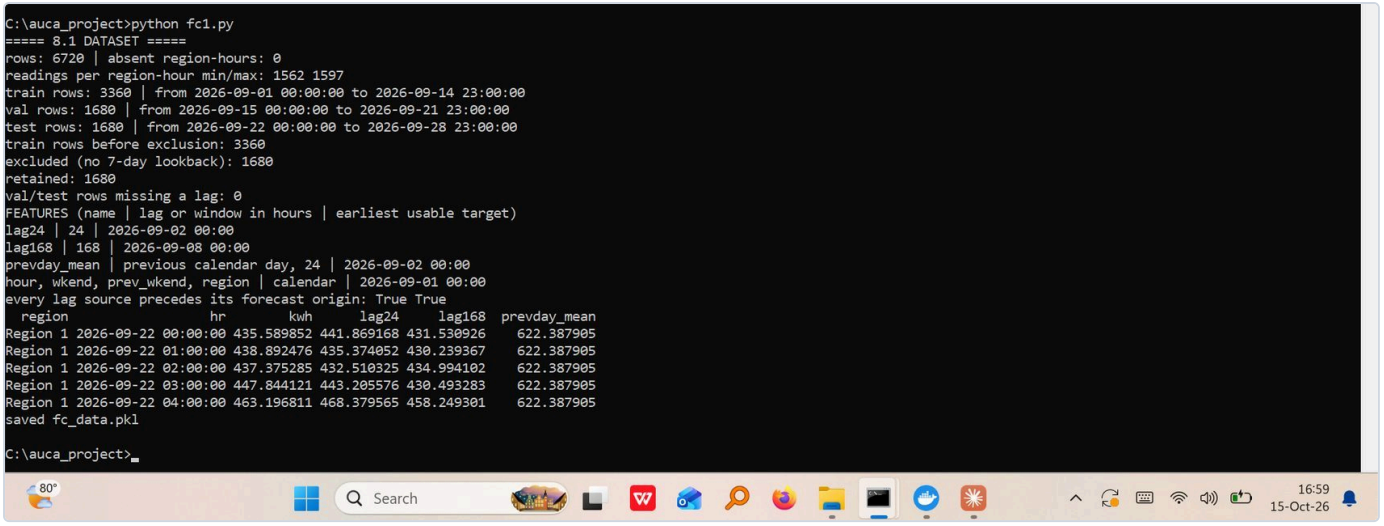
Split (by target timestamp)	Rows	First target hour	Last target hour
Training	3,360	2026-09-01 00:00	2026-09-14 23:00
Validation	1,680	2026-09-15 00:00	2026-09-21 23:00
Test	1,680	2026-09-22 00:00	2026-09-28 23:00

Forecast origin and features. For each target hour the forecast origin is 00:00 at the start of its day, so only information from before that origin may be used; the preceding hour in the forecast day is therefore not a feature.

Feature	Lag or window length (hours)	Earliest usable target timestamp	Meaning
lag24	24	2026-09-02 00:00	consumption of the same region at the same hour on the previous day
lag168	168	2026-09-08 00:00	consumption at the same hour one week earlier
prevday_mean	previous calendar day, 24	2026-09-02 00:00	mean hourly consumption of the previous calendar day
hour	calendar	2026-09-01 00:00	hour of day (0–23, one-hot)
wkend	calendar	2026-09-01 00:00	1 if the target day is Saturday or Sunday
prev_wkend	calendar	2026-09-01 00:00	1 if the previous day was Saturday or Sunday
region	calendar	2026-09-01 00:00	region (one-hot)

Training rows. The initial lookback is 7 days (lag168). Training rows before exclusion: **3,360**; excluded for lacking the lookback (1–7 September): **1,680**; retained: **1,680** (8–14 September). The same 1,680 usable training rows are used for every model configuration. No validation or test row lacks a lag value (0 missing). Preprocessing (standardisation of the numeric predictors) is fitted on the retained training rows only.

Check that every lag source precedes its forecast origin: for all rows, target hour − 24 h and target hour − 168 h are earlier than the 00:00 origin of the target day (both checks printed **True**).

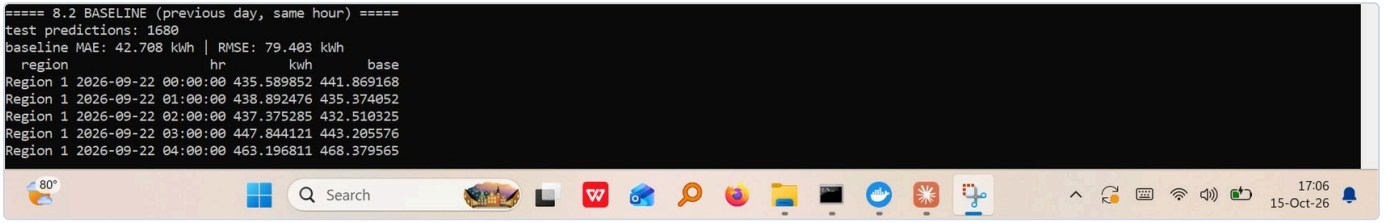


8.1 – dataset preparation: rows, absent hours, reading counts, split sizes and dates, exclusions, feature table, lag-before-origin checks and feature examples

8.2 Evaluate the previous-day baseline

At 00:00 on each test day the baseline forecasts each of the 24 hours with the previous day's consumption at the same hour (`lag24`), for each of the 10 regions. This gives **1,680 test predictions** (7 days × 24 hours × 10 regions).

Method	Predictions	MAE (kWh)	RMSE (kWh)
Previous-day baseline	1,680	42.708	79.403



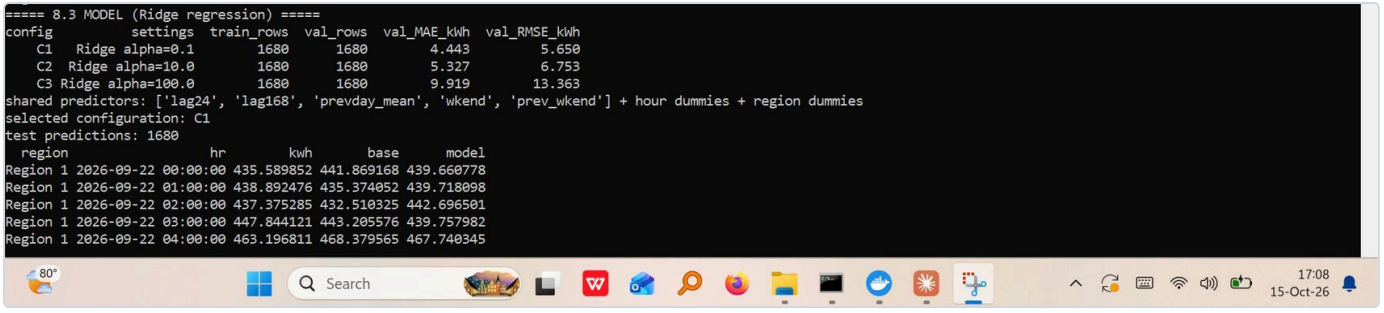
8.2 – baseline run: prediction count, sample predictions and metrics

8.3 Train and evaluate one regression model

Model: Ridge regression (scikit-learn), trained at three regularisation strengths on the 1,680 usable training rows. **Shared predictors:** lag24, lag168, prevday_mean, wkend, prev_wkend, hour (one-hot) and region (one-hot). Numeric and dummy predictors were standardised with a scaler fitted on the training rows only.

Configuration	Model settings	Training rows	Validation rows	Validation MAE (kWh)	Validation RMSE (kWh)
C1 (selected)	**Ridge, alpha = 0.1**	**1,680**	**1,680**	**4.443**	**5.650**
C2	Ridge, alpha = 10	1,680	1,680	5.327	6.753
C3	Ridge, alpha = 100	1,680	1,680	9.919	13.363

C1 has the lowest validation MAE and was frozen (model and scaler) before testing. The test predictions use only information available before each daily forecast origin; completed earlier test days supply later lag values, but no refitting of the model or the scaler was done on test data. Test result: **1,680 predictions, MAE 4.538 kWh, RMSE 5.773 kWh.**



8.3 – training, validation MAE of each configuration, selected settings, prediction count and sample test predictions

8.4 Judge forecast usefulness

Error = predicted − actual (kWh); MAE averages absolute errors, RMSE is the square root of the mean squared errors; both are in kWh over 1,680 test observations (24 hourly predictions per region per test day).

Region	Baseline MAE	Baseline RMSE	Model MAE	Model RMSE	MAE reduction
All regions (1,680)	**42.708**	**79.403**	**4.538**	**5.773**	**89.4%**
Region 1	44.87	83.70	4.66	5.86	89.6%
Region 2	41.63	76.81	4.50	5.63	89.2%
Region 3	42.75	77.61	4.28	5.59	90.0%
Region 4	34.86	64.23	4.03	5.11	88.4%
Region 5	44.40	81.08	5.21	6.59	88.3%
Region 6	39.43	74.77	3.67	4.66	90.7%
Region 7	39.69	73.94	4.40	5.70	88.9%
Region 8	55.97	102.06	5.74	7.11	89.7%
Region 9	43.77	80.18	4.68	5.80	89.3%
Region 10	39.72	74.28	4.21	5.29	89.4%

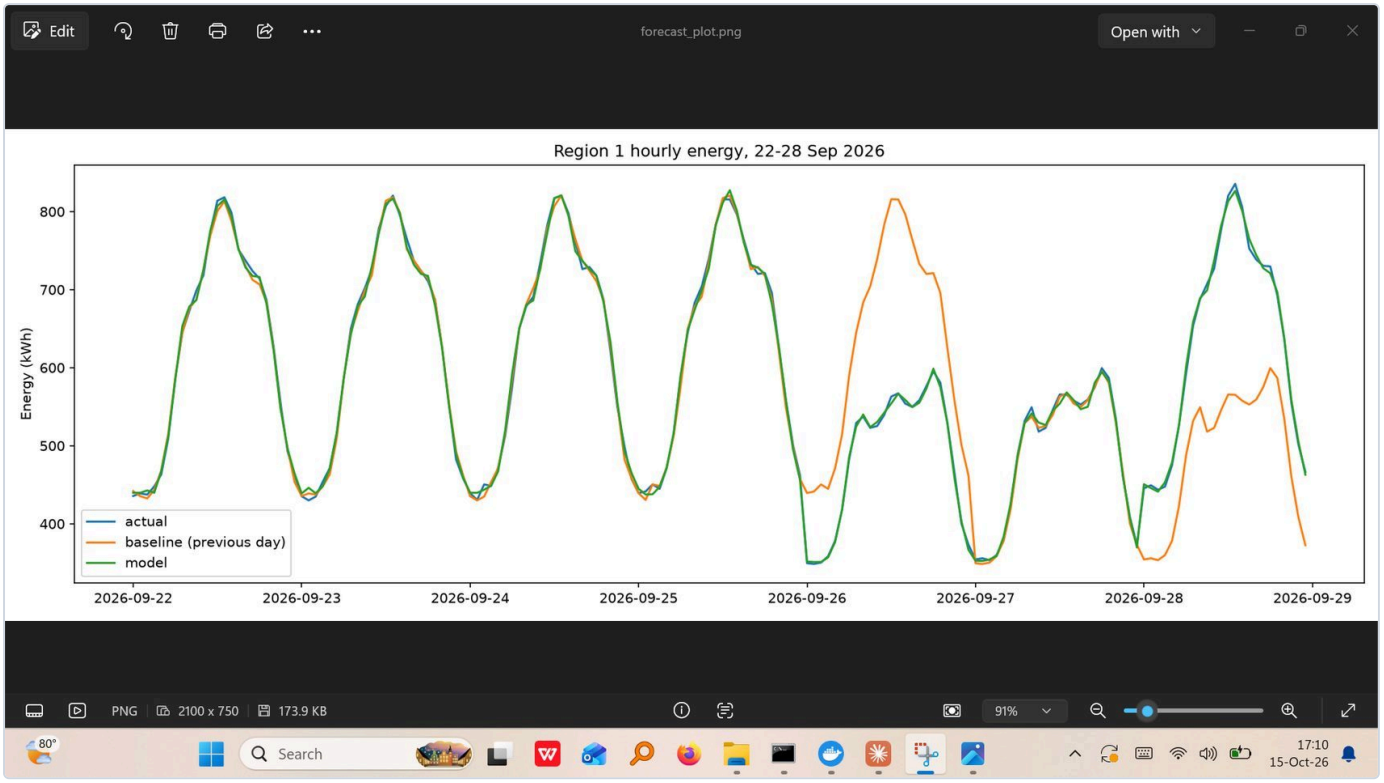
Highest actual test-hour consumption in each region (ties broken by earliest time)

Region	Peak hour	Actual (kWh)	Baseline prediction	Baseline |error|	Model prediction	Model |error|	Model error / actual
Region 1	2026-09-28 13:00	835.61	565.27	270.34	826.49	9.12	1.1%
Region 2	2026-09-28 13:00	780.13	546.91	233.21	773.66	6.46	0.8%
Region 3	2026-09-28 13:00	783.00	554.69	228.32	790.13	7.13	0.9%
Region 4	2026-09-28 18:00	659.90	561.41	98.50	655.94	3.96	0.6%
Region 5	2026-09-28 13:00	828.23	577.36	250.87	825.89	2.35	0.3%
Region 6	2026-09-28 13:00	749.11	504.41	244.70	730.91	18.20	2.4%
Region 7	2026-09-25 13:00	751.05	736.64	14.41	743.93	7.12	0.9%
Region 8	2026-09-25 13:00	1,016.55	1,010.53	6.02	1,012.98	3.58	0.4%
Region 9	2026-09-23 13:00	808.96	796.89	12.07	804.57	4.39	0.5%
Region 10	2026-09-28 13:00	743.03	502.33	240.71	734.49	8.55	1.2%

```
===== 8.4 EVALUATION (error = predicted - actual) =====
OVERALL baseline MAE/RMSE: 42.708 79.403
OVERALL model    MAE/RMSE: 4.538 5.773
   region  base_MAE  base_RMSE  model_MAE  model_RMSE
 Region 1     44.87      83.70       4.66        5.86
 Region 2     41.63      76.81       4.50        5.63
 Region 3     42.75      77.61       4.28        5.59
 Region 4     34.86      64.23       4.03        5.11
 Region 5     44.40      81.08       5.21        6.59
 Region 6     39.43      74.77       3.67        4.66
 Region 7     39.69      73.94       4.40        5.70
 Region 8     55.97     102.06       5.74        7.11
 Region 9     43.77      80.18       4.68        5.80
Region 10     39.72      74.28       4.21        5.29
   region           peak_hour  actual  base_pred  base_abs_err  model_pred  model_abs_err
 Region 1 2026-09-28 13:00:00  835.61     565.27        270.34      826.49           9.12
 Region 2 2026-09-28 13:00:00  780.13     546.91        233.21      773.66           6.46
 Region 3 2026-09-28 13:00:00  783.00     554.69        228.32      790.13           7.13
 Region 4 2026-09-28 18:00:00  659.90     561.41         98.50      655.94           3.96
 Region 5 2026-09-28 13:00:00  828.23     577.36        250.87      825.89           2.35
 Region 6 2026-09-28 13:00:00  749.11     504.41        244.70      730.91          18.20
 Region 7 2026-09-25 13:00:00  751.05     736.64         14.41      743.93           7.12
 Region 8 2026-09-25 13:00:00 1016.55    1010.53          6.02     1012.98           3.58
 Region 9 2026-09-23 13:00:00  808.96     796.89         12.07      804.57           4.39
Region 10 2026-09-28 13:00:00  743.03     502.33        240.71      734.49           8.55
saved forecast_plot.png and test_predictions.csv

C:\auca_project>
```

8.4 – executed metric comparison (overall and per region) and peak-hour error results



8.4 – actual, previous-day baseline and model hourly consumption for Region 1, 22–28 September

> **Observation.** The model improves on the baseline in every region: overall MAE 4.538 kWh against 42.708 kWh (89.4% lower) and RMSE 5.773 kWh against 79.403 kWh (92.7% lower); regional MAE reductions range from 88.3% to 90.7%. The baseline is nearly as good as the model on weekdays but fails when the day type changes: in the Region 1 plot it repeats Friday on Saturday 26 September and Sunday on Monday 28 September. At the peak hours the model is within 2.35–18.20 kWh (0.3–2.4% of the actual value) and the baseline within 6.02–270.34 kWh (up to 32%).

> **Explanation.** The baseline copies yesterday and so cannot know that weekends use about 21% less energy; the model has weekend flags, the same hour a week earlier (lag168) and the previous-day mean, so it learns the weekly pattern. The one large model error (Region 6, 18.20 kWh at its 13:00 peak on 28 September) is 2.4% of the actual value.

Is it useful for utility planning? Yes for day-ahead planning: a typical hourly error of about 4.5 kWh on regional hourly totals of roughly 300–1,000 kWh (roughly 1% or less) and a worst peak-hour error of 18 kWh (2.4%) are small compared with the day-to-day swings that the baseline gets wrong. The peak hours matter most for capacity: the model's peak errors are small, but they are measured on only seven test days, so the result should be re-checked on a longer period, including holidays.

Limitations. (1) The forecast is offline and uses finalised hourly totals. About 1% of readings are missing; the hourly totals are therefore slightly under-stated, consistently in training and testing, but a region with a sudden reporting outage would look like a drop in consumption and bias the next forecast. (2) In live use, about 2% of readings arrive 30–180 minutes late. At the 00:00 forecast origin the last hours of the previous day are not yet complete, so lag24 for the evening hours would be slightly too low (the effect on a full day is about 0.1%, see 5.3), and the 24-hour lag features must be built from the data that have arrived or refreshed later. (3) Only seven test days and one month of history were available, so seasonal effects were not tested.

9 Submit the report and code

9.1 Organize the report

This report follows the subsection order 1.1 to 9.3. Every required screenshot is embedded beside its subsection with the command, result, taskbar and clock; where a later subsection asks only for interpretation (3.3, 4.4, 5.5, 6.2 explanations, 7.4 judgement, 8.4 interpretation) it refers to the measurements and screenshots shown earlier.

9.2 Make the work reproducible

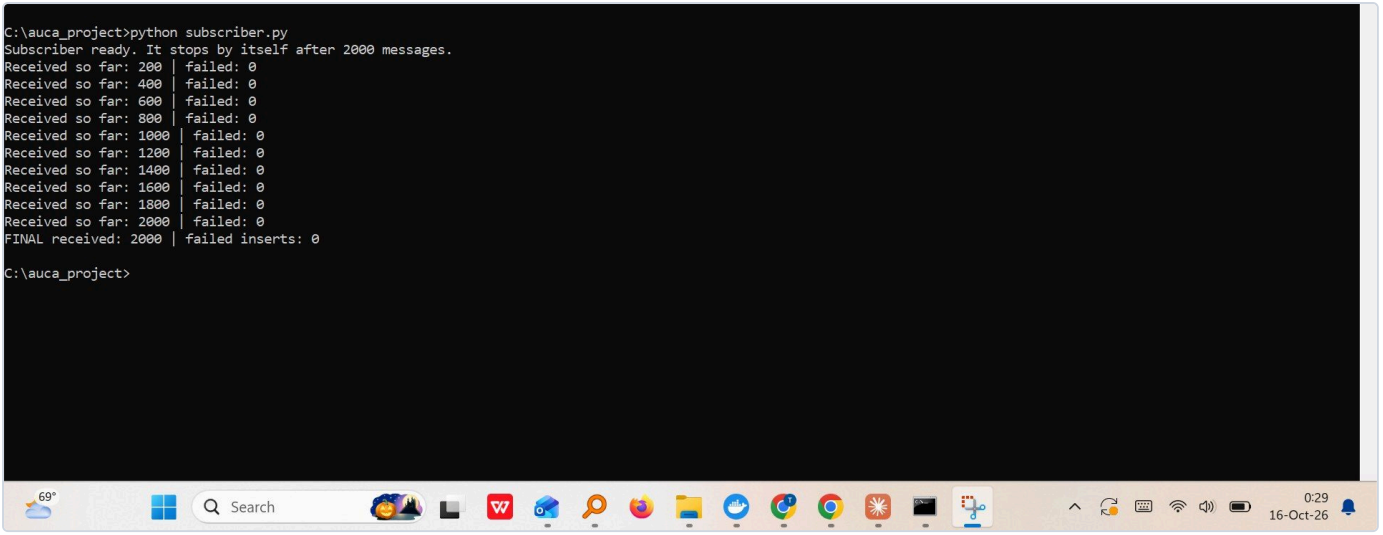
GitHub repository: https://github.com/twahirwajoshua1/auca-smart-grid (the submitted ZIP contains the same final files). The unchanged generator `smart_meter_simulator.py` is included byte for byte (SHA-256 `6b16d6bb3191e528844d0ec75656096d8bfb2e92c99f83e2b75d586bf21cfff8`, identical to the copy kept with the project files).

Required item	Files in the repository
Unchanged generator	smart_meter_simulator.py
Capture and ingestion programs	publisher.py, subscriber.py, generate_csv.py, load_csv.py, load_meters.py, load_compress.py
SQL files	create_tables.sql, clean_duplicates.sql, create_hypertables.sql, create_compressed.sql, create_caggs.sql, analysis_queries.sql
Analysis and model code	chart_5_1.py, anomaly_detect.py, fc1.py, fc2.py (forecast outputs: test_predictions.csv; anomaly output: anomaly_flags.csv)
Recorded benchmark timings	benchmark_chunks.py, benchmark_runs.csv, benchmark_summary.csv, benchmark_compressed.py, benchmark_compressed_runs.csv, benchmark_compressed_summary.csv, compress_timings.csv, dashboard_timings.csv, dashboard_render_runs.csv, dashboard_render_summary.csv
Validation comparison	validation_comparison.csv (Ridge configurations C1–C3; also in Section 8.3)
Dashboard queries and configuration	dash_app.py, dq.py, dash_check.py, dash_render_timing.py (browser timing)
Environment details	ENVIRONMENT.txt (and Section 1.1 of this report)
README with execution order and commands	README.md (12 numbered steps; passwords and connection secrets are the placeholder `YOUR_PASSWORD`)

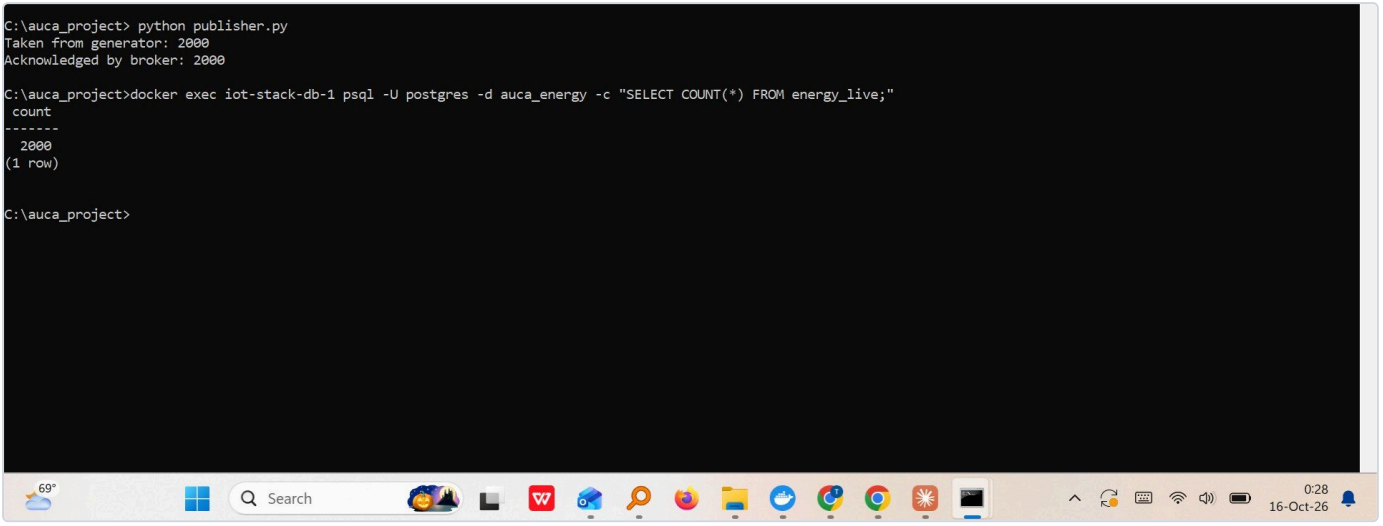
The full generated dataset is kept local (about 1.2 GB); README step 5 regenerates it with `generate_csv.py` and loads it with `load_csv.py`.

README pilot run (9.2.4). The pilot instructions in the README were followed on 16 October 2026 at 00:28–00:29 CAT: `energy_live` was emptied with TRUNCATE, `subscriber.py` was started first, then `publisher.py`, and the stored rows were counted. All four counts from 1.3 were 2,000: taken from the generator 2,000; acknowledged by the broker 2,000;

received by the subscriber 2,000 (0 failed inserts); rows in `energy_live` 2,000. (An earlier repeat at 23:48 on 15 October stored 0 rows; its cause was not established. The run shown here is the repeat that followed.)



9.2 – README pilot, subscriber window (16 October 2026, 00:29): FINAL received 2000, failed inserts 0



9.2 – README pilot, publisher window (00:28): taken from the generator 2000, acknowledged by the broker 2000; SELECT COUNT(*) FROM energy_live returns 2000

9.3 Document contributions

Member	Student ID	Sections and work this member is responsible for explaining and presenting
TWAHIRWA Joshua	101396	Environment and Docker set-up; database structures and meter load (Section 1); pilot publisher and subscriber and the fault corrections (1.3); full-stream capture and loading (2.1); repository, README and report assembly (Section 9).
UBANJE Albert	101402	Data-quality and cleaning queries (2.2–2.3); chunk-interval and compression experiments (Sections 3 and 4) and the storage analysis; continuous aggregates and the freshness test (Section 6).
MUTONI Denyse	101207	Demand, regional and reporting-quality analysis and the anomaly rule (Section 5); dashboards, their verification and timing (Section 7); forecast baseline, Ridge model and evaluation (Section 8); management recommendation (5.5).

> **How to read this table.** The group divided the report by section so that each member studies, and can explain and present, the sections listed against their name. All of the code, results and the report are shared by the three members, and any member can be asked to explain any part of the work (9.3.2).